CS2040S Data Structures and Algorithms

Ryan Joo

AY25/26 Sem 2

This course introduces students to the design and implementation of fundamental data structures and algorithms. The course covers basic data structures (linked lists, stacks, queues, hash tables, binary heaps, trees, and graphs), searching and sorting algorithms, and basic analysis of algorithms.

Prerequisite

If undertaking an Undergraduate Degree THEN(must have completed 1 of CS1010 / CS1010A / CS1010E / CS1010HS / CS1010J / CS1010S / CS1010X / CS1101S / UTC2851 at a grade of at least D AND must have completed 1 of CS1231 / CS1231S / MA1100 / MA1100T at a grade of at least D)

Preclusion

If undertaking an Undergraduate Degree THEN(must not have completed 1 of CS1020 / CS1020E / CS2010 / CS2020 / CS2040 / CS2040C / CS2040DE / CS2040HS at a grade of at least D)

Contents

1 Algorithm Analysis

In computer science, we generally look at how something performs at large scale. In particular, we care about how an algorithm scales as the problem size tends to infinity. This is called **asymptotic behaviour**.

1.1 Asymptotic Notation

Definition 1.1. For a given function $g(n)$, define

$$\Theta(g(n)) \coloneqq \left\{ f(n) : \exists c_1, c_2, n_0 > 1 \text{ such that } c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0 \right\}.$$

If $f(n) \in \Theta(g(n))$, we say that $g(n)$ is an **asymptotically tight bound** for $f(n)$.

Notation. We abuse notation by writing "$f(n) = \Theta(g(n))$" instead of "$f(n) \in \Theta(g(n))$".

Example. Show that $\frac{1}{2}n^2 - 3n = \Theta(n^2)$.

Solution. We need to determine positive constants c_1, c_2 and n_0 such that

$$c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2$$

for all $n \geq n_0$. Dividing by n^2 yields

$$c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2.$$

We can make the right-hand inequality hold for any value of $n \geq 1$ by choosing $c_2 \geq \frac{1}{2}$. Likewise, we can make the left-hand inequality hold for any value of $n \geq 7$ by choosing any constant $c_1 \leq \frac{1}{14}$. Thus choose $c_1 = \frac{1}{14}$, $c_2 = \frac{1}{2}$ and $n_0 = 7$. □

Example. Show that $6n^3 \neq \Theta(n^2)$.

Solution. Suppose, towards a contradiction, that $6n^3 = \Theta(n^2)$. Then there exists c_2 and n_0 such that $6n^3 \leq c_2 n^2$ for all $n \geq n_0$. But then dividing by n^2 yields $n \leq c_2/6$, which cannot possibly hold for arbitrarily large n, since c_2 is a constant. □

Definition 1.2. Given a function $g(n)$, define

$$O(g(n)) \coloneqq \left\{ f(n) : \exists c, n_0 > 1 \text{ such that } f(n) \leq c g(n) \text{ for all } n \geq n_0 \right\}.$$

If $f(n) \in O(g(n))$, we say that $g(n)$ is an **asymptotic upper bound** for $f(n)$.

Notation. We abuse notation and write "$f(n) = O(g(n))$" instead of "$f(n) \in O(g(n))$".

Note that $f(n) = \Theta(g(n))$ implies $f(n) = O(g(n))$, since Θ-notation is a stronger notion than O-notation. Written set-theoretically, we have $\Theta(g(n)) \subseteq O(g(n))$.

Remark. $O(\cdot)$ only expresses an upper bound, but does not necessarily provide a *tight* upper bound. For example, $T(n) = 7n^2 + 5$ is $O(n^2)$, but also $O(n^3)$.

Example. Show that $n^2 + 5n + 10 \in O(n^2)$.

Solution. Pick $n_0 = 1$ and $c = 16$. Then for all $n \geq 1$,

$$n^2 + 5n + 10 \leq n^2 + 5n^2 + 10n^2 = 16n^2.$$

□

Definition 1.3. Given a function $g(n)$, define

$$\Omega(g(n)) \coloneqq \left\{ f(n) : \exists c, n_0 > 1 \text{ such that } f(n) \geq c g(n) \text{ for all } n \geq n_0 \right\}.$$

If $f(n) \in \Omega(g(n))$, we say that $g(n)$ is an **asymptotically lower bound** for $f(n)$.

Remark. Similarly, $\Omega(\cdot)$ only represents a lower bound, which is not necessarily a tight bound.

Notation. Again, we abuse notation and write "$f(n) = \Omega(g(n))$" instead of "$f(n) \in \Omega(g(n))$".

Theorem 1.4. *For any two functions $f(n)$ and $g(n)$,*

$$f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \quad \text{and} \quad f(n) = \Omega(g(n)).$$

Proof. This follows from the definitions of asymptotic notations. □

The asymptotic upper bound provided by O-notation may or may not be asymptotically tight. We use o-notation to denote an upper bound that is not asymptotically tight.

Definition 1.5. Given a function $g(n)$, define

$$o(g(n)) \coloneqq \left\{ f(n) : \forall c > 0, \exists n_0 > 0 \text{ such that } f(n) < cg(n) \text{ for all } n \geq n_0 \right\}.$$

If $f(n) = o(g(n))$, we say that $f(n)$ is **asymptotically smaller** than $g(n)$.

Example. $2n = o(n^2)$, but $2n^2 \neq o(n^2)$.

Intuitively, the function $f(n)$ becomes insignificant relative to $g(n)$ as n approaches infinity:

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0.$$

By analogy, ω-notation is to Ω-notation as o-notation is to O-notation. We use ω-notation to denote a lower bound that is not asymptotically tight.

Definition 1.6. Given a function $g(n)$, define

$$\omega(g(n)) \coloneqq \left\{ f(n) : \forall c > 0, \exists n_0 > 0 \text{ such that } f(n) > cg(n) \text{ for all } n \geq n_0 \right\}.$$

If $f(n) = \omega(g(n))$, we say that $f(n)$ is **asymptotically larger** than $g(n)$.

Example. $n^2/2 = \omega(n)$, but $n^2/2 \neq \omega(n^2)$.

The relation $f(n) = \omega(g(n))$ implies that

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty.$$

That is, $f(n)$ becomes arbitrarily large relative to $g(n)$ as n approaches infinity.

1.2 Comparing Functions

Lemma 1.7 (Transitivity)**.**

(i) If $f = O(g)$ and $g = O(h)$, then $f = O(h)$.

(ii) If $f = \Omega(g)$ and $g = \Omega(h)$, then $f = \Omega(h)$.

(iii) If $f = \Theta(g)$ and $g = \Theta(h)$, then $f = \Theta(h)$.

(iv) If $f = o(g)$ and $g = o(h)$, then $f = o(h)$.

(v) If $f = \omega(g)$ and $g = \omega(h)$, then $f = \omega(h)$.

Proof.

(i) Apply (ii) and (iii).

(ii) By assumption, there exist $c_1, c_2 > 0$ and n_1, n_2 such that

$$f(n) \leq c_1 g(n) \quad \text{for all } n \geq n_1,$$
$$g(n) \leq c_2 h(n) \quad \text{for all } n \geq n_2.$$

Take $n_0 \coloneqq \max\{n_1, n_2\}$. Then for all $n \geq n_0$,

$$f(n) \leq c_1 g(n) \leq c_1 c_2 h(n).$$

(iii) Similar to (ii).

(iv) Let $c > 0$ be given. There exist $n_1, n_2 > 0$ such that

$$f(n) < \sqrt{c} g(n) \quad \text{for all } n \geq n_1,$$
$$g(n) < \sqrt{c} h(n) \quad \text{for all } n \geq n_2.$$

Take $n_0 \coloneqq \max\{n_1, n_2\}$. Then for all $n \geq n_0$, we have $f(n) < ch(n)$.

(v) Similar to (iv).

□

Lemma 1.8 (Reflexivity)**.**

(i) $f = \Theta(f)$.

(ii) $f = O(f)$.

(iii) $f = \Omega(f)$.

Lemma 1.9 (Symmetry)**.** $f = \Theta(g)$ *if and only if* $g = \Theta(f)$.

Lemma 1.10 (Transpose symmetry)**.**

(i) $f(n) = O(g(n))$ *if and only if* $g(n) = \Omega(f(n))$.

(ii) $f(n) = o(g(n))$ *if and only if* $g(n) = \omega(f(n))$.

Proof.

(i) $\boxed{\Rightarrow}$ By assumption, there exist $c > 0$ and $n_0 > 0$ such that $f(n) \leq cg(n)$ for all $n \geq n_0$.
Dividing by c on both sides gives $g(n) \geq \frac{1}{c} f(n)$ for all $n \geq n_0$. Thus $g(n) = \Omega(f(n))$.
$\boxed{\Leftarrow}$ Similar.

(ii)

□

Because these properties hold for asymptotic notations, we can draw an analogy between the asymptotic comparison of two functions f and g and the comparison of two real numbers a and b:

$$f(n) = O(g(n)) \quad \text{is like} \quad a \leq b,$$
$$f(n) = \Omega(g(n)) \quad \text{is like} \quad a \geq b,$$
$$f(n) = \Theta(g(n)) \quad \text{is like} \quad a = b,$$
$$f(n) = o(g(n)) \quad \text{is like} \quad a < b,$$
$$f(n) = \omega(g(n)) \quad \text{is like} \quad a > b.$$

Theorem 1.11. *If $f(n)$ and $g(n)$ are such that*

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = c \in \mathbb{R}^+$$

then $f(n) = \Theta(g(n))$.

Proof. By definition, we need to prove (i) $f(n) = O(g(n))$, and (ii) $f(n) = \Omega(g(n))$.
The given limit implies that there exists $n_0 \in \mathbb{N}$ such that $\frac{1}{2}c \leq \frac{f(n)}{g(n)} \leq 2c$ for all $n \geq n_0$.

(i) $f(n) \leq 2cg(n)$ for all $n \geq n_0$.

(ii) $\frac{c}{2}g(n) \leq f(n)$ for all $n \geq n_0$.

□

Lemma 1.12 (Additivity)**.** *If $f = O(h)$ and $g = O(h)$, then $f + g = O(h)$.*

Proof. By hypothesis, there exist $c_1, c_2 > 0$ and $n_1, n_2 > 0$ such that

$$f(n) \leq c_1 h(n) \quad \text{for all } n \geq n_1,$$
$$g(n) \leq c_2 h(n) \quad \text{for all } n \geq n_2.$$

Take $n_0 = \max\{n_1, n_2\}$. Then for all $n \geq n_0$,

$$f(n) + g(n) \leq (c_1 + c_2)h(n).$$

□

Lemma 1.13. *If $f = O(g)$, then $cf = O(g)$ for any constant c.*

Proof. Since $f = O(g)$, there exists $d > 0$ and $n_0 > 0$ such that

$$f(n) \leq dg(n) \quad \text{for all } n \geq n_0.$$

Then $cf(n) \leq cdg(n)$. □

Lemma 1.14. *If $g = O(f)$, then $f + g \in O(f)$.*

Proof. By hypothesis, there exist $c > 0$ and $n_0 > 0$ such that

$$g(n) \leq cf(n) \quad \text{for all } n \geq n_0.$$

Thus for all $n \geq n_0$,

$$f(n) + g(n) \leq f(n) + cf(n) = (c + 1)f(n).$$

Hence $f + g = O(f)$. □

Lemma 1.15. *If $f = O(a)$ and $g = O(b)$, then $fg = O(ab)$.*

Theorem 1.16. *For any polynomial $p(n) = \sum_{i=0}^{d} a_i n^i$, we have $p(n) = \Theta(n^d)$.*

Proof. For each $i = 0, 1, \ldots, d$, let $F_i(n) = a_i n^i$. We can find $n_i \in \mathbb{N}$ such that for all $n \geq n_i$,

$$F_i = a_i n^i \leq |a_i| n^i \leq |a_d| n^d.$$

Thus, $F_i = O(n^d)$. By additivity of O, we conclude that $F = O(n^d)$. □

The following are common complexity classes:

- $T(n) = O(1)$: Great. This means your algorithm takes only *constant* time. You can't beat this. (Example: Popping a stack.)
- $T(n) = O(\alpha(n))$: The function α is the inverse of the famous *Ackerman's function*. Ackerman's function grows insanely rapidly, and hence its inverse grows insanely slowly. How slowly? If n is less than the number of atoms in the visible universe, then $\alpha(n) \leq 5$. Thus, $\alpha(n)$ is a constant "for all practical purposes". However, formally it is not a constant. In the limit, as $n \to \infty$, $\alpha(n) \to \infty$. It just gets there extremely slowly! Believe or not, there are actually a number of data structures whose running times are $O(\alpha(n))$, but they are not $O(1)$. (Example: Disjoint set union-find.)
- $T(n) = O(\log \log n)$: Super fast! For most practical purposes, this is as fast as a constant time. (Example: Van Emde Boas trees.)
- $T(n) = O(\log n)$: Very good. This is called *logarithmic* time, and is the "gold standard" for data structures based on making binary comparisons. It is the running time of binary search and the height of a balanced binary tree. This is about the best that can be achieved for data structures based on binary trees. (Example: Binary search.)
- $T(n) = O((\log n)^k)$, where k is a constant: This is called *polylogarithmic* time. Not bad, when simple logarithmic time is not achievable. We will often write this as $O(\log^k n)$. (Example: Orthogonal range searching, that is, counting the number of points in a d-dimensional axis-parallel rectangle.)
- $T(n) = O(n^p)$, where $0 < p < 1$ is a constant: An example is $O(\sqrt{n})$. This is slower than polylogarithmic (no matter how big k is or how small p), but is still faster than linear time, which is acceptable for data structure use. (Example: Nearest neighbour searching in d-dimensional space.)
- $T(n) = O(n)$: This is called *linear* time. It is about the best that one can hope for if your algorithm has to look at all the data. (Example: Enumerating the elements of a linked list.)
- $T(n) = O(n \log n)$: This one is famous, because this is the time needed to sort a list of numbers by means of comparisons. It arises in a number of other problems as well. (Example: Sorting, of course.)
- $T(n) = O(n^2)$: *Quadratic* time. (Example: 3Sum: Given a list of n numbers (positive and negative), do any three sum to zero?)
- $T(n) = O(n^k)$, where k is a constant: This is called *polynomial* time. Practical if k is not too large. (Example: Matrix multiplication.)
- $T(n) = O(2^n), O(n^n), O(n!)$: *Exponential* time. Algorithms taking this much time are only practical for the smallest values of n (e.g., $n \leq 10$ or maybe $n \leq 20$).

Function hierarchy:

$$\log^a n < n^b < 2^{cn} < n! < n^{dn} \tag{1}$$

for any constants a, b, c, d.

Theorem 1.17 (Master theorem)**.** *Let $a > 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined by the recurrence*

$$T(n) = aT(n/b) + f(n).$$

Then $T(n)$ has the following asymptotic bounds:

(1) If $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.

(2) If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$.

(3) If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some constant $\varepsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$.

In each of the three cases, we compare the function $f(n)$ with $n^{\log_b a}$. Intuitively, the larger of the two functions determines the solution to the recurrence.

- If, as in case 1, the function $n^{\log_b a}$ is the larger, then the solution is $T(n) = \Theta(n^{\log_b a})$.
- If, as in case 3, the function $f(n)$ is the larger, then the solution is $T(n) = \Theta(f(n))$.
- If, as in case 2, the two functions are the same size, we multiply by a logarithmic factor, and the solution is $T(n) = \Theta(n^{\log_b a} \log n) = \Theta(f(n) \log n)$.

Remark. In the first case, not only must $f(n)$ be smaller than $n^{\log_b a}$, it must be *polynomially smaller*. That is, $f(n)$ must be asymptotically smaller than $n^{\log_b a}$ by a factor of n^{ε} for some constant $\varepsilon > 0$.
In the third case, not only must $f(n)$ be larger than $n^{\log_b a}$, it also must be polynomially larger and in addition satisfy the "regularity" condition that $af(n/b) \leq cf(n)$. This condition is satisfied by most of the polynomially bounded functions that we shall encounter.

Example. Consider

$$T(n) = 9T(n/3) + n.$$

For this recurrence, we have $a = 9$, $b = 3$, $f(n) = n$, and thus we have that $n^{\log_b a} = n^{\log_3 9} = \Theta(n^2)$. Since $f(n) = O(n^{\log_3 9 - \varepsilon})$, where $\varepsilon = 1$, we can apply case 1 of the master theorem and conclude that the solution is $T(n) = \Theta(n^2)$.

1.3 Time Complexity

We shall assume a generic one-processor, **random-access machine** (RAM) model of computation as our implementation technology. In the RAM model, instructions are executed one after another, with no concurrent operations.
In the **RAM model**, we will assume that the following are constant time operations, i.e., every single step that uses any of the following operations costs $O(1)$ time:

- Addition, subtraction, multiplication, division on any integer/fractional value
- Memory accesses
- Comparisons using =, <, > etc.
- Return single integer

Example. To account for the time complexity of a for/while loop, we need to consider how many iterations it has run.

Incrementing a until it reaches n

```
1: a ← 5
2: while a < n do
3:     a ← a + 10
```

The loop runs for $O(n)$ iterations. Each iteration takes $O(1)$ time. Hence the total cost of the loop is $O(n) \times O(1) = O(n)$.

Example.

Nested loop counter

```
1: counter ← 0
2: a ← 0
3: while a < n do
4:     for b = a, ..., n − 1 do
5:         counter ← counter + 1
6:     a ← a + 1
```

In total, Line 5 runs for $1 + 2 + \cdots + n = \frac{n(n+1)}{2} = O(n^2)$ iterations. Each iteration costs $O(1)$ time. Hence, the total cost is $O(n^2) \times O(1) = O(n^2)$.

If/else

```
1: function BRANCH(n)
2:     if n is even then
3:         perform operation costing O(log n)
4:     else
5:         perform operation costing O(n)
```

Example. In the worst case, the cost of an if/else is the maximum across both branches.
In this case, the worst case cost is $O(n)$.

Example. Passing something by reference costs $O(1)$.

```
1: void pass_by_ref(int[] arr) {
2:   foo(arr);
3: }
```

However, copying an array costs $O(n)$, where n is the length of the array.

```
1: void pass_by_ref(int[] arr) {
2:   int[] new_arr = new int[arr.length];
3:   for (int i = 0; i < arr.length; ++i) {
4:     new_arr[i] = arr[i];
5:   }
6:   foo(new_arr);
7: }
```

To analyse the time complexity of recursive algorithms, we use **recurrence relations**.

(1) Come up with a recurrence relation that represents the runtime of the algorithm $T(n)$.

(2) Expand the recurrence relation a few times to spot the pattern.

(3) Figure out when it stops (base case).

(4) Analyse the resulting sum.

Example.

An algorithm to compute exponentiation

```
1: function EXP(base, pow)
2:     if pow = 0 then
3:         return 1
4:     return base × EXP(base, pow − 1)
```

Let $T(b, p)$ be a function representing how many time steps Exp takes to run with input sizes b and p.
In line 4, we make the recursive call, which costs $T(b, p-1)$. Constant time operations (e.g. return statement, multiplication) take $O(1)$; we just write 1 instead of $O(1)$. Hence,

$$T(b,p) = T(b,p-1) + 1.$$

Then unroll the recurrence and try to spot the pattern:

$$\begin{aligned} T(b,p) &= T(b,p-1) + 1 \\ &= T(b,p-2) + 1 + 1 \\ &= T(b,p-3) + 1 + 1 + 1. \end{aligned}$$

The program stops when it reaches the base case. Thus, we stop unrolling when $p = 0$.

$$\begin{aligned} T(b,p) &= T(b,0) + \overbrace{1 + \cdots + 1 + 1}^{p \text{ terms}} \\ &= 1 + p = O(p) \end{aligned}$$

since $T(b,0)$ costs $O(1)$ as it involves a return statement.

Example.

Another algorithm to compute exponentiation

```
1: function Exp(base, pow)
2:     if pow = 0 then
3:         return 1
4:     if pow is even then
5:         return Exp(base, ⌊pow/2⌋) × Exp(base, ⌊pow/2⌋)
6:     else
7:         return base × Exp(base, ⌊pow/2⌋) × Exp(base, ⌊pow/2⌋)
```

We make 2 recursive calls in either line 5 or line 7; each recursive call costs $T(b, p/2)$. Additionally, there is an $O(1)$ cost for calling functions, and the multiplication. Hence, we have

$$T(b,p) = 2T\left(b, \frac{p}{2}\right) + 1.$$

Unrolling the recurrence relation,

$$\begin{aligned} T(b,p) &= 2T\left(b, \frac{p}{2}\right) + 1 \\ &= 4T\left(b, \frac{p}{4}\right) + 2 + 1 \\ &= 8T\left(b, \frac{p}{8}\right) + 4 + 2 + 1 \\ &= 2^k T\left(b, \frac{p}{2^k}\right) + 2^{k-1} + \cdots + 4 + 2 + 1. \end{aligned}$$

The expansion stops at the base case, when $p/2^k$ equals 0. (Note: We are using flooring division.) Thus, $k = \log_2 p + 1$. Hence,

$$\begin{aligned} T(b,p) &= 2^{\log_2 p + 1} T(b,0) + 2^{\log_2 p} + \cdots + 4 + 2 + 1 \\ &= 2^{\log_2 p + 1}(1) + 2^{\log_2 p} + \cdots + 4 + 2 + 1 \\ &= \frac{2^{\log_2 p + 2} - 1}{2 - 1} = 2^{\log_2 p + 2} - 1 = 4p - 1 = O(p). \end{aligned}$$

1.4 Space complexity

Space complexity is determined by considering the **maximum** used space at any point in time. (This is because the Java garbage collector already managed our memory for us.)

Example.

```
1: int logN = Math.log(n);
2: for (int i = 0; i < n; ++i){
3:     int[] newArray = new int[logN];
4: }
```

The total amount allocated is $O(n \log n)$, but if we assume that each allocation is *garbage collected* before the subsequent one, then this program uses $O(\log n)$ space.

Example (Minimum finding)**.**

```
1: int min_so_far = Integer.MIN_VALUE;
2: for (int i = 0; i < n; ++i) {
3:   min_so_far = (min_so_far < arr[i]) ? min_so_far : arr[i];
4: }
```

The array is given as input, so it is not counted. Our algorithm here allocates $O(1)$ **extra** space (to store the variables min_so_far and i), so it takes $O(1)$ space.

2 Searching

2.1 Invariants

Definition 2.1. An **invariant** is a property about the algorithm that does not change, throughout all of its iterations.

We use loop invariants to show the correctness of an algorithm. We must show three things about a loop invariant:

(i) **Initialisation**: It is true prior to the first iteration of the loop.

(ii) **Maintenance**: If it is true before an iteration of the loop, it remains true before the next iteration.

(iii) **Termination**: When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct.

2.2 Binary Search

Input: An array A of n integers sorted in increasing order, and an integer k.
Task: If the integer k appears in the array, output the index at which it appears in the array. Otherwise, output -1.
Algorithm:

(1) Set indices left = 1 and right = n.

The indices track the left and right ends (both inclusive) of the sub-array we are searching.

(2) As long as left < right, find the middle index of this sub-array.

(3) Compare the middle element $A[\text{mid}]$ with k.

- If $k = A[\text{mid}]$, then we have found k. Return mid.
- If $k > A[\text{mid}]$, then k lies in the right half. Update left to this middle position.
- If $k < A[\text{mid}]$, then k lies in the left half. Update right to mid $-$ 1.

Algorithm 2.2 Binary Search

```
1:  procedure SEARCH(A, k)
2:      left ← 1
3:      right ← A.length
4:      while left ≤ right do
5:          mid ← ⌊(left + right)/2⌋
6:          if k = A[mid] then
7:              return mid
8:          else if k > A[mid] then
9:              left ← mid + 1
10:         else
11:             right ← mid − 1
12:     return −1
```

Binary search can also be written recursively:

Algorithm 2.3 Binary Search (Recursive)

```
1:  procedure SEARCH(A, k, left, right)
2:      if left > right then
3:          return −1
4:      mid ← ⌊(left + right)/2⌋
5:      if k = A[mid] then
6:          return mid
7:      else if k > A[mid] then
8:          return SEARCH(A, k, mid + 1, right)
9:      else
10:         return SEARCH(A, k, left, mid − 1)
```

Invariant: At the start of every iteration of the while-loop, if k exists in the array A, then it lies in the sub-array $A[\text{left}, \text{right}]$.

Theorem 2.4. *The Binary Search algorithm is correct.*

Proof.

(i) **Initialisation**: Initially, left = 1 and right = n. If k exists in the array, then it must lie in the whole array $A[1, n] = A[\text{left}, \text{right}]$. Hence, the invariant holds.

(ii) **Maintenance**: Assume the invariant holds at the start of an iteration. Let mid = $\lfloor(\text{left} + \text{right})/2\rfloor$.

- If $k = A[\text{mid}]$, the algorithm returns mid, which is correct.
- If $k > A[\text{mid}]$, then since A is sorted in increasing order,

$$A[i] \leq A[\text{mid}] < k \quad \text{for all } i \leq \text{mid}.$$

  Hence k cannot occur at any index ≤ mid. If k exists, it must lie in $A[\text{mid} + 1, \text{right}]$. The algorithm sets left = mid + 1, so the invariant is preserved.
- If $k < A[\text{mid}]$, then for all $i \geq \text{mid}$,

$$A[i] \geq A[\text{mid}] > k.$$

  Hence k cannot occur at any index ≥ mid. If k exists, it must lie in $A[\text{left}, \text{mid} - 1]$. The algorithm sets right = mid − 1, so the invariant is preserved.

(iii) **Termination**: The loop terminates when left > right. At that point the search interval $A[\text{left}, \text{right}]$ is empty.

By the invariant, if k existed in the array, it would lie in this interval. Since the interval is empty, k does not exist in A. Hence returning −1 is correct.

□

Theorem 2.5. *The runtime of Binary Search is $O(\log n)$.*

Proof. Let $T(n)$ denote the runtime on an array of length n. Then $T(n)$ has the recurrence relation

$$T(n) = T\left(\frac{n}{2}\right) + O(1).$$

Expanding,

$$\begin{aligned} T(n) &= T\left(\frac{n}{2}\right) + 1 \\ &= T\left(\frac{n}{4}\right) + 2 \\ &= \cdots = T\left(\frac{n}{2^k}\right) + k. \end{aligned}$$

When $k = \log n$, we arrive at the base case $T(1)$:

$$T(n) = T(1) + \log n = O(1) + \log n = O(\log n).$$

□

Example. Given an array of n distinct integers. A *peak cell* is a cell such that its value is larger than its neighbours on both sides. For the leftmost and rightmost element, it only needs to be larger than its only neighbour.
Goal: Find a peak cell.

Solution. Binary search:

(1) Check the middle element. If it is a peak element, return it.

(2) If not, at least one neighbour has to be larger than it. Recurse on the side of the larger neighbour.

□

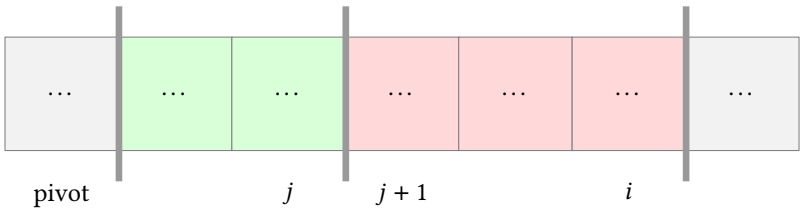
2.3 Partitioning

Input: An array A of n integers
Task: Permute the elements of the array so that $A[1]$ is eventually placed at some index i such that:

- $A[1..i-1]$ only contains elements $< A[i]$;
- $A[i+1..n]$ only contains elements $\geq A[i]$.

We call $A[1]$ the **pivot element**.
Invariant: After the i-th iteration, there exists an index j such that

- all elements in $A[2..j]$ are $< A[1]$;
- all elements in $A[j+1..i]$ are $\geq A[1]$.



Algorithm:

(1) Initially set indices $j = 1$ and $i = 1$.
They mark the boundaries of the sub-arrays $A[2..j]$ and $A[j+1..i]$.

(2) Decide whether $A[i+1]$ belongs in the left or right sub-array.

- If $A[i+1] < A[1]$, then $A[i+1]$ belongs in the left sub-array. Swap $A[i+1]$ with $A[j+1]$. Then increment both i and j.
- If $A[i+1] \geq A[1]$, then $A[i+1]$ belongs in the right sub-array. Simply increment i.

Algorithm 2.6 Partitioning

```
1:  procedure PARTITION(A, n)
2:      i ← 1
3:      j ← 1
4:      while i < n do                          ▷ Compare arr[i + 1] against pivot arr[1]
5:          if arr[i + 1] < arr[1] then
6:              Swap A[i + 1] and A[j + 1]
7:              i ← i + 1
8:              j ← j + 1
9:          else
10:             i ← i + 1
11:     Swap A[1] and A[j]                      ▷ Place pivot in final position
```

Theorem 2.7. *The Partitioning algorithm is correct.*

Proof.

(i) **Initialisation**: Initially, $i = 1$ and $j = 1$. Hence, the sub-arrays $A[2..j]$ and $A[j+1..i]$ are empty, so the invariant holds vacuously.

(ii) **Maintenance**: Assume the invariant holds after the i-th iteration. Then

$$A[2..j] < A[1] \quad \text{and} \quad A[j+1..i] \geq A[1].$$

We shall show that the invariant holds after the $(i+1)$-th iteration.

- If $A[i+1] < A[1]$, the algorithm swaps $A[i+1]$ with $A[j+1]$, then increments both i and j. After the swap, the element placed at position $j+1$ is $< A[1]$, so the region of elements less than the pivot expands to $A[2..j]$. The elements in $A[j+1..i]$ remain $\geq A[1]$. Hence the invariant is preserved.
- If $A[i+1] \geq A[1]$, the algorithm increments i only. Thus $A[i+1]$ is added to the region $A[j+1..i]$ of elements $\geq A[1]$. The invariant is preserved.

(iii) **Termination**: The loop terminates when $i = n$. At this point,

$$A[2..j] < A[1] \quad \text{and} \quad A[j+1..n] \geq A[1].$$

The algorithm swaps $A[1]$ with $A[j]$. Hence, the pivot is placed at index j, and the array satisfies

$$A[1..j-1] < A[j] \quad \text{and} \quad A[j+1..n] \geq A[j].$$

□

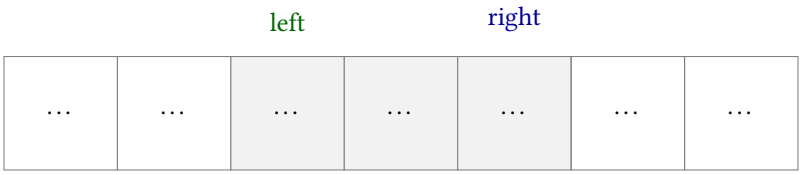
The runtime is $O(n)$, since the algorithm has n iterations, and the work done at each iteration is $O(1)$.

2.4 Quick select

Input: An array A of n integers, and an integer r $(1 \leq r \leq n)$.
Task: Output the r-th smallest integer.
Invariant:

- The r-th smallest element always exists in the sub-array $A[\text{left}, \text{right}]$.

- Every element in $A[1..\text{left}-1]$ is $\leq$ every element in $A[\text{left}, \text{right}]$.
- Every element in $A[\text{left}, \text{right}]$ is $\leq$ every element in $A[\text{right}+1..n]$.



Algorithm:

(1) Set indices left = 1, right = n.

These mark the left and right bounds of the sub-array of interest.

(2) Run the partitioning algorithm on the sub-array $A[\text{left}, \text{right}]$ using $A[\text{left}]$ as the pivot.

Let i be the resulting index of the pivot.

(3) If $i = r$, then we have found the r-th smallest integer. Return $A[i]$.

If $r < i$, then the r-th smallest integer lies in the left half. Update right = $i - 1$, and recurse on $A[\text{left}, i-1]$.

If $r > i$, then the r-th smallest integer lies in the right half. Update left = $i + 1$ and recurse on $A[i+1, \text{right}]$.

Algorithm 2.8 Quick Select

```
1:  procedure QuickSelect(A, n, r)
2:      left ← 1
3:      right ← n
4:      while left ≤ right do
5:          i ← Partition(A, left, right)
6:          if i = r then
7:              return A[i]
8:          else if r < i then
9:              right ← i − 1
10:         else
11:             left ← i + 1
```

Theorem 2.9. *The time complexity of quick select is $O(n^2)$.*

Proof. In the worst case, partitioning only reduces the problem size by 1, i.e., the pivot element ends up on either end of the array.

$$\begin{aligned} T(n) &= O(n) + [O(n-1) + O(1)] + [O(n-2) + O(1)] + \cdots + O(1) \\ &= O(n^2). \end{aligned}$$

□

2.5 Randomised Quick Select

Algorithm:

(1) Set indices left = 0, right = n.

(2) Keep randomly selecting an element in the sub-array $A\left[\text{left}, \text{right}\right)$ to use as a pivot and run $i = \text{Partition}(A, \text{left}, \text{right})$, until the pivot placement i lies in the middle third of $A\left[\text{left}, \text{right}\right)$.

(3) If $i = k$, return $A[i]$.

If $k < i$, update right $= i$. Repeat step 2.

If $k > i$, update left $= i$. Repeat step 2.

Theorem 2.10. *The runtime of Paranoid Quick Select is $O(n)$.*

Proof. In step 3, in either case, the sub-array we recurse on is within $n/3$ and $2n/3$ in size. In the worst case, we recurse on a sub-array of size $2n/3$.
Let $T(n)$ denote the runtime of on an array of length n. Then

$$\begin{aligned} T(n) &= O(n) + T\left(\frac{2n}{3}\right) \\ &= O(n) + O\left(\frac{2n}{3}\right) + T\left(\frac{4n}{9}\right) \\ &= \cdots = \sum_{k=0}^{\log_{3/2} n} \left(\frac{2}{3}\right)^k n = O(n). \end{aligned}$$

□

2.6 Peak Finding

Input: An array A of n integers.
Output: An index i such that $A[i-1] \leq A[i]$ and $A[i+1] \leq A[i]$ Such an index i is called a **peak**.
We shall assume that $A[0] = A[n+1] = -\text{MAX\_INT}$.
Observation: Every finite array contains at least one peak.
Invariant: At every recursive call on a sub-array $A[\text{left}, \text{right}]$, there exists at least one peak in this interval.
Algorithm:

(1) Maintain a search interval $A[\text{left}, \text{right}]$.

(2) Let mid $= \lfloor (\text{left} + \text{right})/2 \rfloor$.

(3) Compare $A[\text{mid}]$ with its neighbours (if they exist).

- If $A[\text{mid}]$ is a peak, return mid.
- If mid $<$ right and $A[\text{mid}+1] > A[\text{mid}]$, recurse on $A[\text{mid}+1, \text{right}]$.
- Otherwise recurse on $A[\text{left}, \text{mid}-1]$.

Algorithm 2.11 Peak Finding

```
1: procedure FindPeak(A, n)
2:     if A[n/2 + 1] > A[n/2] then
3:         FindPeak(A[n/2 + 1..n], n/2)          ▷ Search for peak in right half
4:     else if A[n/2 − 1] > A[n/2] then
5:         FindPeak(A[1..n/2 − 1], n/2)          ▷ Search for peak in left half
6:     elseA[n/2] is a peak
7:         return n/2
```

Theorem 2.12. *The runtime of Peak Finding is $O(\log n)$.*

Proof. The recurrence relation that expresses the runtime is $T(n) = T(n/2) + O(1)$. □

3 Sorting

Input: An array A of n integers.

Output: Modify the array A such that $A[i] \leq A[i+1]$ for all $1 \leq i < n$, and all the original elements are still present.

Summary

Sorting algorithm	Time complexity (worst case)	Time complexity (expected)	Space complexity (worst case)	Stability
Insertion sort	$O(n^2)$	$O(n^2)$		stable
Selection sort	$O(n^2)$	$O(n^2)$		unstable
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	stable
Quick sort	$O(n^2)$	$O(n^2)$	$O(n)$	unstable
Randomised quick sort	unbounded	$O(n \log n)$	$O(\log n)$	unstable

3.1 Insertion sort

Shifting items until their rightful place.

Algorithm:

(1) For each $i \in \{2, \ldots, n\}$, exchange $A[i]$ with the entries that are smaller in $A[1], \ldots, A[i-1]$.

(2) As the index i travels from left to right, the entries to its left are in sorted order in the array.

(3) The array is fully sorted when i reaches the right end.

Algorithm 3.1 Insertion Sort

```
1: procedure InsertionSort(A)
2:     for i = 2, ..., A.length do
3:         ▷ Insert A[i] into the sorted sequence A[1 ... i − 1] ◁
4:         for j = i, ..., 2 do
5:             if A[j − 1] > A[j] then
6:                 Swap A[j] and A[j − 1]
```

Invariant: At the start of the i-th iteration, the sub-array $A[1 \ldots i-1]$ is in sorted order.

Theorem 3.2. *The Insertion Sort algorithm is correct.*

Proof.

(i) **Initialisation**: At the start of the first iteration ($i = 2$), the sub-array $A[1 \ldots i-1] = A[1]$ consists of a single element, which is trivially sorted. Hence the invariant holds initially.

(ii) **Maintenance**: During the i-th iteration, the algorithm moves $A[i]$ leftward through the sorted sub-array $A[1 \ldots i-1]$ by repeated swaps until it reaches its correct position. After this insertion, the sub-array $A[1 \ldots i]$ contains the same elements originally in $A[1 \ldots i]$, but now in sorted order.

Incrementing i for the next iteration preserves the invariant, since the new element $A[i+1]$ will again be inserted into the sorted prefix.

(iii) **Termination**: The loop terminates when $i = n + 1$ (after processing $i = n$). By the invariant, at this point the sub-array $A[1 \ldots n]$ is sorted and contains all the original elements.

Therefore, the entire array is sorted.

□

Theorem 3.3. *The runtime of Insertion Sort is $O(n^2)$.*

Proof. In the worst case, each insertion requires shifting all elements of the sorted prefix. The first iteration ($i = 2$) does at most 1 comparison and swap, the second ($i = 3$) does at most 2, and so on, up to the last iteration ($i = n$) which does at most $n - 1$.
Hence, the total number of comparisons/swaps is

$$\sum_{i=2}^{n}(i-1) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = O(n^2).$$

□

3.2 Selection sort

Getting it right the first time.

Notation. Given an array $A[1 \dots n]$, let $S[1 \dots n]$ denote the sorted version of $A[1 \dots n]$.

Invariant: At the beginning of the i-th iteration, the sub-array $A[1 \dots i-1]$ is the same as the sub-array $S[1 \dots i-1]$.
Algorithm:

(1) For each $i = 1, 2, \dots, n$, find the minimum element in the sub-array $A[i \dots n]$.

(2) Swap it with $A[i]$.

Algorithm 3.4 Selection sort

```
1: procedure SelectionSort(A)
2:     for i = 1, ..., A.length do
3:         minIdx ← find the index of the minimum element in arr[i ... n]
4:         Swap A[i] and A[minIdx]
```

Theorem 3.5. *The Selection Sort algorithm is correct.*

Proof.

(i) **Initialisation**: Before the first iteration ($i = 1$), the sub-array $A[1 \dots 0]$ is empty, which equals the empty sub-array $S[1 \dots 0]$. Hence, the loop invariant holds before the first iteration.

(ii) **Maintenance**: Assume the invariant holds at the beginning of the i-th iteration. Then $A[1 \dots i-1] = S[1 \dots i-1]$.

During iteration i, the algorithm finds the minimum element in $A[i \dots n]$ and swaps it into position $A[i]$.

Since $A[1 \dots i-1]$ already contains the $i-1$ smallest elements in sorted order, the minimum of $A[i \dots n]$ must be the i-th smallest element overall, i.e., $S[i]$.

After the swap, $A[i] = S[i]$ and so $A[1 \dots i] = S[1 \dots i]$. Thus, the invariant holds at the beginning of the $(i+1)$-th iteration.

(iii) **Termination**: The loop terminates when $i = n$. By the invariant, at the beginning of this iteration, $A[1 \dots n] = S[1 \dots n]$. Therefore, the entire array A is sorted.

□

Theorem 3.6. *The runtime of Selection Sort is $O(n^2)$.*

Proof. The `find_min_position` operation involves iterating from index i to n, taking $n - i + 1$ iterations. The swap operation takes $O(1)$ time. The outer loop has n iterations ($i = 1$ to n). Hence, the total number of iterations is

$$\sum_{i=1}^{n}(n - i + 1) = n + (n - 1) + \cdots + 1 = \frac{n(n+1)}{2} = O(n^2).$$

□

3.3 Merge sort

Many useful algorithms are **recursive** in structure: to solve a given problem, they call themselves recursively one or more times to deal with closely related subproblems. These algorithms typically follow a **divide-and-conquer** approach:

(1) **Divide** the problem into a number of sub-problems that are smaller instances of the same problem.

(2) **Conquer** the sub-problems by solving them recursively. If the subproblem sizes are small enough, however, just solve the sub-problems in a straightforward manner.

(3) **Combine** the solutions to the sub-problems into the solution for the original problem.

Merge sort adopts the divide-and-conquer approach.

(1) Divide the array into two halves.

(2) Sort the two sub-arrays recursively using Merge Sort.

(3) Merge the two sorted sub-arrays to produce the sorted answer.

Algorithm 3.7 Merge Sort

```
1: procedure MergeSort(A, left, right)
2:     if left + 1 = right then
3:         return                                   ▷ Empty array, nothing to do
4:     mid ← ⌊(left + right)/2⌋
5:     MergeSort(arr, left, mid)                    ▷ Recurse on left sub-array
6:     MergeSort(arr, mid, right)                   ▷ Recurse on right sub-array
7:     output_arr ← new array of size (right − left)   ▷ Create an extra array to merge into
8:     Merge(arr, left, mid, right, output_arr)     ▷ Run the merge step
9:     Copy output_arr into arr[left ... right − 1]
```

The key operation of Merge Sort is the merging of two sorted sequences.

Theorem 3.8. *The runtime of Merge Sort is $O(n \log n)$.*

Proof. Let $T(n)$ denote the running time on an array of length n. Merge sort on just one element takes constant time. When we have $n > 1$ elements, we break down the running time as follows:

- Divide: The divide step just computes the middle of the subarray, which takes constant time. Thus, $D(n) = O(1)$.
- Conquer: We recursively solve two subproblems, each of size $n/2$, which contributes $2T(n/2)$ to the running time.
- Combine: The Merge procedure on an n-element subarray takes time $O(n)$, and so $C(n) = O(n)$.

At the current level, the total work done $D(n) + C(n) = O(n)$. Hence,

$$T(n) = \begin{cases} O(1) & \text{if } n = 1, \\ 2T(n/2) + O(n) & \text{if } n > 1. \end{cases}$$

Unrolling the recurrence relation,

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n \\ &= 4T\left(\frac{n}{4}\right) + n + n \\ &= 8T\left(\frac{n}{8}\right) + n + n + n \\ &= \cdots = 2^k T\left(\frac{n}{2^k}\right) + k \cdot n. \end{aligned}$$

The recurrence relation stops when we reach the base case $T(1)$, so $k = \log_2 n$. At $T(n/n) = T(1)$, we do $O(1)$ work on a constant problem size. Hence,

$$\begin{aligned} T(n) &= 2^{\log_2 n} T\left(\frac{n}{2^{\log_2 n}}\right) + n \log_2 n \\ &= nT\left(\frac{n}{n}\right) + n \log_2 n \\ &= nO(1) + n \log_2 n = O(n \log n). \end{aligned}$$

□

Theorem 3.9. *The space complexity of Merge Sort is $O(n)$.*

Proof. The algorithm requires an auxiliary array of size $O(n)$ when merging the two sorted sub-arrays.
Since there are $O(\log n)$ levels of recursion, the recursion stack uses $O(\log n)$ additional space (e.g., for pointers and indices). □

Remark. When we recurse, we are not actually creating sub-arrays. We are just letting the sub-recursion alter our current array.

Example (Peak sorting)**.** You are given as input an array $A[1 \ldots n]$ of n distinct integers. The array is guaranteed to be in such a way that the integers are arranged in a "$\wedge$" shape, i.e., there exists an index $1 < i < n$ for which:

(1) $\forall 1 \leq j < i (A[j] < A[i])$

(2) $\forall i < j \leq n (A[j] > A[i])$

An example of such an array is as follows:

$$[1, 5, 7, 10, 11, 12, 20, 9, 8, 3, 2]$$

(a) Describe an algorithm that sorts these types of arrays as efficiently (in terms of worst-case asymptotic running time) as possible.

(b) Give the tightest possible running time bound of your algorithm.

Solution. The algorithm is as follows:

(1) Find the index j that is larger than its neighbours.

This can be done with a simple loop to check each element.

(2) Copy out the first j elements into an array.

Copy out the remaining $n - j$ elements into another array, and then reverse it.

(3) Run the merge step on the two arrays.

The runtime is $O(n)$. □

3.4 Quick sort

Algorithm:

(1) Partition the array

(2) Sort the two sub-arrays recursively using Quick Sort.

(3) Recombine the arrays.

Algorithm 3.10 Quick Sort

```
1: procedure QuickSort(arr, left, right)        ▷ Runs quick sort on the sub-array [left, right)
2:     if left + 1 = right then
3:         return                                ▷ Empty array, nothing to do
4:     pivot_idx ← Partition(arr, left)          ▷ Partition the array using arr[left] as pivot
5:     QuickSort(arr, left, pivot_idx)
6:     QuickSort(arr, pivot_idx + 1, right)
```

Theorem 3.11. *The runtime of Quick Sort is $O(n^2)$.*

Proof. In the worst case scenario, the pivot is the smallest or largest element, so it ends up on the leftmost or rightmost end of the array. This only reduces the problem size by 1.
At every level, we need to spend time running partition on arrays of size n, $n-1$, $n-2$, …, 1; each run is linear time in the size of the array being partitioned.

$$\begin{aligned} T(n) &= O(n) + [O(n-1) + O(1)] + [O(n-2) + O(1)] + \cdots + O(1) \\ &= O(n^2). \end{aligned}$$

□

Theorem 3.12. *The space complexity of Quick Sort is $O(n)$.*

Proof. It is tempting to think because we do not use an auxiliary array, we only use extra $O(1)$ space. But we need to keep track of the recursive calls being made, in the form of a recursive call stack.
Each recursive call must store information such as the $left$ and $right$ indices, which requires $O(1)$ space per call. In the worst case, there are n levels. Hence, up to n recursive calls may exist on the call stack at the same time. □

3.4.1 Three-way partitioning

Standard Quick Sort has to deal with the problem of many duplicate elements. In the extreme case, if all elements are the same, then running partition does not change anything, so in total all partitions run in $O(n^2)$.
To deal with duplicates, we do 3-way partition. The specification is as follows: Suppose we run QuickSort(l, r) on an array A, where l and r are pointers to the left- and right-most indices of the subarray to be sorted. After 3-way partitioning using some pivot p, the array is permuted such that there exist indices a and b such that:

- $A[l \dots a]$ only contains elements $< p$
- $A[a+1 \dots b]$ only contains elements $= p$
- $A[b+1 \dots r]$ only contains elements $> p$

Implementation:

(1) Perform 3-way partition: Apply partition once to split into $< p$ and $\geq p$, then another time on the part $\geq p$ to split into $= p$ and $> p$.

(2) Recurse on $< p$ and $> p$.

Then our new quick sort runs in

$$T(n) = 2T\left(\frac{n}{2}\right) + 2O(n \log n).$$

3.4.2 Randomised quick sort

Standard quick sort performs poorly when the pivot consistently produces highly unbalanced partitions (i.e., the partitioning pivot ends up too close to the sides). To mitigate this, we introduce **randomisation** to ensure that, with high probability, the pivot produces a reasonably balanced split.

A **randomised algorithm** makes random choices during execution. Its performance is analysed using **expected runtime**, which is the expectation of the running time over the algorithm's internal randomness.

Lemma 3.13. *Suppose an experiment succeeds with probability p independently on each trial. Let X be the number of trials until the first success (including the successful trial). Then $\mathbb{E}[X] = \frac{1}{p}$.*

Proof. For each $n = 1, 2, \ldots,$

$$\mathbb{P}(X = n) = (1-p)^{n-1}p.$$

Hence,

$$\mathbb{E}[X] = \sum_{n=1}^{\infty} n(1-p)^{n-1}p.$$

Using the identity

$$\sum_{n=1}^{\infty} nr^{n-1} = \frac{1}{(1-r)^2} \qquad (|r| < 1)$$

with $r = 1 - p$, we obtain

$$\mathbb{E}[X] = p \cdot \frac{1}{p^2} = \frac{1}{p}.$$

□

The **random partitioning algorithm** repeatedly samples a pivot uniformly at random until its final position lies in the middle third of the array. In the worst case, we split the array into $\frac{2}{3}$ and $\frac{1}{3}$.

Algorithm 3.14 Random Partition

Require: Array $A[1 \ldots n]$
Ensure: Index k such that $k \in [\lfloor n/3 \rfloor, \lceil 2n/3 \rceil]$

```
repeat
    p ← RandomInteger(1, n)        ▷ Randomly pick an integer in [1, n] as pivot
    k ← Partition(A, p)            ▷ Do partitioning
until ⌊n/3⌋ ≤ k ≤ ⌈2n/3⌉
return k
```

Proposition 3.15. *The expected runtime of Random Partition is $O(n)$.*

Proof. Since we randomly pick an element to be a partition pivot, the probability that it lies in the middle third is $\frac{1}{3}$. By the lemma, the expected number of times we partition the array before we return is 3 times. Hence, the expected runtime is $3 \times O(n) = O(n)$. □

Algorithm 3.16 Randomised Quick Sort

```
procedure ParanoidQuicksort(arr, left, right)
    if left + 1 = right then
        return
    pivot_idx ← RandomPartition(arr)      ▷ Run random partition on arr to get a good pivot
    ParanoidQuicksort(arr, left, pivot_idx)      ▷ Recurse on left sub-array
    ParanoidQuicksort(arr, pivot_idx + 1, right)      ▷ Recurse on right sub-array
```

Theorem 3.17. *The expected runtime of Randomised Quick Sort is $O(n \log n)$.*

Proof. Let $T(n)$ denote the *expected* runtime on an array of length n. Let $P(n)$ denote the runtime of partitioning the array before recursing (i.e., the amount of work done at the current level of recursion). Then

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + P(n),$$

where we recurse on two sub-arrays with sizes within $\frac{n}{3}$ and $\frac{2n}{3}$.
The depth of the deepest part of the tree is $\log_{3/2} n$, so the height of the tree is $O(\log n)$. At each level, the work done is $O(n)$. Hence, the total work done is $O(n \log n)$. □

Theorem 3.18. *The space complexity of Randomised Quick Sort is $O(\log n)$.*

Proof. The recursion depth is $O(\log n)$. Each recursive call uses constant space. Hence, the total space usage is $O(\log n)$. □

3.5 Bubble sort*

Compare and swap
Idea: Repeatedly scan the array from left to right and swap adjacent elements that are out of order. Each pass moves the largest remaining unsorted element to its correct final position.

Algorithm 3.19 Bubble Sort

```
1: procedure BubbleSort(A[1 ... n])
2:     repeat
3:         swapped ← false
4:         for j ← 1 to n − 1 do
5:             if A[j] > A[j + 1] then
6:                 Swap A[j] and A[j + 1]
7:                 swapped ← true
8:         n ← n − 1                          ▷ Largest element fixed at position n + 1
9:     until swapped = false
```

Theorem 3.20. *The time complexity of Bubble Sort is $O(n^2)$.*

Proof. In the worst case (array in strictly decreasing order), every comparison results in a swap.
During the first pass, the inner loop performs $n - 1$ comparisons. During the second pass, it performs $n - 2$ comparisons. Continuing in this way, the total number of comparisons is

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = O(n^2).$$

Each iteration performs $O(1)$ work, so the total running time is $O(n^2)$. □

Remark. If the input array is already sorted, the algorithm terminates after one pass. Hence, the best-case running time is $O(n)$.

Invariant: At the end of the j-th iteration, the biggest j items are correctly sorted in the final j position of the array.

3.6 Counting Sort

Counting Sort is a non-comparative sorting algorithm that operates by counting the number of objects that have each distinct key value. It is particularly efficient when the range of input values, k, is not significantly larger than the number of items to be sorted, n.

Given an array A of n positive integers in $[1, k]$, the algorithm uses a **histogram** to track frequencies.
Algorithm:

(1) **Initialize:** Create an array $histo$ of size k, initialized to all 0.

(2) **Count:** Iterate through the input array A. For each element $x \in A$, increment $histo[x]$ by 1.

(3) **Reconstruct:** Iterate through the $histo$ array from $v = 1$ to k. For each value v, output the integer v exactly $histo[v]$ times into the sorted result array.

Theorem 3.21. *The runtime of Counting Sort is $O(n + k)$.*

Proof.

- Initialization takes $O(k)$ time.
- A single pass over the input array A takes $O(n)$ time.
- Iterating through the $histo$ array (length k) and performing of n output operations takes $O(n + k)$ time.

Hence, total time complexity is $O(n + k)$. □

Radix Sort

Together with a stable counting sort and a low digit count in the integer input, sorting in linear time can be achieved. From the least significant digit, stable counting sort is performed on this digit and this will continue until all digits are sorted. By the end of this process a sorted array of integers is obtained.

4 Elementary Data Structures

A **data structure** is a way to store data, with algorithms that support operations on the data; a collection of supported operations is called an **interface** (also API or ADT). To compare the two, an interface is a specification: what operations are supported (the problem!); a data structure is a representation: how operations are supported (the solution!).
To give a data structure, we generally need to specify:

(1) How our data will be organised.

(2) The set of operations that it supports and furthermore how each data structure operation updates the data that we store.

For efficiency, each data structure has some **invariant**, which must be maintained by every operation.

Summary of API

Array:

- Read: $O(1)$
- Append: $O(1)$ amortised
- Size: $O(1)$

Linked list:

- Set: $O(n)$
- Delete: $O(n)$
- Lookup: $O(n)$
- Size: $O(1)$
- Append: $O(1)$

Binary search tree:

-

AVL tree:

- Insert: $O(\log n)$
- Delete: $O(\log n)$
- Rank: $O(\log n)$
- Select: $O(\log n)$
- Size: $O(1)$

Hash table:

Operation	**Description**	**Expected time**	**Worst-case time**
INSERT($key, value$)		$O(1)$	$O(n)$
DELETE(key)		$O(1)$	$O(n)$
LOOKUP(key)	Given a key, return the value associated the key	$O(1)$	$O(n)$
SIZE()			$O(1)$
LISTPAIRS()	Returns a linked list of key-value pairs that currently exist in the table	$O(1)$	$O(1)$

Separate chaining:

Operation	Worst-case	Expected (under SUHA)
Lookup	$O(n)$	$O(n/m)$
Insert	$O(n)$	$O(n/m)$
Delete	$O(n)$	$O(n/m)$

Linear probing:

Operation	Worst-case	Expected (under SUHA)
Lookup	$O(n)$	$O(1)$
Insert	$O(n)$	$O(1)$
Delete	$O(n)$	$O(1)$

Binary heap / priority queue:

- Insert
- Decrease key
- Pair< priority , Value> extract_min_priority (): remove the item with minimum priority
- Pair< priority , Value> peek_min(): find out which item has minimum priority without removing it
- Size
- PQ make_pq(ArrayList<Pair<priority , Value>> array):

4.1 Array

Definition 4.1. An **array** is a **fixed-size** data structure that holds items of a certain type.

An array supports the following operations.

- READ($index$): Access the element at the specified index; this takes $O(1)$ time.
- SET($index, item$):
  (1) Find the length of the array: $O(1)$ time
  (2) Make a new array that is one element longer: $O(n)$ time
  (3) Copy over the old elements: $O(n)$ time
  (4) Set the new element: $O(1)$ time
  (5) Delete the old array: $O(n)$ time
- SIZE(): Access the length variable and return its value; this takes $O(1)$ time.

4.2 Linked List

A **linked list** is a recursive data structure that is either empty (**null**) or consists of a **node** containing

(1) an element

(2) a reference to the next node in the list (or **null** for the tail node).

```
struct LinkedList {
  LinkedList *head;  // pointer to the first node
  LinkedList *tail;  // pointer to the last node
  int size;          // number of nodes in the list
};
```

- SET($index, item$): Start at the head node. Traverse the list node by node until reaching the node at the specified index. Update the node's element to $item$.

  In the worst case, setting the last element takes $O(L)$ time.

- DELETE($index$): Start at the head node. Traverse to the node immediately before the one to delete (at $index - 1$). Update its next pointer to skip over the node at $index$. If deleting the tail node, update the list's tail reference. Decrement $size$.

- LOOKUP($index$): Start at the head node. Traverse node by node until reaching the node at the specified index. Return the node's element.

  In the worst case, to access the i-th element, that takes $O(i)$ memory accesses. An element at the end of the array would take $O(L)$ time.

- SIZE(): Access the $size$ variable and return it; this takes $O(1)$ time.

- APPEND(n): Set the current tail node's $next$ reference to the new node n. Update the list's tail pointer to n. Increment size.

  All these steps take $O(1)$ time.

- Prepend: similar to append

The following are several variations of linked lists:

- **Untailed vs. Tailed**:

  Typically, a standard linked list is stored by keeping a reference only to its first node (i.e. head).

  A **tailed** linked list maintains an additional reference to the last node (i.e., tail).

- **Singly vs. Doubly linked**:

  By default, we assume a linked list is **singly** linked, meaning each node stores a pointer only to the next node.

  A **doubly** linked list uses nodes that store pointers to both the next and previous nodes, enabling bidirectional traversal.

see tutorial 1 for more

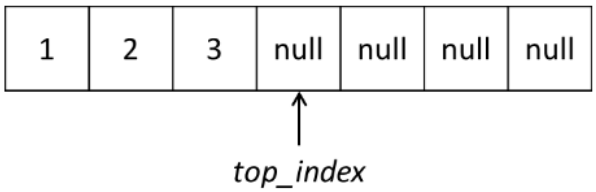
4.3 Stack and Queue

Definition 4.2. A **stack** is a Last In First Out (LIFO) collection of elements.

A stack supports the following operations in $O(1)$ time:

- PUSH($item$): Adds an element to the stack
- POP: Removes the last element that was added to the stack
- PEEK: Returns the last element added to the stack (without removing it)

A stack can be implemented in Java using a fixed-size array.



Maintain top_index that represents the index of the top of the stack.

- To perform a push, place the new element at top index and increment top index.
- To perform a pop or peek, return $A[top\_index - 1]$ (and subsequently decrement top index in a pop operation).

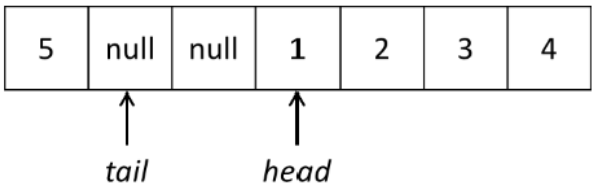
All of these operations are achievable in O(1) time.

Definition 4.3. A **queue** is a First In First Out (FIFO) collection of elements.

A stack supports the following operations in $O(1)$ time:

- ENQUEUE(Q, x): Adds element x to the back of the queue
- DEQUEUE(Q): Removes and returns the element at the front of the queue
- PEEK(Q): Returns the next item to be dequeued (without removing it)

A queue can be implemented in Java using a fixed-size array.



Maintain two indices, *head* and *tail* to represent the head and tail of the queues respectively (both starting at 0).

- To perform enqueue, place the new element at arr[tail] and increment tail.
- To perform dequeue or peek, return the element at head (and increment head in a dequeue operation).

Again, all the operations would be in O(1).
We might find that after performing $l - 1$ enqueues and $k < l$ dequeues (where l is the array's capacity), we would be enqueuing at index l which is out of the array's index bounds. At the same time, we find that array indices 0 to $k - 1$ are all unused.
The idea here is to **wrap around** and use 0 as the next index instead of l, so we can reuse the unused spaces at the front of the array. A simple way to achieve this is to replace increments $tail + 1$ with $(tail + 1) \mod l$.

4.4 Monotonic stack

A **monotonic stack** is a stack that maintains its elements in a fixed order either increasing or decreasing.
When a new element is pushed, it is compared with the top of the stack. If the order is violated, elements are popped until the property is restored, and then the new element is pushed.

```
1: int[] a = new int[n]; // contains the sequence of inputs
2: Stack<Integer> st = new Stack<>();
3: for (int i = 0; i < n; i++) {
4:   while (!st.empty() && a[st.peek()] >= a[i]) {
5:     st.pop();
6:   }
7:   st.push(i);
8: }
```

The double loops may seem like $O(n^2)$ time complexity. However, the elements in the sequence is only added or removed from the stack with a maximum of 2 operations per element. Thus, it results in a runtime of $O(n)$.

Example (Sliding Window Maximum)**.**
LeetCode: https://leetcode.com/problems/sliding-window-maximum/
You are given an array of integers nums and an integer k. There is a sliding window of size k that starts at the left edge of the array. The window slides one position to the right until it reaches the right edge of the array.
Return a list that contains the maximum element in the window at each step.

Solution. The idea is to use a monotonically decreasing deque, so that we can efficiently track the maximum inside the sliding window. This guarantees that:

- The front of the deque always holds the index of the current window's maximum.
- Smaller elements behind a bigger one are useless (they can never become the max later), so we remove them when pushing a new number.
- If the element at the front falls out of the window, we remove it.

Algorithm:

(1) Use a deque to store indices of elements in decreasing order of their values.

(2) Expand the window by moving the right pointer:

Before inserting the new index, remove indices whose values are smaller than the new value (they cannot be future maximums).

Add the new index to the deque.

(3) If the left pointer passes the front index, remove it (it's outside the window).

(4) Once the window reaches size k, the front of the deque is the maximum; add it to the output.

(5) Slide the window and repeat.

□

4.5 Max stack & queue

A **max stack** is a stack that also allows us to efficiently retrieve the maximum element. It supports the following operations:

(1) Push(x): Add element x to the top of the stack.

(2) Pop(): Remove and return the element from the top of the stack (most recently added element).

(3) Get-Max(): Return the maximum element currently in the stack without removing it.

Idea: Use a single stack where each entry is stored as a pair. The first part of the pair represents the actual element, while the second part keeps track of the maximum value in the stack up to that point.
This way, every push automatically updates the running maximum, and the top of the stack always gives us direct access to the current maximum.

(1) To push an element x:

- If the stack is empty, push (x, x) since the element itself is the maximum.
- Otherwise, compare the new element with the current maximum (the second value at the top of the stack). The new pair becomes $(x, \max(x, \text{currentMax}))$.

(2) To pop, remove the top pair.

(3) To get max, return the second value of the top pair.

A **max queue** is a FIFO queue that also allows us to retrieve the maximum element currently in the queue in $O(1)$. It supports the operations:

(1) Enqueue(x)

(2) Dequeue()

(3) Get-Max()

(4) Add-To-All(v)

To build a max queue, we use two max-stacks called $in-queue$ and $out-queue$.

(1) To enqueue x, push $(x, \max(x, \text{currentMax}))$ into the in-queue.

(2) To dequeue, since our elements are in the wrong order, first reverse it into the out-queue. While in-queue is non-empty, pop and push into out-queue. NOTE: Only push the value, not the entire pair. Remember to update max seen so far.

Then pop from out-queue.

(3) To get max, return the maximum across both stacks (access top of each stack).

(4) To avoid iterating through all elements, create a new variable called $offset$. Then an element's true value = value in queue + $offset$. To run Add-To-All(v), add v to $offset$.

To enqueue x, we actually enqueue $x - offset$.

To dequeue, we dequeue from the queue, then add $offset$.

To get max, we get max of our queue, then add $offset$.

5 Binary Search Tree

Graph Theory

Definition 5.1. A **directed graph** G is a pair (V, E), where V is a finite set and E is a binary relation on V. The set V is called the **vertex set** of G, and its elements are called **vertices**. The set E is called the **edge set** of G, and its elements are called **edges**.

In an **undirected graph** $G = (V, E)$, the edge set E consists of unordered pairs of vertices, rather than ordered pairs. That is, an edge is a set $e = \{u, v\}$, where $u, v \in V$, $u \neq v$.
An edge is said to **connect** its endpoints; two vertices that are connected by an edge are called **adjacent** vertices.
An edge is said to be **incident on** each of its endpoints, and two edges incident on the same endpoint are called **adjacent edges**.

Definition 5.2. The **degree** of a vertex is the number of edge connections incident at that vertex.

Definition 5.3. A **loop** is an edge which joins a vertex to itself.

Definition 5.4. A graph is **simple** if it has no loops and no multiple edges.

Definition 5.5. A **walk** of length k from a vertex u to a vertex v in a graph $G = (V, E)$ is a sequence

$$v_0, v_1, v_2, \ldots, v_k$$

of vertices such that $v_0 = u$, $v_k = v$, and $(v_{i-1}, v_i) \in E$ for $i = 1, 2, \ldots, k$.

Intuitively, a walk is simply a way of getting from vertex u to vertex v. It is permissible to repeat edges and vertices.

Definition 5.6. A **trail** is a walk in which all edges are distinct. Note that vertices may be repeated.

Remark. Walks and trails of length 0 are permitted. Specifically there is a walk/trail of length 0 from each vertex to itself.

Definition 5.7. A **path** is a walk in which all vertices are distinct. Note that if vertices are distinct then so are edges.

Remark. We can't have a path of length 0 because we can't repeat vertices.

Definition 5.8. A **cycle** is a path $v_0, v_1, \ldots, v_k$ in which $v_0 = v_k$ and the path contains at least one edge. A graph with no cycles is **acyclic**.

Definition 5.9. A graph is **connected** if for any two vertices there is at least one path joining those vertices.

Definition 5.10. A **tree** is a connected, acyclic, undirected graph.

Lemma: a tree of n nodes has exactly $n - 1$ edges.

Definition 5.11. A **DAG** is a directed, acyclic graph.

Definition 5.12. A **complete graph** is when all vertices have edges to every other vertex.

Definition 5.13. A **subtree** is a sub-graph of the original graph that is also a tree.

Definition 5.14. A **rooted tree** is a tree in which one of the vertices is distinguished from the others, called the **root** of the tree. A vertex of a rooted tree is called a **node** of the tree.
Moving down from the root node is to move to the **children**. Moving towards the root is to move to the **parent**.

Definition 5.15. A **binary tree** T is a structure recursively defined on a finite set of nodes that either

- contains no nodes, or

- is composed of three disjoint sets of nodes: a **root node**, a binary tree called its **left sub-tree**, and a binary tree called its **right sub-tree**.

That is, each node either has 0, 1, or 2 children.

Definition 5.16. A graph $G = (V, E)$ is **weighted** if every edge has an associated **weight**, typically given by a **weight function** $w : E \to \mathbb{R}$.

5.1 Organisation

Based on linked lists and arrays, we specify the indices for these operations. Moving away from using positions and indices for our operations, **associative data structures** store **key-value pairs**; each key is associated with a value.

Definition 5.17. A **binary search tree** (BST) is organised in a binary tree. Each node stores

- k: a key
- $value$: a value
- $left$: a pointer to its left child
- $right$: a pointer to its right child

For simplicity, assume that the keys are distinct.
For now, we only have access to the entire tree via a reference to the **root node**, so everything has to start there.
The keys in a BST are always stored in such a way as to satisfy the **BST invariant**:

> Let x be a node in a BST.
>
> - If y is a node in the left sub-tree of x, then $y.key < x.key$.
> - If y is a node in the right sub-tree of x, then $y.key > x.key$.

Remark. For simplicity, the values are less important, so sometimes we will only include the keys in our explanation.

5.2 Operations

5.2.1 Searching

Search operation: Given a pointer to the root of the tree and a key k, if a node with key k exists, return the value associated with the node; otherwise, return null.
Algorithm:

(1) Start at the root node, and go down the tree.

(2) For each node x encountered, compare $x.key$ with k.

(3) If $x.key = k$, then we found the correct node.

If $x.key > k$, then k lies in the left sub-tree of x. Recurse on the left sub-tree of x.

Else if $x.key < k$, then k lies in the right sub-tree of x. Recurse on the right sub-tree of x.

Algorithm 5.18 Searching on BST

```
1: procedure SEARCH(x, k)
2:     if x = null then
3:         return null                                  ▷ Cannot find node with key k
4:     if x.key = k then
5:         return x.value
6:     else if x.key > k then
7:         return SEARCH(x.left, k)
8:     else
9:         return SEARCH(x.right, k)
```

Theorem 5.19. *Given a tree with n nodes, the runtime of* SEARCH *is* $O(n)$.

Proof. The nodes encountered during the recursion form a simple path downward from the root of the tree. In the worst case, we traverse the entire height of the tree. Assume our keys are integers, each of the n comparisons takes $O(1)$ time. □

5.2.2 Insertion

Insertion operation: Given a key-value pair (k, v), insert a new node into the BST where it stores (k, v).
For simplicity, we will only insert at leaf positions.
Algorithm:

(1) Start at the root node.

(2) At the current node x, compare $x.key$ with k.

(3) If $k < x.key$, recurse on the left (to maintain the invariant).
If $k > x.key$, recurse on the right (to maintain the invariant).

(4) The moment we recurse out of the tree, we insert the new node as a leaf.

Algorithm 5.20 Insertion operation on BST

```
1: procedure INSERT(k, v)
2:     root ← INSERT(root, k, v)
3: procedure INSERT(x, k, v)                          ▷ Helper function
4:     if x = null then                               ▷ Reached a leaf node
5:         return new Node(k, v)
6:     if k < x.key then
7:         x.left ← INSERT(x.left, k, v)              ▷ Recurse on left sub-tree, update reference
8:     else
9:         x.right ← INSERT(x.right, k, v)
10:    return x                                       ▷ Return the updated subtree root
```

Theorem 5.21. *The time complexity of the insertion operation in a BST is* $O(n)$.

5.2.3 Deletion

Deletion operation: Given a key, remove the node containing the key from the BST.
Idea: find the node first, then remove it. Consider the cases:

Case 1: The node has 0 children (i.e., a leaf node).

Simply remove it from the tree, i.e., set it to null.

Case 2: The node has 1 child.

Delete it, and attach the child to that node's parent.

Case 3: The node has 2 children.

We remove the current node, and replace it with another node. Viable replacement nodes are:

- largest node in left sub-tree (**predecessor**),
- smallest node in right sub-tree (**successor**).

Remark. We cannot simply replace node k with node n, because node n might have children.

Note that n cannot have a left-child, as it would contradict the minimality of n. Thus, node n has at most one child, so we can fall back to the easier cases for deletion.

(1) Find the successor of node k (as the minimum node on the right sub-tree).

(2) Make a copy of node n.

(3) Delete it from the tree.

(4) Replace node k with node n.

The runtime is $O(n)$, since finding the successor takes $O(n)$.

5.2.4 Rank

Size augmentation: Each node holds a variable *size*, which represents the size of the sub-tree rooted at that node.

Thus, every operation needs to maintain this *size* variable.

Rank operation: Given a key k, output which rank k appears in the tree (indexed from 0).

Notice that the rank equals the number of elements smaller than the value we are looking for.

Algorithm:

(1) Start at the root node, and go down the tree.

(2) Suppose we are at a node x.

- If $x.key = k$, then we have found the item we are looking for. Return the size of the left sub-tree of x.
- If $x.key > k$, then key belongs in the left sub-tree. Recurse on the left sub-tree of x.
- If $x.key < k$, then recurse on the right sub-tree of x. Whatever answer we obtain, we need to add the size of the left sub-tree + 1 (for the current node).

Algorithm 5.22 Rank operation on BST

```
1: procedure RANK(x, k)
2:     if x.key = k then
3:         return x.left.size
4:     else if x.key < k then
5:         return RANK(x.right, k) + x.left.size + 1
6:     else
7:         return RANK(x.left, key)
```

now computing size costs $O(1)$, since we access the variable itself.

Theorem 5.23. *Time complexity is* $O(h)$.

Modify insertion: After we insert, traverse back up from the inserted leaf to the root, and update size for every node that we traversed by. As the recursion unwinds, we recompute $current\_node.size$ as

$$current\_node.left.size + current\_node.right.size + 1$$

The deletion operation can be modified similarly.

5.2.5 Select

Select operation: Given a rank (indexed from 0), output the key which has that rank.
Idea: Suppose we are at node x.

(1) If $x.left.size$ = rank, then we have found the key. Return $x.key$.

(2) If $x.left.size >$ rank, then the key lies in the left sub-tree. Recurse on left sub-tree.

(3) If $x.left.size <$ rank, then the key lies in the right sub-tree. Recurse on right sub-tree.

Algorithm 5.24 Select operation on BST

```
1:  procedure SELECT(x, rank)
2:      if x.left = null then                ▷ Check if left sub-tree is null; handle null pointer exception
3:          left_size ← 0
4:      else
5:          left_size ← x.left.size
6:      if left_size = rank then             ▷ Current node is the desired rank
7:          return x.key
8:      else if left_size > rank then        ▷ Desired element lies in left sub-tree
9:          return SELECT(x.left, rank)      ▷ Recurse on left sub-tree
10:     else                                 ▷ Desired element lies in right sub-tree
11:         return SELECT(x.right, rank − left_size − 1)   ▷ Recurse on right sub-tree
```

5.3 Problem Solving

For now we assume:

(1) Our binary search tree is worst case height $O(\log n)$.

(2) Our binary search tree is augmented to support rank, select operations in $O(\log n)$.

(3) Our binary search tree is augmented to return the size of the tree in $O(1)$.

Example:

> **Input:** An array A of n unique integers.
> **Output:** The number of swaps performed by Insertion Sort on the input array A in order to sort it.

Observation: The number of swaps an element at index i makes equals the number of elements in $A[0, i)$ larger than it.

To leverage this observation, we want to store the elements in $A[0, i)$ in some data structure, such that we can quickly find out how many are larger than it. Binary search trees are great here!

Recall that tree.rank(x) equals the number of items smaller than x. Thus, the number of items in the BST larger than x is

$$\text{tree.size}() - \text{tree.rank}(x) - 1$$

where we exclude x itself.

Count number of swaps performed by Insertion Sort

```
1: total ← 0                                              ▷ Total number of inversions
2: tree ← empty BST
3: for i = 0, ..., n − 1 do
4:     tree.insert(A[i])                                  ▷ Insert the i-th element into the tree
5:     total ← total + (tree.size() − tree.rank(A[i]) − 1)
6:                                                        ▷ Count elements greater than arr[i] already inserted, update total
```

5.4 Print

All the keys in a BST can be printed out in sorted order by a simple recursive algorithm, called an **inorder tree walk**. This algorithm is so named because it prints the key of the root of a sub-tree between printing the values in its left sub-tree and printing those in its right sub-tree.

To use the following procedure to print all the elements in a BST T, we call INORDER-TREE-WALK($T.root$):

Algorithm 5.25 Inorder Tree Walk in BST

```
1: procedure INORDER-TREE-WALK(x)
2:     if x ≠ null then
3:         INORDER-TREE-WALK(x.left)
4:         print x.key
5:         INORDER-TREE-WALK(x.right)
```

The correctness of the algorithm follows by induction directly from the binary-search-tree property.

Theorem 5.26. *If x is the root of an n-node subtree, then the time complexity of INORDER-TREE-WALK(x) is $\Theta(n)$.*

In-order traversal: left subtree, node, right subtree Pre-order traversal: node, left subtree, right subtree Post-order traversal: left subtree, right subtree, node

see tutorial 3

Given a pre-order traversal result $A[1 \dots n]$ of a binary search tree T, suggest an algorithm to reconstruct the original tree T.

(1) Set the root node to be the first element $A[1]$.

(2) Find the position of the first element *less* than this value (denoted as i_1), and the position of the first element *larger* than this value (denoted as i_2).

(3) Recurse on both $A[i_1 \dots i_2 - 1]$ and $A[i_2 \dots n]$ to obtain two BST.

(4) Set left child of root to be BST returned from first sequence, and right child to be BST returned from the second sequence.

6 Balanced Binary Search Tree

Currently, BST operations are $O(h)$, so we want our BST to have small height for efficiency. Hence, we need a balanced BST.

6.1 AVL tree

Definition 6.1. An **AVL tree** is a binary search tree that is **height balanced**: for each node x, the heights of the left and right sub-trees of x differ by at most 1.

To implement an AVL tree, we maintain an extra attribute in each node: $x.h$ is the **height** of node x. Height is defined as the number of edges from root to bottom of tree. Recursively,

$$x.height = \max\{x.left.height, x.right.height\} + 1.$$

After an insertion, as we return from our recursion, re-update the heights as above. Heights can be maintained similarly for deletion.

Lemma 6.2. *An AVL tree of height $h \geq 0$ has $\Omega(\phi^h)$ nodes, where $\phi = (1 + \sqrt{5})/2$.*

Proof. Let $N(h)$ denote the minimum number of nodes in any AVL tree of height h. We will generate a recurrence for $N(h)$ as follows. First, observe that a tree of height zero consists of a single root node, so $N(0) = 1$. Also, the smallest possible AVL tree of height one consists of a root and a single child, so $N(1) = 2$.

For $n \geq 2$, let h_L and h_R denote the heights of the left and right subtrees respectively. Since the tree has height h, one of the two subtrees must have height $h - 1$, say, h_L. To minimise the total number of nodes, we should make the other subtree as short as possible. By the AVL invariant, this implies that $h_R = h - 2$.

Counting the root node plus the numbers of nodes in the left and right subtrees, we obtain the following recurrence for the total number of nodes:

$$\begin{aligned} N(h) &= 1 + N(h_L) + N(h_R) \\ &= 1 + N(h-1) + N(h-2) \end{aligned}$$

and $N(0) = 1$, $N(1) = 2$.

While $N(h)$ is not quite the same as the Fibonacci sequence, by an induction argument[1] we can show that for large h, there is a constant c such that

$$N(h) \geq c\phi^h.$$

□

Theorem 6.3. *An AVL tree with n nodes has height $O(\log n)$.*

Proof. From the above lemma, up to constant factors we have $n \geq \phi^h$, which implies that $h \leq \log_\phi n = \log n / \log \phi$. Since $\phi > 1$ is a constant, so is $\log \phi$. Therefore, h is $O(\log n)$. (If you work through the math, the actual bound on the height is roughly $1.44 \log n$.) □

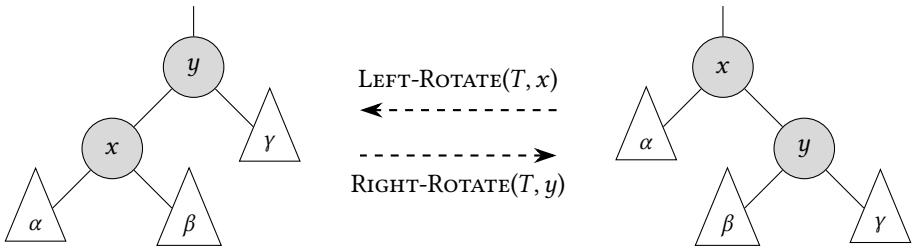
[1] For the base cases, $N(0) = 1$ and $\phi^0 = 1$, so choose $c = 1$; $N(1) = 2$ and $\phi^1 \approx 1.618$, so choose $c = 1$.
Assume that for some $h \geq 2$, we have $N(h-1) \geq c\phi^{h-1}$ and $N(h-2) \geq c\phi^{h-2}$. Then

$$\begin{aligned} N(h) &= 1 + N(h-1) + N(h-2) \\ &\geq 1 + c\phi^{h-1} + c\phi^{h-2} \\ &= 1 + c\phi^{h-2}(\phi + 1) \\ &= 1 + c\phi^{h-2}(\phi^2) = 1 + c\phi^h. \end{aligned}$$

For large h, the constant 1 becomes negligible compared to $c\phi^h$, so $N(h) \geq c\phi^h$.

6.1.1 Tree rotations

Since insertion and deletion operations modify the height of a subtree, they may violate the AVL invariant. To maintain the AVL invariant, we change the pointer structure through **rotation**.



Left rotation on a node x:

(1) Make right child y the new parent, adopts x as left child

(2) x goes down, and adopts y's left sub-tree as right child

Right rotation on node y:

(1) Make left child x the new parent, adopts y as left child

(2) y goes down, and adopts x's left sub-tree as right child

Remark. To remember the operations, note that in left rotation, the nodes x and y move in the left direction. The right rotation is similar.

Note that a rotation operation preserves the BST invariant: the keys in α precede $x.key$, which precedes the keys in β, which precede $y.key$, which precedes the keys in γ.

Definition 6.4. Fix a node y.

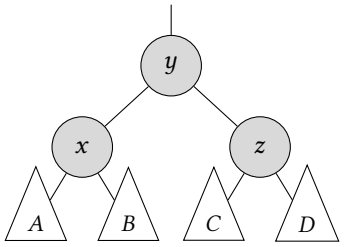
- If the left sub-tree of y is taller, we call node y **left heavy**.
- If the right sub-tree of y is taller, We call node y **right heavy**.

Intuition: if one side is too tall, we will rotate it such that the taller sub-tree moves up:

- A right rotation deals with a left heavy tree.
- A left rotation deals with a right heavy tree.

6.1.2 Insertion

Suppose an AVL tree maintained the AVL invariant on all nodes prior to insertion.



If the insertion operation does not violate the AVL invariant, then we do nothing. Otherwise, let y be the *first* node at which the AVL invariant is violated. WLOG assume we inserted into the right sub-tree of y. Thus h_z must have changed, i.e., increased by 1.

Since the AVL invariant was violated, we must have

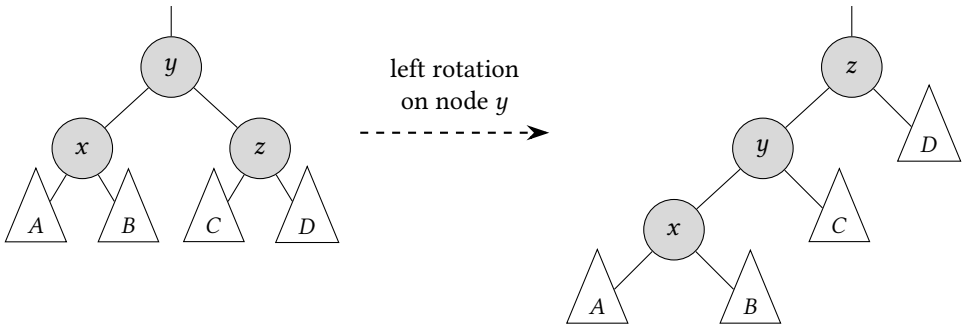
$$h_z = h_x + 2.$$

Since the insertion caused a change in h_z, we have either

$$h_C + 1 = h_D \quad \text{or} \quad h_C = h_D + 1.$$

The first case is called **right-right heavy** (since the right sub-tree of y is right heavy); the second case is called **right-left heavy** (since the right sub-tree of left heavy).

Case 1: Do a left rotation on node y

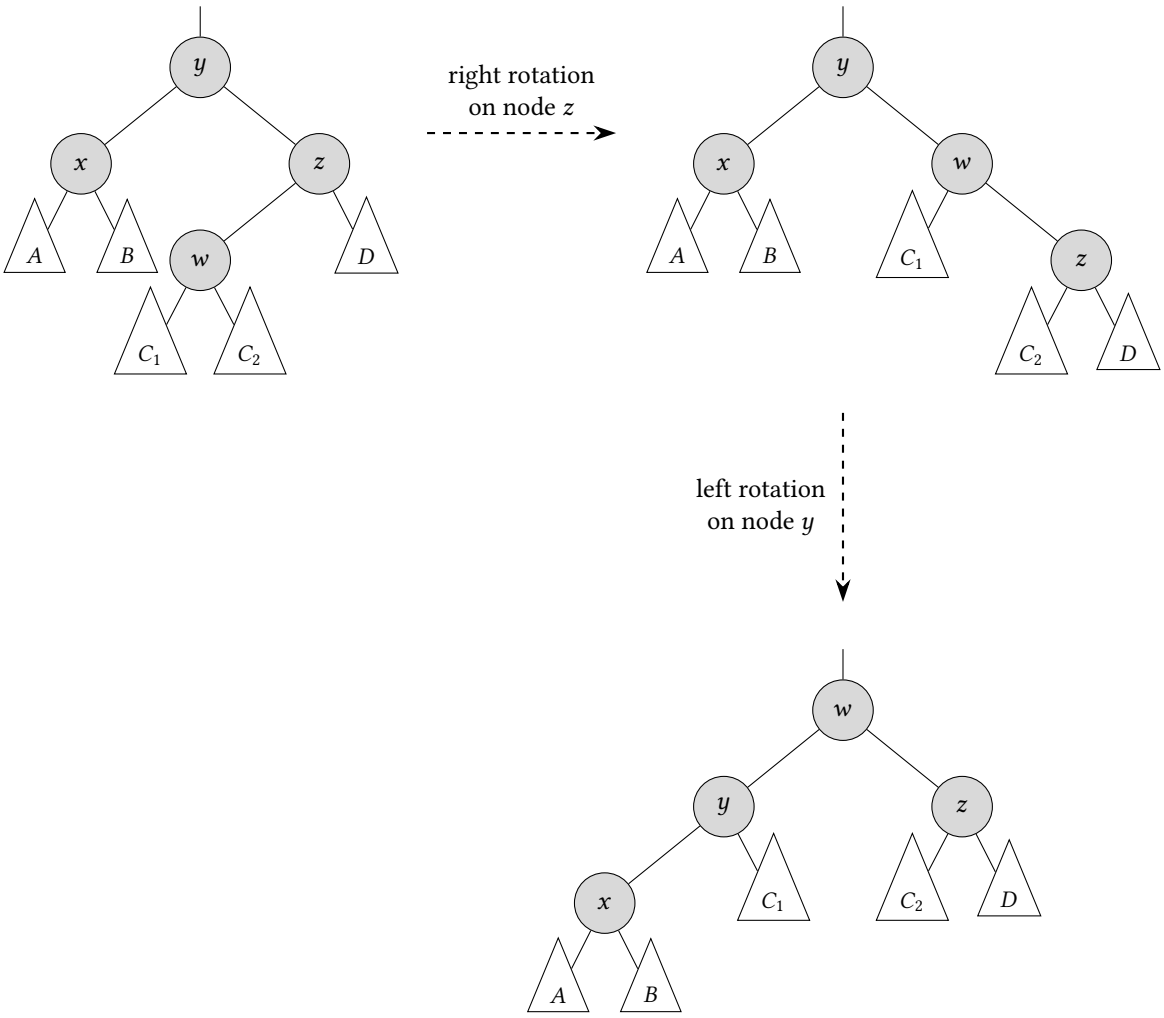


- h_x remains unchanged.
- Since $h_D > h_C$, we must have $h_D = h_z - 1 = h_x + 1$, so $h_C = h_x$.
  Both sub-trees of node y have height h_x, so $h_y = h_x + 1$.
- Since $h_D = h_x + 1 = h_y$, we have $h_z = h_x + 2$.

It is easily seen that the AVL invariant is maintained at every node.

Case 2: Do a right rotation on node z, then a left rotation on node y.

(In the diagram, we expand the left sub-tree of node z.)



Prior to rotations, $h_z = h_x + 2$ implies that $h_C = h_x + 1$ and $h_D = h_x$. Thus, $h_w = h_x + 1$, and $h_{C_1}, h_{C_2} \in \{h_x - 1, h_x\}$.

After the rotations,

$$h_y = \max\{h_x, h_{C_1}\} + 1 = h_x + 1,$$
$$h_z = \max\{h_{C_2}, h_D\} + 1 = h_x + 1,$$
$$h_w = \max\{h_y, h_z\} + 1 = h_x + 2.$$

Hence, it is easily seen that the AVL invariant maintained for all nodes.

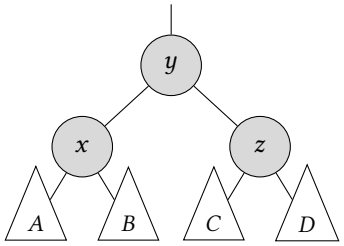
```
1:  Node insert(node, key, value):
2:      if node == null:
3:          return new Node(key, value)
4:      if key < node.key:
5:          node.left = insert(node.left, key, value)
6:      else:
7:          node.right = insert(node.right, key, value)
8:      <Recompute node.height based on node.left.height and node.right.height>
9:      <Decide if rebalancing is needed, and if so rotated as needed>
10:     return node
```

Deciding how to rebalance:

```
1: if cnode.left.height + 2 == cnode.right.height: # left heavy case
2:     if cnode.left.right.height + 1 == cnode.left.left.height: # left-right heavy case
3:         cnode.left = left_rotate(cnode.left) # update the heights too
4:     cnode = right_rotate(cnode)
5: elif cnode.right.height + 2 == cnode.left.height: # right heavy case
6:     if cnode.right.left.height + 1 == cnode.right.left.height: # right-left heavy case
7:         cnode.right = right_rotate(cnode.right) # update the heights too
8:     cnode = left_rotate(cnode)
```

6.1.3 Deletion

Suppose an AVL tree maintained the AVL invariant on all nodes prior to deletion.



If the deletion operation does not violate the AVL invariant, then we do nothing. Otherwise, let y be the *first* node at which the AVL invariant is violated. WLOG assume we deleted into the right sub-tree of y. Thus h_z must have changed, i.e., decreased by 1.

Since we deleted from the right, and the AVL invariant was violated, we have

$$h_x = h_z + 2.$$

Lemma. *$h_C = h_D$ after deletion.*

Proof. Suppose, towards a contradiction, that $h_C \neq h_D$. WLOG assume $h_C = h_D + 1$.

Since the deletion triggered a decrease in h_z, we must have that the deletion occurred in the taller sub-tree C. This means that prior to deletion, $h_C = h_D + 2$.

But that means that the AVL invariant is violated at node z, contradicting the minimality of y. □

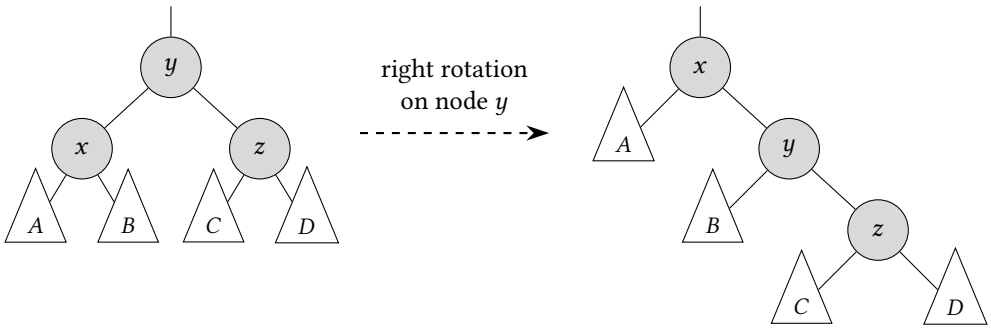
Hence, $h_C = h_D = h_x - 3$.

Since the left sub-tree rooted at x has height h_x, we have either

$$h_A = h_x - 1 = h_z + 1 \quad \text{or}$$
$$h_A = h_x - 2 = h_z.$$

The first case is called **left-left heavy**; the second case is called **left-right heavy**.

Case 1: Do a right rotation on node y.



h_z remains unchanged.

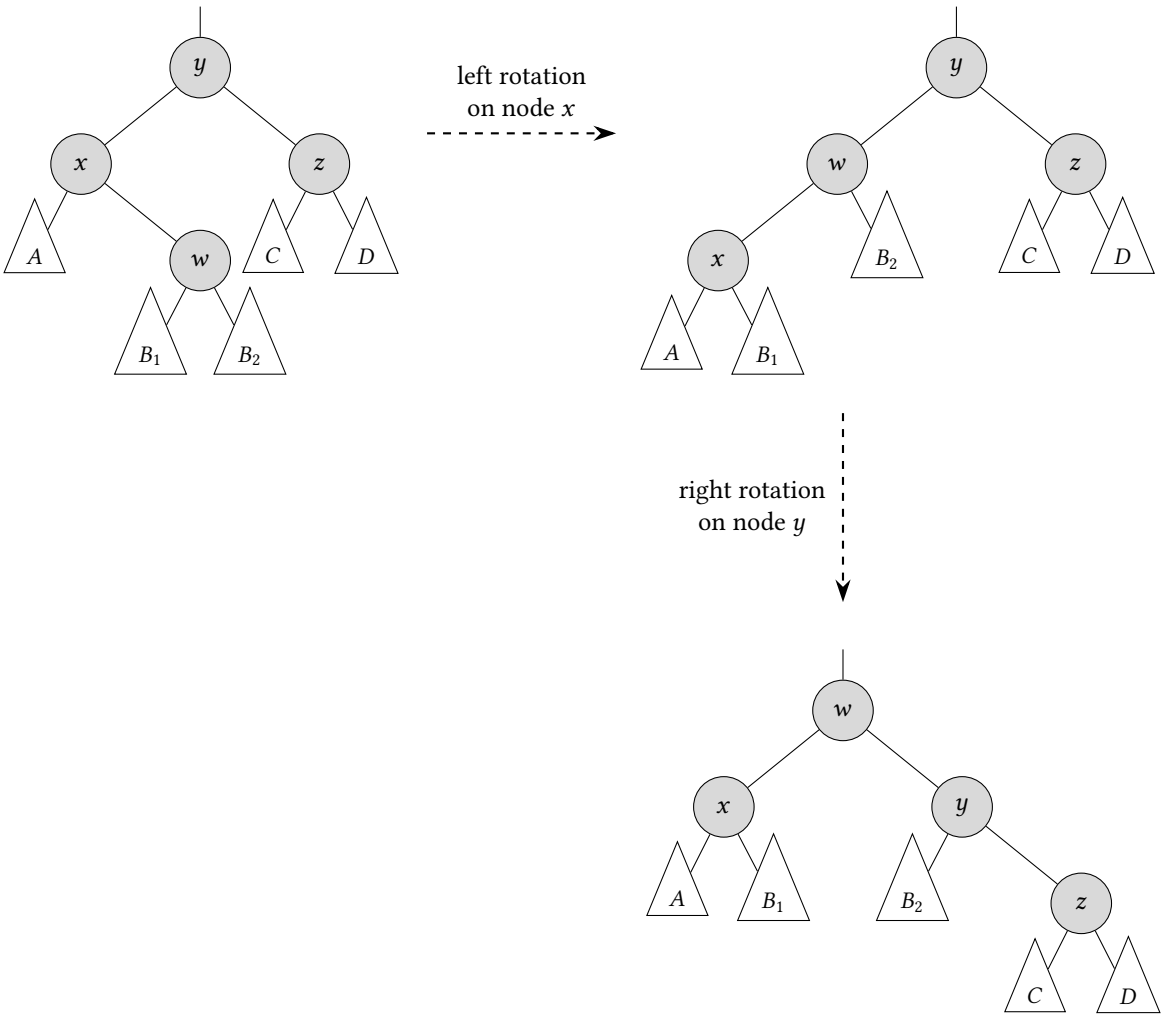
In this case, h_B is either $h_z + 1$ or h_z.

After rotation, h_y is either $h_z + 2$ or $h_z + 1$. Thus, the AVL invariant is maintained at node y.

For either value of h_y, the AVL invariant maintained at node x.

Case 2: Do a left rotation on node x, then a right rotation on node y.

(In the diagram, we expand the right sub-tree of node x.)



Note that

$$h_A = h_z$$
$$h_B = h_z + 1$$
$$h_C = h_D = h_z - 1$$

The AVL invariant is maintained at y, sinze z and h_{B_2} differ by height 1 at most.

The same applies for node x, since A and B_1 differ by height 1 at most.

For node w, node x has either height $h_z + 1$ or $h_z + 2$; node y has either height $h_z + 1$ or $h_z + 2$. In all possible cases, they will only differ by height 1. Thus, the AVL invariant is maintained at node w.

see Telegram group for a table on tree rotations

6.2 Using two bBSTs

Example (Middle of the Pack)**.** Nodle wants to adopt huskies but is quite particular about their size. A husky can be represented by a pair of integers (i, s_i), where i is the unique ID for the husky, and s_i is the size of the Husky. Every time Nodle sees a Husky, he wants to keep track of its size, and every once in a while, he wants to know the IDs of the huskies with the median size, or max size, or minimum size.
Your goal is to help Nodle design a data structure that supports the following operations:

(1) AddHusky(i, si): Add a husky i that has size si.

(2) MedianHusky(): Return the id of the husky with the median size.

(3) MinimumHusky(): Return the id of the husky with the minimum size.

(4) MaximumHusky(): Return the id of the husky with the maximum size.

You may assume that the husky sizes are unique integer values, and we will only find the median value when there is an odd number of huskies added.

midterm-sample-2 Q6

Example. Given positive integers m and k that are fixed and known in advance, create a data structure that supports the following operations:

- Add-Element(x): adds an element x
- Find-Average(): If the number of elements is $< m$, output -1. Otherwise, consider only the most recently m inserted elements. Output the average of the values of the largest k elements.

Working assumption: All elements we insert throughout the run are unique.

Solution. **Idea:** Since we need to somehow maintain largest k elements, perhaps a relevant data structure is a bBST, since it supports the Rank and Select operations.
Maintain 2 trees, both initially empty. The left tree will contain the smallest $m - k$ elements. The right tree will contain the largest k elements; we keep track of its total.
Then to implement Find-Average(), simply return the total of the right tree divided by k, which runs in $O(1)$ time.
To keep track of the m most recent elements, use a FIFO queue of size up to m. To insert the first m elements, enqueue all of them. Put them in the right tree. Eventually when the right tree grows to size $k + 1$, take the minimum of the tree, remove it from the right tree, put it in the left tree.
If the queue currently has size m, and we want to AddElement(x), first dequeue an element r, and delete r from the tree it belongs to.

- If r belongs to the left tree, then insert x into the right tree (update total as needed), then delete min from the right tree (update total again) and insert into the left tree.

- If r belongs to the right tree, then insert x into the left tree, then delete max from the left tree and insert into the right tree (update total again).

In both cases, we maintain the invariant: everything in the right tree is larger than everything in the left tree. □

Example (Storing intervals)**.** Design a data structure that stores disjoint and closed intervals $[x, y]$, where x and y are integers, and $x < y$. The desired list of operations is as follows:

- `void insert_interval (int left , int right )` - Inserts an interval starting at left and ending at right (inclusive). Furthermore, if the insertion causes the occurrence of overlapping intervals, merge them into a single interval.
- `Pair<int, int> get_interval (int x)` - Returns the left and right bound of the interval that x is in. If x is not in an interval, return NULL.
- `void expand_left (int x)` - Decreases the left bound of the interval containing x by 1. If there is no such interval, do nothing. Furthermore, if the expansion causes the occurrence of overlapping intervals, merge them into a single interval.
- `void expand_right(int x)` - Increases the right bound of the interval containing x by 1. If there is no such interval, do nothing. Furthermore, if the expansion causes the occurrence of overlapping intervals, merge them into a single interval.

Note that, by means of the above set of operations, if there are ever two stored intervals that are "overlapping" (either due to insertion, or either expansion operation), they should be merged into a single interval.

Solution. Maintain 2 trees, one to store all the left bounds, and one to store all the right bounds. Alternatively, store a single tree, and mark each node with a **marker** to indicate left/right bound.
For example, $[a, b]$, $[c, d]$, $[e, f]$ would potentially look like:

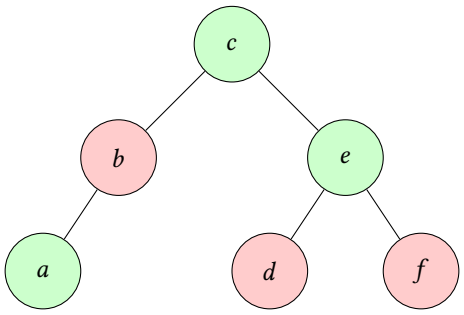


Figure 1: Left bounds are marked in green, right bounds are marked in red.

The main challenge is figuring out how to keep the tree in the right state every time an insert happens. Since an insertion of $[x, y]$ may erase a lot of nodes, we need to identify the set of nodes to delete.
Idea: If we find the predecessor of x and successor of y for an interval $[x, y]$, then we can find out what interval is behind and in front of it.

- Case 1: predecessor is a right bound, successor is a left bound.

  Delete anything in between x and y. Then insert (x, y).
- Case 2: predecessor is a right bound, successor is a right bound.

  Delete anything in between x and y. Then insert x only.
- Case 3: predecessor is a left bound, successor is a left bound.

  Delete anything in between x and y. Then insert y only.

- Case 4: predecessor is a left bound, successor is a right bound.

  Delete anything in between x and y. Insert nothing.

Once you have that maintained, looking up is also just a question of predecessor and successor. The only thing you need to worry about is when expanding. Just lookup the bound + 1 or bound - 1 depending on which direction, and potentially delete some nodes. □

6.3 (a, b)-tree

6.4 Red-black tree

6.5 Trie

7 Hash Table

7.1 Organisation

Let n denote the **number of keys** currently inserted into the table, and m denote the **table size** (i.e., size of the array).
With direct addressing, an element with key k is stored in slot k. With hashing, this element is stored in slot $h(k)$; that is, we use a **hash function** h to compute the slot from the key k. Here, h maps the universe U of keys into the slots of a **hash table** $T[0..m-1]$:

$$h\colon U \to \{0, 1, \ldots, m-1\}.$$

We say that an element with key k **hashes** to slot $h(k)$; we also say that $h(k)$ is the **hash value** of key k.

Example. Suppose that the hash function is $h(k) = 5k + 1$. To insert key 10 into a hash table of size 8, we evaluate $h(10) = 51$, so we look at index $h(10) \mod 8 = 3$, i.e., index 3.

The general flow of inserting a key k into a hash table of size m is:

(1) Compute the hash $h(k)$.

(2) Look at index $h(k) \mod m$.

(3) If the slot at that index is empty, insert x there.

(4) Otherwise, handle collisions somehow.

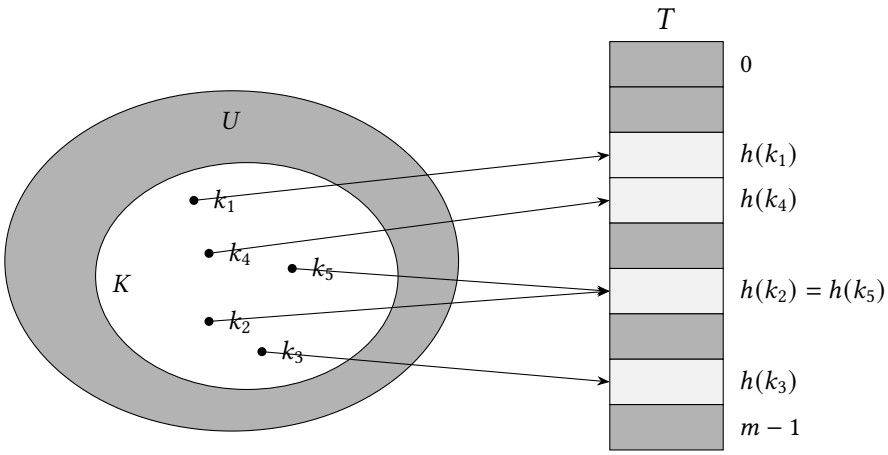


Figure 2: Using a hash function h to map keys to hash-table slots. Since keys k_2 and k_5 map to the same slot, they collide.

There is one hitch: two keys may hash to the same slot. We call this situation a **collision**. Fortunately, we have effective techniques for resolving the conflict created by collisions.

(1) Separate chaining

(2) Open addressing

Ideally, we want hash functions to behave both randomly and deterministic at the same time. The intuition is that if the function is random enough, it distributes the keys over the m possible table locations, in order to minimise collisions.

Example. We cannot have a totally random function such as $h(k) = rand(0, m)$, samples a random number between 0 and $m - 1$. This makes it impossible to lookup, since the index is random.

A good hash function satisfies (approximately) the **Simple Uniform Hashing Assumption** (SUHA): each key is equally likely to hash to any of the m slots, independently of where any other key has hashed to.
Given table of size m, a simple uniform hash h satisfies

- **Uniform**: All keys equally likely to hash to any position.

$$\mathbb{P}(h[k]) = \frac{1}{m}.$$

- **Independence**: All keys' hash values are independent of the other keys.

$$\mathbb{P}(h[k_1] = i \mid h[k_2] = i) = \mathbb{P}(h[k_1] = i).$$

Combining these two properties implies that for any two distinct keys, the probability that they collide is $\frac{1}{m}$:

$$\forall k_1, k_2 \in U \text{ where } k_1 \neq k_2, \quad \mathbb{P}(h[k_1] = h[k_2]) = \frac{1}{m}.$$

Example. You are given n keys. You want to hash these keys perfectly to m slots, where $m \geq n$. This means that there should be no collisions between the keys. What is the probability that a random function from the n keys to the m slots achieves this?

Solution. The total number of hashing outcomes is n^m. If there are no collisions between the keys, then the hash values are all distinct; there are $m(m-1)\cdots(m-n+1) = \frac{m!}{(m-n)!}$ such outcomes. Hence, the probability is $\frac{m!}{m^n(m-n)!}$. □

Example. If you choose a random hash function from n keys to m slots, what is the expected number of pairs of distinct keys that collide?

Solution. For each pair of distinct keys (i, j), define

$$X_{ij} = \begin{cases} 1 & h(i) = h(j) \\ 0 & \text{otherwise.} \end{cases}$$

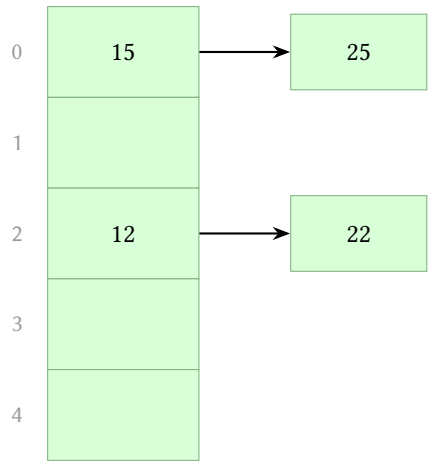
Since the probability of two distinct keys that collide is $\frac{1}{m}$, we have $\mathbb{E}[X_{ij}] = \frac{1}{m}$.
There are $\binom{n}{2} = \frac{n(n-1)}{2}$ unordered pairs of distinct keys. Let X be the number of pairs of distinct keys that collide. Thus,

$$\mathbb{E}[X] = \mathbb{E}[\sum_{1 \leq i < j \leq n} X_{ij}] = \sum_{i<j} \mathbb{E}[X_{ij}] = \binom{n}{2} \cdot \frac{1}{m} = \frac{n(n-1)}{2m}.$$

□

7.2 Separate Chaining

Every slot in the table is for a **linked list**. Each list is initially empty.
When multiple elements are hashed into the same slot index, then these elements are inserted into a singly-linked list, known as a **chain**.



Lookup

To look up item x,

(1) Evaluate the hash $h(x)$

(2) Look up the list at index $h(x) \mod m$.

(3) Check if x exists in that list.

If it does, return the value associated with x. Otherwise, return null.

In the worst case, all n items are on the same chain. Then lookup is the same as searching on a linked list; this takes $O(n)$ time in worst case.

Insertion

To insert item x,

(1) Evaluate the hash $h(x)$

(2) Look up the list at index $h(x) \mod m$.

(3) Check if x exists in that list.

If it does, overwrite the existing value associated with key x. Otherwise, append $(x, value)$ to the list.

In the worst case, all n items are on the same chain. Then insertion is the same as insertion on a linked list; this takes $O(n)$ time in worst case.

Deletion

To delete item x,

(1) Evaluate the hash $h(x)$

(2) Look up the list at index $h(x) \mod m$.

(3) Check if x exists in the list.

If it does, delete it.

In the worst case, all n items are on the same chain. Then deletion is the same as deletion on a linked list; this takes $O(n)$ time in worst case.

Complexity

Definition 7.1. Given a hash table T with m slots that stores n elements, we define the **load factor** α for T as n/m, that is, the average number of elements stored in a chain.

Remark. It is far more likely that two items will hash into separate chains, with probability $1 - \frac{1}{m}$. This probability is high especially if m is large.

Theorem 7.2. *Under SUHA, the expected length of a chain is* $\frac{n}{m}$.

Proof. Let α_j denote the number of items hashing to position j. Then

$$\alpha_j = \sum_{i=1}^{n} H_{i,j},$$

where

$$H_{i,j} = \begin{cases} 1 & \text{if item } i \text{ hashes to } j \\ 0 & \text{otherwise} \end{cases}$$

Then

$$\begin{aligned} \mathbb{E}[\alpha_j] &= \mathbb{E}[\sum_{i=1}^{n} H_{i,j}] = \sum_{i=1}^{n} \mathbb{E}[H_{i,j}] \\ &= \sum_{i=1}^{n} \mathrm{P}[H_{i,j} = 1] \times 1 + \mathrm{P}[H_{i,j} = 0] \times 0 \\ &= n \times \mathrm{P}[H_{i,j} = 1]. \end{aligned}$$

Since $\mathrm{P}[H_{i,j} = 1] = \frac{1}{m}$, we conclude that $\mathbb{E}[\alpha_j] = \frac{n}{m}$. □

As long as $n/m = O(1)$, this means that the expected runtime of our three operations will be $O(1)$. We call n/m the **load factor**.

This means that we do not expect the chain length to far exceed n/m, so the expected runtime of lookup, insertion, and deletion is $O(n/m)$.

7.3 Open Addressing - Linear Probing

In **open addressing**, we store the key-value pairs within the table itself instead of on a separate chain.

In **linear probing**, the hash table is searched sequentially that starts from the original location of the hash. If in case the slot is occupied, then we check the next slot, and so on until we find an empty slot.

Lookup

To look up a key k,

(1) Start at index $h(k) \mod m$.

(2) Iterate until we either find an empty table slot or we have tried all m slots.

(3) If we find the slot where k is stored, return the required value. Otherwise, return null.

Insertion

To insert a key-value pair (k, v),

(1) If size $= m$, notify the user of an error.

(2) Start at index $h(k) \mod m$.

(3) Iterate until we either find an empty table slot or the slot that holds k.

(4) Store (k, v) at the slot we found.

(5) Update size by adding 1 if we inserted into an empty slot.

Deletion

When we remove a key from the table, we run the risk of breaking another "run". Thus, when we delete a key, we remember its current index idx. Then we keep going and see if any items have hash value $\leq idx$. If it does, put that item into index idx. Repeat.
To delete a key k,

(1) Start at index $h(k) \mod m$.

(2) Iterate for at most m iterations until either we find an empty table slot or the slot that holds k.

If it is empty, then k does not exist in the table. Thus, we just return.

If we found the slot that holds k, delete it and set idx to remember our current index. Decrement $size$.

(3) Iterate forwards until we find an element with key k' such that $h(k') \leq idx$.

Move it into slot idx. Update idx to the current index. Continue iterating.

7.4 Dynamic Table Sizing & Amortized Analysis

An array is a fixed-size contiguous sequence of elements. However, it is extremely common for programmers to not know exactly how large an array needs to be up front. Instead, they need something more general than this, an array where you can increase the size and add new elements over time.
A **dynamic array** is an array that automatically resizes itself based on how many items are currently stored. It stores

- capacity c: how many elements it can fit
- size n: how many elements it currently has

The standard solution is to use **doubling resizing**.

Case 1: If $n < c$, simply append elements to the array, i.e., insert the new element in the next available position, increment n.

Case 2: If $n = c$ (the current array is full), we do the following **resizing strategy**:

(1) Allocate a new array of size $2c$ (twice the size of the old array)

(2) Copy over all n existing elements from the old array

(3) Delete the old array

(4) Append the new element.

Note that Step 1 costs $O(n)$ time, Step 2 costs $O(n)$ time. Thus, the worst-case runtime of append is $O(n)$.
Suppose there is a sequence of n operations $o_1, o_2, \ldots, o_n$. Let $f(n)$ be the worst-case time complexity of any operation. Let $t(i)$ be the time complexity of the i-th operation o_i, and let $T(n)$ be the time complexity of all n operations:

$$T(n) = \sum_{i=1}^{n} t(i).$$

Without amortized analysis, if we only think worst-case each time, we may conclude that:

$$T(n) = n \cdot f(n)$$

However, $n \cdot f(n)$ could be grossly wrong (i.e., too high/not tight).

amortized analysis is a strategy for analysing a sequence of operations to show that the average cost per operation is small, although a few operations within the sequence might be expensive.

Definition 7.3. In the **bankers method**, we charge the i-th operation a fictitious **amortized cost** $c(i)$.

This fee is consumed to perform the operation, and any amount that is not immediately consumed is stored in the bank to be used by subsequent operations.

The idea: impose an extra charge on inexpensive (but frequent) operations and use it to pay for expensive (but rare) operations later on. At all times, we must ensure that the bank balance must not go negative, i.e., $\sum_{i=1}^{n} t(i) \leq \sum_{i=1}^{n} c(i)$ for all n.

Thus, the total amortized costs provides an upper bound on the total true costs.

Theorem 7.4. *The amortized cost of append is 3.*

Proof. We will use the following charging scheme: Whenever we insert a new element into the array, we will pay a credit of 2 and leave it on the list element. Therefore, the cost of an insertion not counting any resizing operation is 3.

Now consider what happens when a resizing operation is triggered. The array must be full ($c = n$), and the final half of the elements ($n/2$ of them) each possess an unused credit of 2. Therefore, there is a credit of n in the data structure. We consume this credit to pay for the cost of moving the n elements to the new array (which costs exactly n), so the amortized cost of the resizing procedure is zero.

Hence, the amortized cost of any append operation, whether it triggers a resizing or not is always 3. □

7.5 Hashing Problems

How to use hash tables

7.5.1 Find Most Frequently Occuring

Input: An array A of n integers.

Output: The most frequently occurring elements.

Algorithm:

(1) Traverse the array once, while maintaining a hash table *counts* that records the frequency of each element encountered.

For each key-value pair, the key is the value of the element, the value is the **frequency** of the element.

(2) Retrieve all key-value pairs, find maximum frequency, then output all keys with that frequency.

Algorithm 7.5 Find most frequently occuring

```
1:  Initialise an empty hash table counts
2:  for each element e in A do
3:      if Lookup(counts, e) = null then                ▷ If e is not a key in counts,
4:          Insert(e, 0)                                ▷ insert key e with value 0 into counts
5:      v ← Lookup(e)
6:      Insert(e, v + 1)                                ▷ Increment frequency by 1
7:  P ← ListPairs(counts)                               ▷ Retrieve all key-value pairs
8:  maxFreq ← 0                                         ▷ Find maximum frequency
9:  for each (k, v) in P do
10:     if v > maxFreq then
11:         maxFreq ← v
12: for each (k, v) in P do                             ▷ Output keys with maximum frequency
13:     if v = maxFreq then
14:         output k
```

Time complexity:

- Expected time: $O(n)$, because each insert and lookup in the hash table takes expected time $O(1)$.
- Worst-case time: $O(n^2)$ if all keys collide, since then each insertion or lookup takes worst-case time $O(n)$.
- Extracting the maximum frequency takes $O(n)$ time, since we loop through all key-value pairs.

7.5.2 Find Uniques

Input: An array A of n integers.
Output: A list of integers that occur uniquely in A.
Algorithm:

(1) Traverse the array once, while maintaining a hash table *counts* that records the frequency of each element encountered. (Similar to first half of Problem 1.)

(2) Retrieve all key-value pairs, find keys with value = 1.

Algorithm 7.6 Unique elements

```
1:  Initialise an empty hash table counts
2:  for each element e in A do                    ▷ Iterate from first to last element of A
3:      if LOOKUP(e) = null then                  ▷ If e is not a key in counts,
4:          INSERT(e, 0)                          ▷ insert key e with value 0 into counts
5:      v ← LOOKUP(e)
6:      INSERT(e, v + 1)                          ▷ Increment frequency by 1
7:  P ← LISTPAIRS(counts)                         ▷ Retrieve all key-value pairs
8:  for each (k, v) in P do
9:      if v = 1 then
10:         output k
```

7.5.3 Longest Sub-array with Unique Elements

Input: An array A of n integers.
Output: Indices start and end such that $A[\text{start}, \text{end})$ is the **longest** possible sub-array with only unique elements.

The approach we adopt is called **sliding window**.

(1) Initialise window indices: Initialise start and end pointers to the first element of the array.

(2) Expand the window: Check a condition to determine whether to expand the window. If the condition is satisfied, increment the end pointer.

(3) Process the window: Once the window size meets the desired criteria or condition, perform the required computations or operations on the elements within the window.

(4) Adjust the window size: If the window size exceeds the desired criteria, adjust the window by moving the start pointer. Repeat until the window size matches the desired criteria, and update the window accordingly.

Invariant: The sub-array $A[\text{left}, \text{right})$ contains only unique elements, and is as large as possible.
Algorithm:

(1) Maintain two pointers left and right, and a hash table tracking elements in the current window.

(2) Advance right by one. If $A[\text{right}]$ is already in the window, advance left until all elements in $A[\text{left}, \text{right})$ are unique again.

(3) Update the hash table to reflect the current window.

Algorithm 7.7 Find Longest Subarray with Unique Elements

```
1:  max_pair ← [0, 0)
2:  left ← 0, right ← 0                                  ▷ Initialise pointers
3:  seen ← empty hash table                              ▷ Initialise hash table
4:  while right < n do
5:      while LOOKUP(seen, arr[right]) ≠ null do         ▷ If arr[right] is already in window,
6:          DELETE(seen, arr[left])                      ▷ remove arr[left] from hash table,
7:          left ← left + 1                              ▷ advance left pointer
8:      INSERT(seen, arr[right])                         ▷ Else, insert arr[right] into hash table
9:      right ← right + 1                                ▷ Advance right pointer
10:     Update max_pair with [left, right) if this subarray is larger
11: return max_pair
```

Since we advance left at most n times, and advance right at most n times. We are making at most n lookup, insert and remove operations. Hence, the expected time is $n \times O(1) = O(n)$.

Remark. This is optimal because the input must be scanned at least once.

7.5.4 Rabin Karp - For string searching*

Problem: Given two strings s and pattern, consisting of lowercase English alphabets, find all starting indices where pattern occurs as a substring in s.

Let n be the length of s, m be the length of pattern.

The naive solution is to compare pattern against all positions in s, which runs in $O(mn)$ time.

Rabin-Karp algorithm:

(1) Compute the hash of pattern.

(2) Compare pattern's hash with the hashes of all substrings (of same length as pattern) of s.

If the hashes match, then do a character-by-character check to confirm (to avoid errors due to hash collisions).

```
1: function RabinKarp(string s[1.. n], string pattern[1.. m])
2:     hpattern := hash(pattern[1.. m]);
3:     for i from 1 to n−m+1
4:         hs := hash(s[i .. i+m−1])
5:         if hs = hpattern
6:             if s[i .. i+m−1] = pattern[1.. m]
7:                 return i
8:     return not found
```

The expected runtime is $O(n + m)$. Computing the hash of pattern and the first substring of text takes $O(m)$, and sliding the window over the text with hash comparisons takes expected $O(n)$. Character-by-character check is rarely needed.

The worst-case runtime is $O(nm)$. If many hash collisions occur, each substring may require a full character comparison.

8 Binary Heap

Sometimes it's useful for data structures to return a reference after you've done an insertion. That way, when you need to change the value itself or remove it efficiently, you can pass in the reference that you were given.

A **priority queue** contains **priority-value pairs**.

Basically if we insert a bunch of values, each with a priority. A priority queue tends to be able to give you the value with the **smallest priority** (and is able to remove that one from the queue).

we're going to implement the priority queue API, using a binary heap data structure.

As usual, since values are less important than priorities, we will only illustrate our binary heaps today with priorities.

8.1 Organisation

Definition 8.1. A **binary heap** organises its nodes in the form of complete binary tree. A **complete binary tree** is a tree where (informally):

(i) Every node has either 0, 1, or 2 children.

(ii) Every level of the tree is **full** except the last level (which does not need to be).

(iii) The last level is filled from left to right.

There are two types of binary heaps:

- **Min heap**: The priorities of parent ≤ children for all subtrees.
- **Max heap**: The priorities of parent ≥ children for all subtrees.

By default, we will work with min heaps. Binary heap invariant:

The priorities of the children are ≥ the priority of the parent.

Based on the invariant,

- the node with the maximum priority must be some leaf node;
- the node with the minimum priority must be the root node.

Array-Backed Implementation

Although conceptually a tree, the priority-value pairs of the heap are stored in an array. Nodes are labelled from left to right starting at index 1.

```
class BinaryHeap {
  ArrayList<Pair<Priority, Value>> array;
  int running_id;
}
```

For a node at index i:

$$\text{left child} = 2i, \quad \text{right child} = 2i + 1, \quad \text{parent} = \lfloor i/2 \rfloor.$$

Three-array design: To support efficient updates using IDs, three arrays are maintained:

- main_arr: stores priority-value pairs
- id_to_pos: maps ID → position in heap
- pos_to_id: maps position → ID

These arrays are updated whenever nodes are swapped.

Height

Theorem 8.2. *The height of a binary heap is $O(\log n)$.*

Proof. Since a binary heap is a binary tree, it fulfils the AVL height invariant. Thus, its height is $O(\log n)$. □

8.2 Operations

Insertion

To insert a new element:

(1) Note that the first empty position is always at index $size + 1$.

Thus, insert the element at index $size + 1$ to maintain shape.

(2) **Bubble up**: Restore heap order by keep swapping with its parent until either it is ≥ than its parent node, or it is the root node.

(3) Assign and return a unique ID.

Peek-Min

Since the minimum element is stored at the root, peek-min returns the element at index 1. This takes $O(1)$ time.

Decrease-Key

To decrease the priority of an element:

(1) Locate the node using its ID.

(2) Update its priority.

(3) Bubble up until heap order is restored.

Since priorities only decrease, the node can only move **upward**.

Extract-Min

To remove the minimum element (i.e., root node):

(1) Swap the root node with the last node - now the shape is maintained.

(2) Remove the last node.

(3) **Bubble down**: If root node violates ordering invariant, restore heap order by swapping with the *smaller* of its two children, until leaf node or ≥ both its children.

Heapify

Heapify constructs a heap from an existing array.

(1) Shift elements down by 1, so that the array indexing starts at 1.

(2) The right-most internal node has index $size/2$. Apply bubble-down at indices $size/2, \ldots, 1$.

Remark. Only internal nodes need to be processed.

Time Complexity

We count in the worst case how many swaps each operation does. Each swap takes $O(1)$ time.
Therefore:

- Insert: $O(\log n)$, since insert sends one node up the tree once
- Decrease-key: $O(\log n)$, same remarks
- Extract-min: $O(\log n)$, since it bubbles a node down the tree
- Peek-min: $O(1)$, access the first index
- Size: $O(1)$, access a variable
- Heapify runs in $O(n)$ time, instead of $O(n \log n)$ as expected. See amortised analysis.

Amortised Analysis of Heapify

We analyse the total cost of running BubbleDown on all internal nodes.
Observations:

- The cost of heapifying a complete binary tree is at most the cost for a *full complete binary tree* of the same height. (A full tree contains the maximum number of nodes at each level, hence yields a worst-case upper bound.)
- The worst-case cost of BubbleDown on a node is bounded by the height of that node, i.e., its distance to a leaf.
- In a full complete binary tree, the worst case cost of BubbleDown is the same, regardless of the path, since the distance is the same either way.

Credit scheme: Assign two credits to every node. Whenever a swap occurs during BubbleDown, we charge the child node one credit.
Since we are only counting costs, the path does not matter. Thus, for every node, BubbleDown follows this path: swap it once with its left child, thereafter only swapping with its right child.

Theorem 8.3. *We can actually formally prove that we'll never run out of dollars (but for the sake of time we won't). But suffice it to say this shows we need at most 2 swaps per node! So O(n) swaps needed. So heapify is in O(n) time!*

Because each swap moves a node closer to a leaf, the credits are sufficient to pay for all swaps. Thus each node contributes at most a constant number of swaps.
Hence the total number of swaps across all nodes is $O(n)$.
Therefore, heapify runs in $O(n)$.

9 Graphs

Preliminaries

For simplicity, we shall make the following core simplifying assumptions:

- The vertices are numbered $1, \ldots, |V|$ in some arbitrary manner.
- Unless otherwise specified, our graphs may be directed/undirected, and weighted/unweighted.

9.1 Representations of Graphs

Typical operations to support:

- Is-Adjacent(i, j): Returns whether there is an edge from node i to node j, and the weight (if any).
- Get-Neighbours(i): Returns an array of all the neighbouring nodes, and weights to those neighbours (if any).
- In-Degree(i): Returns the in-degree of a node i, i.e., number of edges pointing to node i.
- Out-Degree(i): Returns the out-degree of a node i, i.e., number of edges that leave node i.

Adjacency Matrices

Given a graph of n nodes and m edges, the **adjacency matrix** representation of the graph is an $n \times n$ matrix $A = (a_{ij})$ such that

$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

If the graph is weighted, then a_{ij} store the weights:

$$a_{ij} = \begin{cases} w(i, j) & \text{if } (i, j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Remark. In actual implementation, the $n \times n$ matrix is stored as an array of n arrays, where each array is n elements long.

Remark. In an undirected graph, (u, v) and (v, u) represent the same edge, so the adjacency matrix A of an undirected graph is symmetric. In some applications, it pays to store only the entries on and above the diagonal of the adjacency matrix, thereby cutting the memory needed to store the graph by half.

We now implement the operations:

- Is-Adjacent(i, j): Lookup a_{ij}. Return true if $a_{ij} = 1$, false if $a_{ij} = 0$.

  The runtime is $O(1)$, since accessing an element of an array can be done in constant time.
- Get-Neighbours(i): Iterate on the i-th row, filter out indices j for which $a_{ij} \neq 0$.

  The runtime is $O(n)$, since we iterate over a row of n elements.
- In-Degree(i): Iterate on the i-th column, count the number of indices j such that $a_{ji} \neq 0$.

  The runtime is $O(n)$, since we iterate over a column of n elements.
- Out-Degree(i): Iterate on the i-th row, count the number of indices j such that $a_{ij} \neq 0$.

  The runtime is $O(n)$, since we iterate over a row of n elements.

An adjacency matrix stores n^2 integers, so it uses $O(n^2)$ space.

Adjacency List

Given a graph of n nodes and m edges, the **adjacency list** representation of the graph is an array Adj of n arrays, one for each vertex.

For each $u \in V$, the adjacency list $Adj[u]$ contains all the vertices v such that there is an edge $(u, v) \in E$; that is, $Adj[u]$ consists of all the vertices adjacent to u.

[figure]

We support the following operations:

- Is-Adjacent(i, j): Access $Adj[i]$. Check if j is in $Adj[i]$.

  If $Adj[i]$ is sorted, then performing binary search on $Adj[i]$ of length $\deg(i)$ costs $O(\log \deg(i))$ time. If $Adj[i]$ is not sorted, then we do a linear pass on the array, which costs $O(\deg(i))$ time.

- Get-Neighbours(i): Return $Adj[i]$.

- In-Degree(i): Go through all the nodes, and count the number of nodes that point to node i.

  If lists are not sorted, then the runtime is $O(n + m)$, where $O(n)$ to go through each node, $O(m)$ to go through all the edges.

  If lists are sorted, then the runtime $O(n \log(m/n))$ by running binary search (exercise: prove this)

- Out-Degree(i): Return the size of $Adj[i]$.

Remark. By default, the neighbour lists are **unsorted**. (There is typically little reason to assume so, and it requires the nodes to come with comparable IDs.)

An adjacency list stores n lists, and the total size of the sub-lists is m elements. Thus, it uses $O(n + m)$ space.

Edge List

Given a graph of n nodes and m edges, we store it as an array E of m elements.

- If the graph is unweighted, then its elements are pairs (u, v) that represent the edge (u, v).
- If the graph is weighted, then its elements are triples (u, v, w) that represent the edge (u, v) with weight w.

We support the following operations:

- Is-Adjacent(i, j): Check whether $(i, j) \in E$.

  Since we do a linear pass on E, the runtime is $O(m)$.

- Get-Neighbours(i): Filter E for pairs such that the first coordinate is i.

  Since we do a linear pass on E, the runtime is $O(m)$.

- In-Degree(i): Filter E for pairs such that the second coordinate is i. Return the size of the filtered list.

  Since we do a linear pass on E, the runtime is $O(m)$.

- Out-Degree(i): Filter E for pairs such that the first coordinate is i. Return the size of the filtered list.

  Since we do a linear pass on E, the runtime is $O(m)$.

An edge lists only stores the edges, so it uses $O(m)$ space.

Summary

	Adjacency matrix	Adjacency list	Edge list
IsAdjacent	$O(1)$	$O(\deg(i))$	$O(m)$
GetNeighbours	$O(n)$	$O(1)$	$O(m)$
InDegreeOf	$O(n)$	$O(n+m)$	$O(m)$
OutDegreeOf	$O(n)$	$O(1)$	$O(m)$
Space	$O(n^2)$	$O(n+m)$	$O(m)$

Depending on what you want to optimise for, and what operations you use, different data structures might be preferred.

9.2 Graph Traversal

9.2.1 Depth-First Traverse

Input: A graph $G = (V, E)$ and a source node s. (Note: the graph may not be connected or undirected.)
Output: A list of nodes that can be visited starting from node s.
Algorithm:

(1) Initialize a list $visited$ of length $|V|$, with all values set to **false**. It tracks the vertices that we have already visited.

(2) Start the traversal at the source vertex s. Mark s as visited.

(3) Retrieve the list of neighbours of vertex s. For each neighbour w of s, if n is not visited, recursively on w.

(4) After all neighbours of s have been processed, output all vertices v for which $visited[v]$ = **true**.

Algorithm 9.1 Depth-First Traverse

```
1: visited = array of length |V| full of false
2: procedure DFT(G, v)
3:     visited[v] = true
4:     for all w adjacent to v do
5:         if visited[w] == false then
6:             DFT(G, w)
7: DFT(G, s)
```

Theorem 9.2. *The runtime of Depth-First Traverse is $O(V + E)$.*

Proof. In DFS, each vertex is visited at most once, and for each vertex v, we iterate over its neighbours.

- The initialisation of the $visited$ array takes $O(V)$.
- Marking a vertex as visited takes $O(1)$ time. Since each vertex is visited only once, this takes $O(V)$ time.
- Assuming the graph is stored using an adjacency list, it costs $O(1)$ to get the neighbour list of a vertex.

  For each vertex v, we iterate through its neighbours, so at most $\deg(v)$ iterations. In the worst case, we visit all the nodes, so iterate on all of its neighbours:

  $$\sum_{v \in V} \deg(v) = 2|E| = O(E)$$

  by the handshake theorem. Since there are $O(E)$ iterations, and each iteration costs $O(1)$, this costs $O(E)$.

Hence, the time complexity is $O(V) + O(V) + O(E) = O(V + E)$. □

9.2.2 Breadth-First Traverse

Notice that depth-first search tends to keep exploring a path until it cannot continue, then it breaks out of the recursion and "backtracks" up its path to find another path to explore. So it goes for **depth** first.

Breadth-first takes the other approach where it explores path based on **breadth**. That is, we visit all the nodes of distance 0, then all nodes of distance 1, then all nodes of distance 2, and so on.

Algorithm:

(1) Initialize a boolean list $visited$ of length $|V|$ full of **false**, which indicates whether a vertex has been visited or not.

Initialize a (FIFO) queue $Q = [s]$ to only hold vertex s. The queue stores the vertices to visit.

(2) Mark s as visited.

(3) While the queue is not empty (i.e., we still have vertices to visit), dequeue x from Q.

Find all neighbours of x which have not been visited, and add them into Q, mark them as visited.

Algorithm 9.3 Breadth-First Traverse

```
1:  procedure BFT(G, s)
2:      Q = [s]
3:      visited = array of length V full of false
4:      visited[s] = true
5:      while Q ≠ ∅ do
6:          x = DEQUEUE(Q)
7:          for all y adjacent to x do
8:              if visited[y] == false then
9:                  ENQUEUE(Q, y)
10:                 visited[y] = true
11:     return visited
```

Theorem 9.4. *The runtime of Breadth-First Traverse is $O(V + E)$.*

Proof. Similar to DFT.

- The overhead for initialization is $O(V)$.
- Each vertex is enqueued and dequeued at most once. The operations of enqueuing and dequeuing take $O(1)$ time, and so the total time devoted to queue operations is $O(V)$.
- Since the procedure scans the adjacency list of each vertex only when the vertex is dequeued, it scans each adjacency list at most once. Since the sum of the lengths of all the adjacency lists is $2E$, the total time spent in scanning adjacency lists is $O(E)$.

Therefore, the total running time of BFT is $O(V + E)$. □

9.3 Adapting BFT

There are many graph problems that have the same core idea, but are distinct enough that you can't simply run DFS or BFS. Essentially, you need to **adopt** the graph traversal algorithm (by adding a few lines here and there) to actually properly solve the problem.

9.3.1 Connectedness

Problem: Given an undirected graph $G = (V, E)$, determine if the graph is connected.

Idea: If the graph is connected, then we can pick any node and run BFS/DFS from it, and every other node must be reachable from that starting node.

9.3.2 s-t reachability

Problem: Given a directed graph $G = (V, E)$, a source vertex s and a target vertex t, return **true** if there exists a path from s to t, and **false** if otherwise.

Idea: Run BFT from s, and check if t is ever visited.

Algorithm 9.5 s-t Reachability

```
1:  procedure REACHABLE(G, s, t)
2:      Q = [s]
3:      visited = array of length |V| full of false
4:      visited[s] = true
5:      while Q ≠ ∅ do
6:          x = DEQUEUE(Q)
7:          if x == t then                          ▷ Termination condition: t is visited
8:              return true
9:          for all y adjacent to x do
10:             if visited[y] == false then
11:                 ENQUEUE(Q, y)
12:                 visited[y] = true
13:     return false
```

Problem: Given a directed graph $G = (V, E)$, return **true** if every vertex can reach every other vertex, and **false** if otherwise.

Idea:

(1) Pick any node v, run BFS/DFS from v to see if it can reach every node.

(2) Flip all the edges. Run BFS/DFS again.

The first traversal tells you whether v can reach every vertex, and the second traversal tells you whether every vertex can reach v. Then for any two vertices s and t, we can reach $s \to v \to t$ by passing through v.

9.3.3 Cycle detection in undirected graphs

Problem: Given an undirected graph $G = (V, E)$, return **true** if a cycle exists in the graph, **false** if it doesn't.

Idea: Traverse from a node, and if we revisit itself again, then output **true**; otherwise, output **false**. We also need to remember the nodes that we have visited before, so that we only start a traversal from unvisited nodes, to avoid redundant traversals.

Algorithm 9.6 Cycle Detection

```
1:  visited = array of length V full of false      ▷ Make this global, to use across all calls
2:  procedure BFT(G, s)
3:      Q = [s]
4:      visited[s] = true
5:      while Q ≠ ∅ do
6:          x = DEQUEUE(Q)
7:          if visited[x] == true then              ▷ Key modification
8:              return true
9:          for all y adjacent to x do
10:             if visited[y] == false then
11:                 ENQUEUE(Q, y)
12:                 visited[y] = true
13:     return visited
14: for each v ∈ V do
15:     BFT(G, v)
```

If any call returns true, we know a cycle exists. Hence, $O(V + E)$ time.

9.3.4 Graph colouring / flood fill

Problem: Given an undirected graph $G = (V, E)$, output an array component_id of n integers, such that nodes u and v belong in the same connected component if and only if component_id[u] = component_id[v].

Furthermore, the numbers across connected components should be different.

Idea: Re-use the same traversal algorithm as before (a BFS that tries to visit every node in the graph, marks out every node in a connected component as visited), but:

- Before we start, set a running id current_id = 1
- Every time we need to do a fresh traversal, set every node in that component to have current_id. Then increment current_id by 1.

Algorithm 9.7 Graph Colouring

```
1:  id = 1
2:  component_id = array of length V
3:  visited = array of length V full of false
4:  procedure Colour(G, s)
5:      Q = [s]
6:      visited[s] = true
7:      while Q ≠ ∅ do
8:          x = Dequeue(Q)
9:          component_id[x] = id                          ▷ Set id
10:         for all y adjacent to x do
11:             if visited[y] == false then
12:                 Enqueue(Q, y)
13:                 visited[y] = true
14:     return visited
15: for each v ∈ V do
16:     if visited[v] == false then
17:         Colour(G, v)
18:     id + +
```

9.3.5 Counting components

Problem: Given an undirected graph $G = (V, E)$, count the number of connected components in G.

Idea: Run the graph colouring algorithm, and output the maximum possible id.

Algorithm 9.8 Counting Components

```
1: id = 1
2: component_id = array of length V
3: visited = array of length V full of false
4: for each v ∈ V do
5:     if visited[v] == false then
6:         Colour(G, v)
7:     id + +
8: return id − 1
```

9.3.6 Bipartite graph

A graph is **bipartite** if its vertices can be partitioned into two disjoint sets such that no edge connects two vertices within the same set.

Problem: Given an undirected graph $G = (V, E)$, determine if G is a bipartite graph.

A graph is bipartite if and only if there is a valid 2-coloring. Use DFS or BFS to traverse the graph while assigning colors to the vertices. Start from any node and assign it color 0 (WLOG). Then assign all of its neighbours color 1.

Continue traversing the graph, alternating colors for each newly visited vertex (i.e., a vertex's neighbors should receive the opposite color). During the traversal, if we encounter an edge that connects two vertices with the same color this indicates that the graph cannot be properly two-colored, and therefore the graph is not bipartite. If the traversal completes without any such conflict, the graph is bipartite.

9.3.7 Diameter of tree

Problem: Given an undirected tree T, output a pair of nodes $u, v \in T$ such that their distance between them is maximized.

Idea:

(1) Start from any node and run BFS/DFS to find the furthest node v.

(2) Then run BFS/DFS again from that node to find the furthest node u from v.

9.4 Adapting DFT

When visiting a node x, we might want to:

(1) Do something before visiting our neighbours (pre-order operations)

(2) Do something after visiting our neighbours (post-order operations)

These are the two parts that we will modify.

Algorithm 9.9 Depth-First Traverse general template

```
1: visited = array of length |V| full of false
2: procedure DFT(G, v)
3:     visited[v] = true
4:     ▷ Pre-order operations ◁
5:     for all w adjacent to v do
6:         if visited[w] == false then
7:             DFT(G, w)
8:     ▷ Post-order operations ◁
9: DFT(G, s)
```

9.4.1 Cycle detection in directed graphs

Problem: Given a directed graph $G = (V, E)$, return **true** if the graph has a directed cycle, false if otherwise.

Observation: If we have visited a node, but then returned the recursion on it, then it is not part of a cycle. If we are within an active recursive call, and visited a node twice, then there is a cycle.

Algorithm:

(1) Start a depth-first search (DFS) from the source node s.

Maintain a boolean array $visited[1..n]$ to keep track of nodes that have been visited.

Maintain a boolean array $in\_stack[1..n]$ to record whether a node is currently in the recursion stack.

(2) If s has already been visited, return true if s is currently in the recursion stack, and false otherwise.

(3) Otherwise, mark s as visited, and mark it as being in the recursion stack.

(4) For each neighbour n of s, recursively perform DFS on n.

(5) If any recursive call returns true, propagate true upwards.

(6) After exploring all neighbours, pop s from the recursion stack.

(7) Return false.

Algorithm 9.10 Cycle detection in directed graphs

```
1:  visited ← list of length |V| full of false
2:  in_stack ← list of length |V| full of false
3:  procedure CycleDetection(G, v)
4:      visited[v] ← true
5:      in_stack[v] ← true
6:      for all w adjacent to v do
7:          if visited[w] = false then
8:              if CycleDetection(G, w) then
9:                  return true
10:         else if in_stack[w] = true then
11:             return true
12:     in_stack[v] ← false
13:     return false
14: for each vertex v ∈ V do
15:     if visited[v] = false then
16:         if CycleDetection(G, v) then
17:             return true
18: return false
```

Although the algorithm contains an outer loop over all vertices, the total running time remains linear. Observe that each vertex is visited at most once, and each directed edge is explored at most once during the DFS.

Hence, across all DFS calls, each vertex is processed exactly once, contributing $O(V)$. For each vertex, its adjacency list is scanned once, so all edges are examined exactly once, contributing $O(E)$. Therefore, the total time complexity is $O(V + E)$.

9.4.2 Topological sorting

Definition 9.11. A **topological sort** of a directed, acyclic graph (DAG) $G = (V, E)$ is a linear ordering of all its vertices such that if G contains an edge (u, v), then u appears before v in the ordering.

Remark. In other words, a topological sort is a graph traversal in which each node v is visited only after all its dependencies are visited.

Problem: Given a directed, acyclic graph $G = (V, E)$, output a topological sort of G.

Algorithm:

(1) Initialize a linked list T, which stores all the nodes seen so far in valid topological ordering.

Initialize a visited list.

(2) Start the traversal at the source vertex s. Mark s as visited.

(3) Retrieve the list of neighbours of vertex s. For each neighbour w of s, if w is not visited, recursively perform toposort on w.

(4) After all neighbours of s have been processed, prepend s to T.

(5) Return T.

Algorithm 9.12 Toposort

```
1: T = empty linked list
2: visited = array of length |V| full of false
3: procedure TOPOSORT(G, v)
4:     visited[v] = true
5:     for all w adjacent to v do
6:         if visited[w] == false then
7:             TOPOSORT(G, w)
8:     Prepend v to T
9: TOPOSORT(G, s)
10: return T
```

Remark. We used a linked list for the topolist T, so that prepend takes $O(1)$ time.

9.4.3 Triangles in graph

Problem: Given an undirected graph G, count the number of triangles (3-cycles) in the graph.

Idea: Run DFS from every node u to only depth 3. If there are nodes that link back to node u on that depth, then we get a triangle (need to check that all 3 nodes on our path are different).

Do this for all nodes, then divide the final count by 6.

10 Single Source Shortest Path

The **single source shortest path** (SSSP) problem finds the minimum weight paths from one source vertex to all other vertices in a graph.

10.1 Directed, unweighted (BFS)

Problem: Given a directed, unweighted graph $G = (V, E)$, and a source node s, output an array $dist$ such that for all nodes v, $dist[v]$ is the shortest distance from node s to node v.

The idea is to modify BFT: starting from s, explore the nodes based on nodes with distance 1, then distance 2, and so on.

Algorithm:

(1) Maintain a visited list. Maintain a distance array $dist$. Set $dist[s] = 0$. Initialize a (FIFO) queue.

(2) Enqueue s into the queue. Mark s as visited.

(3) While the queue is not empty, dequeue a node x.

(4) For all neighbours y of x, if y is not visited, update its distance. Mark y as visited. Enqueue y into the queue.

Algorithm 10.1 SSSP on directed, unweighted graph

```
1:  procedure SSSP(G, s)
2:      Q = [s]
3:      visited = array of length |V| full of false
4:      dist = array of length |V|
5:      visited[s] = true, dist[s] = 0
6:      while Q ≠ ∅ do
7:          x = DEQUEUE(Q)
8:          for all y adjacent to x do
9:              if visited[y] == false then
10:                 ENQUEUE(Q, y)
11:                 visited[y] = true
12:                 dist[y] = dist[x] + 1
13:     return d
```

10.2 Directed, potentially negative weights (Bellman-Ford)

Problem: Given a directed weighted graph $G = (V, E)$ **without negative cycles**, and a source node s, output an array $dist$ such that for all nodes v, $dist[v]$ is the shortest distance from node s to node v.

Remark. If the graph has a negative weight cycle, then the shortest path is not well-defined, because you'd rather live in the cycle!

Definition 10.2.

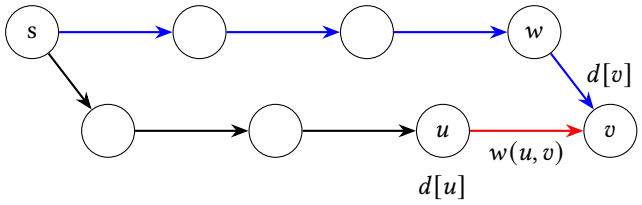
- The distance from u to v, denoted $\delta(u, v)$, is the length of the **actual** shortest path from u to v (if it exists); and is ∞ otherwise.
- For each vertex v, maintain an **estimate** $d[v]$ of the length $\delta(s, v)$ of the shortest path to v, i.e., length of current shortest path from s to v.

Path relaxation: Consider an edge from a vertex u to v with weight $w(u, v)$. Suppose we have already processed u so that we know $d[u] = \delta(s, u)$ and also computed a current estimate for $d[v]$. Then

- There is a (shortest) path from s to u with length $d[v]$.

- There is a path from s to v to length $d[v]$.

Combining this path from s to u with the edge (u, v), we obtain another path from s to v with length $d[u] + w(u, v)$.



If $d[u] + w(u, v) < d[v]$, then replace the old path $\langle s, \ldots, w, v\rangle$ with the new shorter path $\langle s, \ldots, u, v\rangle$. Hence, we update

$$d[v] = d[u] + w(u, v).$$

Path relaxation

```
1: procedure RELAX(u, v)
2:     if d[u] + w(u, v) < d[v] then
3:         d[v] = d[u] + w(u, v)
```

The algorithm relaxes the edges, progressively decreasing the values of $d[v]$ until it reaches the actual shortest path $\delta(s, v)$.

Algorithm:

(1) Initially set $d[s] = 0$ and all other $d[v] = \infty$.

(2) Relax all edges. It is claimed that $V - 1$ iterations of relaxations are sufficient to correctly calculate the lengths of all shortest paths in the graph.

Algorithm 10.3 Bellman Ford

```
1: procedure BELLMAN-FORD(G, s)
2:     ∀v ∈ V, d[v] ← ∞                    ▷ Set initial distance estimates
3:     d[s] ← 0                            ▷ Set distance to source node trivially as 0
4:     for i = 1, ..., n − 1 do
5:         for (u, v) ∈ E do
6:             RELAX(u, v)
```

Theorem 10.4. *The runtime of Bellman-Ford is $O(VE)$.*

Proof. There are $O(V)$ iterations. Each iteration involves relaxing $O(E)$ edges, and each relaxation costs $O(1)$. □

10.2.1 Negative cycle detection

Bellman Ford's algorithm allow us to detect **negative weight cycles** if they are present in the graph.

(1) Run Bellman-Ford's algorithm, which involves $V - 1$ iterations.

(2) Relax all the edges one more time i.e., the V-th time.

(3) If there were no changes on the last iteration, then there is no negative cycle in the graph.

If there are changes, then a negative cycle is present.

10.3 Directed acyclic, potentially negative weights (toposort)

In a DAG, we can find a good way to "order" the edges so that if we relaxed them in this sequence, we get all the correct distances in a single pass.

It turns out that for a DAG, if we toposort the nodes, and relax them in toposort order, we would only need to do this in 1 pass!

Algorithm 10.5 SSSP on DAG

```
1: procedure SSSP-DAG(G, s)
2:     topolist ← TOPOSORT(G, s)
3:     for each node v in topolist do
4:         Relax all outgoing edges for node v
```

Theorem 10.6. *The runtime of SSSP on a DAG is $O(V + E)$.*

Proof. Toposort costs $O(V + E)$. Then, there are $O(V)$ iterations, each costs $O(1)$. □

Theorem 10.7 (Correctness)**.** *At the termination of DAG-Bellman-Ford, $d[v] = \delta(s, v)$ for all $v \in V$.*

Proof. If v is not reachable from s then $d[v] = \infty$ since $d[v] \geq \delta(s, v)$ by the upper bound property. Now assume v is reachable from s and that a shortest path from s to v is $\langle v_0, v_1, \ldots, v_j \rangle$ with $v_0 = s$ and $v_j = v$. Since we have topologically sorted the graph, we process the edges in order $(v_0, v_1), (v_1, v_2), \ldots, (v_{j-1}, v_j)$ since each edge in the path is one farther from s than its preceding edge in the path. Thus by the path relaxation property, $d[v] = \delta(s, v)$. □

10.4 Directed, non-negative weights (Dijkstra)

Problem: Given a directed, non-negatively weighted graph $G = (V, E)$, and a source node s, output an array $dist$ such that for all nodes v, $dist[v]$ is the shortest distance from node s to node v.

We consider a greedy approach, by visiting the node with the smallest distance estimate.

Algorithm:

(1) Maintain a priority queue of the nodes based on their distance estimates, so that we know which node has the smallest distance estimate so far.

(2) Extract a node out of the priority queue (using extract min), relax outgoing edges, thereby reducing distance estimates.

(3) Repeat until the priority queue is empty.

Algorithm 10.8 Dijkstra (classic version)

```
1: procedure DIJKSTRA(G, s)
2:     ∀v ∈ V, d[v] = ∞                          ▷ Set distance estimates
3:     d[s] = 0
4:     Q ← priority queue with all vertices
5:     while Q ≠ ∅ do
6:         u = EXTRACT-MIN(Q)
7:         for each neighbour v of u do
8:             if d[u] + w(u, v) < d[y] then      ▷ If estimate improves
9:                 d[v] = d[v] + w(u, v)          ▷ Relax - update distance estimate for v
10:                DECREASE-KEY(Q, v, d[v])
    return d
```

Theorem 10.9. *The runtime of Dijkstra's algorithm is $O((V + E) \log V)$.*

Proof. We do Extract-Min at most V times, since the heap starts with V vertices. Each Extract-Min operation runs in $O(\log V)$ time.

We do Decrease-Key at most E times, since we relax each outgoing edge at most once. Each Decrease-Key operation runs in $O(\log V)$ time.

Hence, the overall time complexity is $O(V \log V + E \log V)$. □

A practical difficulty with the classic version of Dijkstra's algorithm is the reliance on the Decrease-Key operation. As seen earlier, supporting this efficiently typically requires implementing a heap with additional bookkeeping (such as tracking element indices). In practice, however, most standard libraries in languages such as Java and C/C++ do not provide priority queues with native support for Decrease-Key or arbitrary deletion. They usually only support efficient insertion and Extract-Min.

To address this, we adopt a lazy deletion strategy. Instead of updating priorities within the heap, we simply insert new entries whenever a better distance estimate is found. This means that multiple copies of the same node may coexist in the priority queue, each with different priorities. Importantly, this does not affect correctness because each node is processed only once: the first time it is extracted from the queue corresponds to its shortest distance. Any subsequent extractions of the same node are ignored.

This is handled using a `visited` array. When a node is extracted, we check whether it has already been processed. If so, we skip it; otherwise, we mark it as visited and finalize its distance. In this way, even though duplicates may exist in the heap, only the first occurrence is meaningful.

Algorithm 10.10 Dijkstra (lazy deletion)

```
1:  procedure DIJKSTRA(G, s)
2:      ∀v ∈ V, d[v] ← ∞
3:      d[s] ← 0
4:      Q ← empty priority queue
5:      INSERT(Q, (s, 0))
6:      visited[v] ← false for all v ∈ V
7:      while Q ≠ ∅ do
8:          (u, p) ← EXTRACT-MIN(Q)
9:          if visited[u] then
10:             continue
11:         visited[u] ← true
12:         d[u] ← p
13:         for each neighbour v of u do
14:             w ← w(u, v)
15:             INSERT(Q, (v, d[u] + w))
```

In this version, the priority queue may contain multiple entries for the same node, but its total size is bounded by the number of edges. Each edge can cause at most one insertion, so the heap holds at most $O(E)$ elements. Consequently, there are at most $O(E)$ Insert and Extract-Min operations, each taking $O(\log E)$ time. Therefore, the overall running time is $O(E \log E)$.

10.5 Other Variants of SSSP

10.5.1 Multi-Source BFS

Problem: Given an unweighted graph $G = (V, E)$ and a list of source vertices S, output an array D such that for each vertex v, $D[v]$ is the shortest path distance of vertex v if we could start from any of the vertices in S.

The naive solution is to run BFS from each source vertex, and collect all the distance arrays. Then for all vertices v, the final distance is the minimum across all the arrays. However, the runtime is $O(V(V + E))$, which is slow.

Algorithm:

(1) Create a super-source node, and connect it to all source vertices with edges of weight 0.

(2) Run BFS from the super-source node.

(3) When it's done, subtract all distances by 1.

10.5.2 Teleporting SSSP

Problem: Given a weighted graph $G = (V, E)$ and a set $T \subseteq V$ of vertices where we can teleport between any pair at cost 0, as well as two vertices s and t, output the shortest path from s to t.

The naive idea is to add edges of weight 0 between any pair of teleportation vertices, and then run Dijkstra's algorithm.

However, we added $O(T^2)$ edges, so the runtime becomes $O((E + T^2)\log(E + T^2))$, which is slow if $T \approx V$.

Idea: Instead of forming a complete graph on T, we construct a single cycle connecting all vertices in T in some fixed order. This only adds T edges.

(1) Sort vertices in T by their id.

(2) Let $T = \{v_1, v_2, \ldots, v_k\}$. Add zero-weight edges (v_i, v_{i+1}) for $1 \leq i < k$, and also (v_k, v_1) to complete the cycle.

(3) For all vertices $v \notin T$, no modification is needed.

(4) Run Dijkstra's algorithm from the source vertex s on the modified graph.

The runtime is $O((V + E)\log V)$.

- Sorting the teleporation set T takes $O(V \log V)$.
- Since we added T edges, the resulting graph has V vertices and $E + T$ edges.

  Thus, Dijkstra's algorithm runs in $O((V + E + T)\log V)$, which simplifies to $O((V + E)\log V)$ since $T \leq V$.

10.5.3 SSSP with Expressways

Problem: Given a positively weighted, directed graph $G = (V, E)$, suppose may choose up to k edges whose weight becomes 0. Output the shortest path from s to all other vertices.

Idea: Use **layered graph construction**. We construct $k + 1$ copies (layers) of the graph. Each vertex $v \in V$ is replicated as

$$v_0, v_1, \ldots, v_k,$$

where the index i represents that exactly i free edges have been used.

For every directed edge $(u, v) \in E$ with weight w, we add:

- **Normal traversal:** edges (u_i, v_i) with weight w for all $0 \leq i \leq k$.
- **Free usage:** edges (u_i, v_{i+1}) with weight 0 for all $0 \leq i < k$.

Then moving between layers corresponds to using one of the allowed free edges.

Solution. We run Dijkstra's algorithm on the expanded graph starting from s_0. The answer for each vertex v is

$$\min_{0 \leq i \leq k} \text{dist}(v_i),$$

since we may end at any layer depending on how many free edges were used.

```
1: Dijkstra(G, s, k):
2:   for each v in V and i in [0.. k]:
3:       dist[v][i] = infinity
4:       visited[v][i] = false
5:
```

```
 6:   dist[s][0] = 0
 7:   push (0, s, 0) into min-heap
 8:
 9:   while heap is not empty:
10:       (d, u, i) = extract_min()
11:
12:       if visited[u][i]:
13:           continue
14:       visited[u][i] = true
15:
16:       for each edge (u, v) with weight w:
17:           // normal traversal
18:           if dist[v][i] > d + w:
19:               dist[v][i] = d + w
20:               push (dist[v][i], v, i)
21:
22:           // use free edge if available
23:           if i < k and dist[v][i+1] > d:
24:               dist[v][i+1] = d
25:               push (dist[v][i+1], v, i+1)
```

The layered graph has $(k+1)|V|$ vertices. Each original edge induces $O(k)$ transitions, so the total number of edges is $O(k|E|)$.

Running Dijkstra on this graph takes

$$O\big((k|V| + k|E|)\log(k|V|)\big) = O\big(k(|V| + |E|)\log|V|\big).$$

10.5.4 All pairs shortest path

Problem: Find the shortest paths between every pair of vertices in a weighted graph $G = (V, E)$.

Suppose vertices are numbered $1, 2, \ldots, n$. Let Shortest-Path(i, j, k) be the length of shortest path from i to j using vertices only from the set $\{1, 2, \ldots, k\}$. Then the solution is Shortest-Path(i, j, n) for any $1 \le i, j \le n$.

We can choose to either

- not use vertex k, then the value is Shortest-Path$(i, j, k-1)$;
- use vertex k, then the path can be decomposed as:
  (1) a path from i to k that uses the vertices $\{1, 2, \ldots, k-1\}$, followed by
  (2) a path from k to j that uses the vertices $\{1, 2, \ldots, k-1\}$,

  so the value is Shortest-Path$(i, k, k-1)$ + Shortest-Path$(k, j, k-1)$.

Hence, we obtain the recursive formula

$$\text{Shortest-Path}(i, j, k) = \min \begin{cases} \text{Shortest-Path}(i, j, k-1) \\ \text{Shortest-Path}(i, k, k-1) + \text{Shortest-Path}(k, j, k-1) \end{cases}$$

where the base case is Shortest-Path$(i, j, 0) = w(i, j)$.

The Floyd-Warshall algorithm first computes Shortest-Path(i, j, k) for all (i, j) pairs for $k = 0$, then $k = 1$, $k = 2$ and so on, until $k = n$.

Algorithm 10.11 Floyd-Warshall algorithm

```
1:  D ← n × n array of minimum distances initialized to ∞
2:  for each edge (u, v) do
3:      D[u][v] = w(u, v)
4:  for each vertex v do
5:      D[v][v] = 0
6:  for k = 1, ..., n do
7:      for i = 1, ..., n do
8:          for j = 1, ..., n do
9:              if D[i][j] > D[i][k] + D[k][j] then
10:                 D[i][j] = D[i][k] + D[k][j]
```

The runtime is $O(V^3)$ because of the triple nested **for** loops.

10.6 Problem Solving

There are generally two approaches when solving graph problems:

(1) Modify existing algorithms directly.

(2) Treat algorithms as a black box, but transform the input graph.

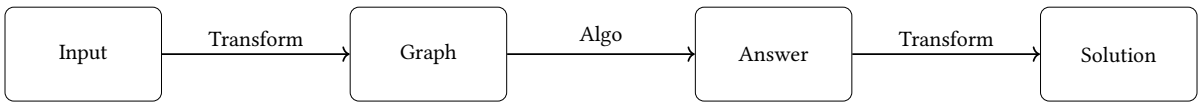
In practice, both approaches complement each other. Conceptually, it is often easier to think in terms of transforming the graph. For implementation, it is often simpler to modify the algorithm itself.

Graph modelling: Many problems can be reduced to graph problems by:

(1) Defining states as nodes.

(2) Defining valid transitions as edges.

(3) Running a suitable graph algorithm (e.g. BFS, Dijkstra, Bellman-Ford).



In addition, there are several common **graph tricks** to extend our graph algorithms to solve a huge class of problems.

(1) Add extra nodes.

(2) Add extra edges.

(3) Copy the graph into multiple layers.

10.6.1 Example: Combination Lock

Problem: Consider a combination lock consisting of d digits, where each digit takes a value from 0 to 9. The lock starts at the configuration $00\cdots0$, and we are given a set B of length n containing forbidden configurations.

In one move, we may increment or decrement a single digit by 1 (with wraparound allowed, i.e., $9 \to 0$ and $0 \to 9$). The objective is to determine the minimum number of moves required to reach a target configuration t, while avoiding all configurations in B.

We model the problem as a graph:

- Each valid configuration is a node.

- There is an edge between two nodes if one move transforms one into the other.
- The graph is undirected and unweighted.

Under this construction, the problem reduces to finding the shortest path from the start node to the target node.
Solution: Run BFS from the starting node.
Algorithm:

(1) Enumerate all d-digit configurations. For each configuration x, include it as a node if $x \notin B$.

(2) For each valid configuration x, generate all possible neighbours y by incrementing or decrementing exactly one digit (with wraparound). If $y \notin B$, add an edge between x and y.

(3) Run BFS starting from the initial configuration $00\cdots0$ and compute the shortest distance to the target configuration t. The BFS distance directly yields the minimum number of moves.

Complexity analysis. The running time depends on how efficiently the forbidden set B is stored. If B is implemented using a hash table, membership queries run in expected $O(1)$ time.
There are 10^d possible configurations. Each configuration has exactly $2d$ neighbors (two directions per digit). Hence, the total number of edges is $O(d \cdot 10^d)$.
Since BFS runs in linear time in the size of the graph, the overall expected complexity is:

$$O(10^d + d \cdot 10^d) = O(d \cdot 10^d).$$

10.6.2 Reachable Negative Cycle Detection

Problem: Given a graph $G = (V, E)$ with potentially negative weights, detect if there is a negative cycle reachable from the source vertex s.
Idea: Use Bellman-Ford. Bellman-Ford tries to reduce distance estimates for a node v, if there is a node u from which: $dist[u] + weight(u, v) < dist[v]$
Algorithm:

(1) Run $n - 1$ rounds of relaxation.

(2) Run one additional round.

(3) If any nodes still have their distance estimates reduce, then there is a negative cycle reachable by s.

Runtime: $O(VE)$.

10.6.3 Advanced Example: Grid Disconnection

Problem: Given an $n \times n$ grid with node weights, find the minimum cost set of vertices whose removal disconnects top-left from bottom-right.
Idea: Transform into a shortest path problem.

- Create a graph where each node contributes its weight as incoming edge weights.
- Add edges from neighbours (including diagonals).
- Introduce a source (top-right) and target (bottom-left).

Solution: Run Dijkstra to find shortest path.
Insight: The shortest path corresponds to the minimum cost cut.
Runtime: $O(n^2 \log n)$.

11 Union-Find Disjoint Sets

11.1 Disjoint-set operations

A **disjoint-set data structure** maintains a collection $\mathcal{S} = \{S_1, S_2, \ldots, S_k\}$ of disjoint dynamic sets. We identify each set by a **representative**, which is some member of the set.
Let x denote an element. We wish to support the following operations:

- MAKE-SET(x): Creates a new set whose only member (and thus representative) is x.
- UNION(x, y): Merge the sets that contain x and y, say S_x and S_y, into a new set $S_x \cup S_y$ that is the union of these two sets.
- IN-SAME-SET(x, y): Return whether x and y belong to the same set.
- FIND-SET(x): Returns a pointer to the representative of the (unique) set containing x.

We are given n items, initially each in its own disjoint set. We make a simplifying assumption: assume elements are labelled $1, 2, \ldots, n$.

11.2 Implementation

To implement of disjoint sets, we represent sets by rooted trees, with each node containing one member and each tree representing one set. In a **disjoint-set forest**, each member points only to its parent. The root of each tree contains the representative and is its own parent.

Heuristics

The first heuristic is **union by rank**.

- For each node x, maintain a **rank**, which is an upper bound on the height of x (the number of edges in the longest simple path between x and a descendant leaf).
- In union by rank, make the root with smaller rank point to the root with larger rank during a UNION operation.

(**Idea:** Always attach the shorter tree under the taller one.)
The second heuristic is **path compression**. We use it during FIND-SET operations to make each node on the find path point directly to the root. (**Idea:** Whenever we find the root of a node, make all nodes along the path point directly to the root. **Effect:** Flattens the tree over time, making future operations faster.)

Pseudocode for disjoint-set forests

When M AKE -S ET creates a singleton set, the single node in the corresponding tree has an initial rank of 0.
MAKE-SET(x)

1: $x.p = x$
2: $x.rank = 0$

Find set: we write this recursively

(1) In the base case, just return the element itself.

(2) Otherwise, head on to your parent to find the root. Once you find it, we now actively update x's parent to also being the root.

(3) Then return it in case anyone else needs it.

FIND-SET(x)

```
1: if x ≠ x.p then
2:     x.p = FIND-SET(x.p)
3: return x.p
```

The UNION operation has two cases, depending on whether the roots of the trees have equal rank.

- If the roots have unequal rank, make the root with higher rank the parent of the root with lower rank, but the ranks themselves remain unchanged.
- If the roots have equal ranks, arbitrarily choose one of the roots as the parent and increment its rank. (Ranks only increase when two equal-rank trees are merged.)

UNION(x, y)

```
1: LINK(FIND-SET(x),FIND-SET(y))
```

LINK(x, y)

```
1: if x.rank > y.rank then
2:     y.p = x
3: else
4:     x.p = y
5:     if x.rank = y.rank then
6:         y.rank = y.rank + 1
```

IS-SAME-SET(x, y)

```
1: return FIND-SET(x) = FIND-SET(y)
```

Complexity

If there are n elements, the combined union-by-rank and path-compression heuristic runs in time $O(\alpha(n))$ amortised for each operation. This means that m disjoint-set operations cost $O(m\alpha(n))$.

We now define a very quickly growing function and its very slowly growing inverse.

Definition 11.1. The **Ackermann function** is defined by

$$\begin{aligned} A(0, n) &= n + 1 \\ A(k + 1, 0) &= A(k, 1) \\ A(k + 1, n + 1) &= A(k, A(k + 1, n)). \end{aligned}$$

Definition 11.2. The **inverse Ackermann Function** is defined by

$$\alpha(n) = \min\{k : A(k, k) \geq n\}.$$

11.3 Connected components

The disjoint-set data structure can be used to determine the connected components of an undirected graph. The procedure CONNECTED-COMPONENTS uses the disjoint-set operations to compute the connected components of a graph. Then the procedure SAME-COMPONENT answers queries about whether two vertices are in the same connected component.

Algorithm 11.3 Connected Components

```
1:  procedure CONNECTED-COMPONENTS(G)
2:      for each vertex v ∈ V do
3:          MAKE-SET(v)
4:      for each edge (u, v) ∈ E do
5:          if FIND-SET(u) ≠ FIND-SET(v) then
6:              UNION(u, v)
7:  procedure SAME-COMPONENT(u, v)
8:      if FIND-SET(u) = FIND-SET(v) then
9:          return true
10:     else return false
```

12 Minimum Spanning Tree

Let $G = (V, E)$ be a connected, weighted undirected graph.

Definition 12.1. A **spanning tree** is a subgraph T of G which is a tree, and contains every vertex in V.

We define the total weight of a spanning tree to be the sum of weights of edges in the spanning tree:

$$w(T) = \sum_{(u,v)\in T} w(u, v).$$

Definition 12.2. A **minimum spanning tree** (MST) of G is a spanning tree of minimum weight.

Lemma 12.3. *Let G a connected, weighted, undirected graph. If all edge weights are distinct, then G has a unique MST.*

Proof. For a contradiction, suppose there are two distinct MSTs T_1 and T_2. Since $T_1 \neq T_2$, they differ at some edge. Let e be the edge in just one of T_1 and T_2 with the smallest weight; WLOG assume e is in T_1 but not T_2.

Let $e = \{u, v\}$. Since T_2 is a MST, T_2 must contain a path from u to v which is not the edge e. Thus, if we add e to T_2, then we get a cycle.

If all the edges in the cycle were in T_1, then T_1 would contain a cycle, which it cannot. Thus, the cycle must contain an edge f not in T_1.

By minimality of e (and the fact that all edge weights are different), we must have $w(f) > w(e)$. Hence, if we replace f by e, we get a spanning tree with smaller total cost. This yields the desired contradiction. □

Generic MST algorithm

Our two algorithms (Kruskal's and Prim's) both use a **greedy strategy**, where on each iteration we add one of the graph's edges to the minimum spanning tree. We do this until we have $n - 1$ edges.

On each iteration, we claim that our current set of edges A is a subset of a minimum spanning tree. The goal is to prove that the new edge is a **safe edge**, meaning we can add it to the edge set and still have it be a subset of some minimum spanning tree. This allows us to grow a MST at each step.

Algorithm 12.4 Generic MST Algorithm

```
1: procedure GENERIC-MST(G)
2:     A ← ∅
3:     while A does not form a spanning tree do
4:         Find a safe edge for A
5:         Add e to A
```

How can we find a safe edge?

Definition 12.5. Let $G = (V, E)$ be a connected and undirected graph. We define:

- A cut $(S, V - S)$ is a partition of V.
- An edge e crosses the cut $(S, V - S)$ if one of its endpoints is in S, and the other is in $V - S$.
- A cut **respects** a set A of edges if no edge in A crosses the cut.
- An edge is a **light edge** crossing a cut if its weight is the minimum of any edge crossing the cut.

Theorem 12.6. *Let $G = (V, E)$ be a connected, undirected weighted graph. Suppose $A \subseteq E$ be such that A is contained in some MST for G. Let $(S, V - S)$ be any cut of G that respects A. Then any light edge crossing the cut $(S, V - S)$ is safe for A.*

It means that we can find a safe edge by 1. first finding a cut that respects A, 2. then finding the light edge crossing that cut. That light edge is a safe edge.

We will use a **cut-and-paste argument** (pay attention since this sort of argument is used a lot).

Proof. Let T be a minimum spanning tree that includes A, and suppose (u, v) is a light edge crossing the cut $(S, V - S)$. If T contains the edge (u, v), then we are done. Thus, assume T does not contain (u, v). Then we construct another minimum spanning tree T' as follows:

Since the minimum spanning tree T must connect all the vertices in the graph, there must exist a path p from u to v that lies entirely in T, and does not include (u, v). We can combine this path with the edge (u, v) to form a cycle.

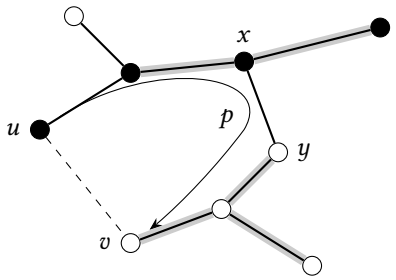


Figure 3: Black vertices are in S, and white vertices are in $V - S$. The edges in the minimum spanning tree T are shown, but the edges in the graph G are not. The edges in A are shaded, and (u, v) is a light edge crossing the cut $(S, V - S)$. The edge (x, y) is an edge on the unique simple path p from u to v in T. To form a minimum spanning tree T' that contains (u, v), remove the edge (x, y) from T and add the edge (u, v).

Since u and v are on opposite sides of the cut, at least one of the edges on the path p must cross the cut; call this edge (x, y). (Observe that (x, y) is not in A, since the cut respects A.)

Now we can replace (x, y) with (u, v) to form a new tree, $T' = T - \{(x, y)\} \cup \{(u, v)\}$. We show that T' is a minimum spanning tree.

Observe that T' is a spanning tree, since there are still $n - 1$ edges, and all vertices are still connected to each other, since any path that would have used the edge (x, y) can use the path through (u, v) instead. Furthermore, T' has smaller total weight than T, because (u, v) is a light edge crossing the cut, and (x, y) also crosses the cut, so $w(u, v) \leq w(x, y)$, and thus,

$$w(T') = w(T) - w(x, y) + w(u, v) \leq w(T).$$

Since T is a minimum spanning tree, T' must also be a minimum spanning tree.

Finally, we show that (u, v) is a safe edge for A. Since $A \subseteq T$ and $(x, y) \notin A$, we have $A \subseteq T'$. Thus, $A \cup \{(u, v)\} \subseteq T'$, and since T' is a minimum spanning tree, (u, v) is safe for A. □

Corollary 12.7. *Consider the connected components in the forest formed by the vertex set and the edges A. Any light edge connecting two components in the forest is a safe edge for A.*

You may visualize our minimum spanning tree algorithms as follows. We slowly build up our spanning tree one edge at a time.

- At the start of our algorithm, our set A has no edges, and the connected components in the graph $G_A = (V, A)$ are just disconnected vertices.
- At any point in the algorithm, all components in this graph are trees (since if they had cycles, they could not be subsets of a spanning tree).
- Any safe edge for A must merge two trees in G_A (because otherwise it would create a cycle).
- Our algorithms must iterate $n - 1$ times, since there must be exactly $n - 1$ edges in a tree.

12.1 Kruskal's algorithm

Algorithm:

(1) Create a forest initially consisting of a separate single-vertex tree for each vertex in the input graph.

(2) Sort graph edges by weight.

Since we need to sort edges in increasing weights, use an edge list sorted based on weights to process them in order.

(3) Greedily add the edge with smallest weight to the forest, provided it does not form a cycle. This combines two trees into a single tree.

We use a UFDS to check whether two vertices are in the same component. If yes, then continue, since adding the edge would create a cycle. If no, then add the edge into T, and union the components containing u and v.

(4) Stop when we have added $V - 1$ edges.

Algorithm 12.8 Kruskal's Algorithm

```
1:  procedure KRUSKAL(G)
2:      F ← ∅
3:      for each v ∈ V do
4:          MAKE-SET(v)
5:      Sort edge set E in ascending order of weight
6:      for every edge (u, v) ∈ E do
7:          if FIND-SET(u) ≠ FIND-SET(v) then        ▷ u and v are not in the same set
8:              F ← F ∪ {(u, v)}                     ▷ Add the edge to T
9:              UNION(u, v)                          ▷ Union the sets containing u and v
10:         if |F| = V − 1 then return F
11:     return FAIL
```

Theorem 12.9. *The runtime of Kruskal's algorithm is $O(E \log V)$.*

Proof.

- We call MAKE-SET once for each vertex, so that costs $O(V)$ in total.
- Sorting the edge set E costs $O(E \log E) = O(E \log V)$.
- For each edge, we perform a constant number of disjoint-set operations. Each operation runs in $O(\alpha(V))$. Thus, the total cost is $O(E\alpha(V))$.

□

Theorem 12.10. *Kruskal's algorithm is correct.*

Proof. It suffices to show that the edge added at every iteration is a safe edge.

When Kruskal's considers an edge (u, v), it checks if u and v are in different connected components. Let S be the set of vertices in the connected component containing u; the other vertices form $V - S$.

Since S is a full connected component in the current forest A, no edge in A currently crosses from S to $V - S$. Thus, the cut respects A.

Kruskal's examines edges in increasing order of weight. If (u, v) is the first edge it finds that connects two different components, it must be the minimum-weight edge crossing that specific cut.

Since (u, v) is the light edge crossing the cut $(S, V - S)$, by Theorem 12.6, it is a safe edge. □

12.2 Prim's algorithm

Algorithm:

(1) Initialize an empty set T to store the MST.

Initialize a priority queue Q to store the edges currently under consideration.

Create a boolean array *in_tree* to track which vertices have been added to the MST.

(2) Choose an arbitrary starting vertex s. Mark it as *in_tree*.

For every neighbor v of s, insert the edge (s, v) into the priority queue Q, using the edge weight $w(s, v)$ as the priority.

(3) While the priority queue is not empty, greedily extract the edge (u, v) with the smallest weight.

(4) If both vertices u and v are already in the tree, then adding (u, v) would form a cycle; skip it and continue to the next smallest edge in Q.

Else, if vertex v is not yet in the tree, add the edge (u, v) to T, and mark v as *in_tree*.

(5) Since v is a new addition to the tree, it brings in new potential edges. Examine all neighbors x of v and insert these new incident edges (v, x) into the priority queue Q.

(6) Repeat until the priority queue is empty. Then T contains the MST.

Algorithm 12.11 Prim's Algorithm

```
1:  procedure PRIM(G)
2:      T ← empty MST
3:      Q ← empty priority queue
4:      in_tree ← array of size |V| set to all false
5:      in_tree[s] ← true
6:      for each neighbour v of s do
7:          INSERT(Q, (w(s, v), (s, v)))
8:      while Q ≠ ∅ do
9:          (u, v) ← EXTRACT-MIN(Q)
10:         if in_tree[u] and in_tree[v] then
11:             continue
12:         if in_tree[u] then
13:             new ← v
14:         else
15:             new ← u
16:         Add (u, v) to T
17:         in_tree[new] ← true
18:         for each neighbour x of new do
19:             if in_tree[x] = false then
20:                 INSERT(Q, (w(new, x), (new, x)))
```

Theorem 12.12. *The runtime of Prim's algorithm is $O(E \log V)$.*

Proof.

- Initialization takes $O(V)$ time.
- In the worst case, every edge is inserted into the priority queue. Each insertion takes $O(\log E)$ time, so total time is $O(E \log E)$.
- Each edge is extracted from the priority queue at most once. Each EXTRACT-MIN operation takes $O(\log E)$ time, so total time is $O(E \log E) = O(E \log V)$.

- To get neighbours of each vertex, iterate over its adjacency list. Across the entire algorithm, this takes $O(E)$.

□

Theorem 12.13. *Prim's algorithm is correct.*

Proof. It suffices to show that the edge added at every iteration is a safe edge.

Let S be the set of vertices already included in the growing tree, and $V - S$ be the vertices not yet reached. The set of edges A already in the tree only connects vertices within S. Therefore, no edge in A crosses the cut $(S, V - S)$, meaning the cut respects A.

At each iteration, Prim's algorithm selects the edge (u, v) with the minimum weight such that $u \in S$ and $v \in V - S$. This is a light edge crossing the cut. By Theorem 12.6, it is a safe edge. □

12.3 Boruvka's algorithm

Algorithm:

(1) Initialize all vertices as individual components.

(2) Initialize MST as empty.

(3) While there are more than one components, do following for each component:

Find the cheapest edge that connects this component to any other component.

Add this edge to MST if not already added, and if adding it does not form a cycle.

(4) Return MST.

Boruvka's algorithm can be shown to take $O(\log V)$ iterations of the outer loop until it terminates, and thus runs in $O(E \log V)$ time.

13 Dynamic Programming

Introduction

Recall the **divide-and-conquer** paradigm:

(1) Partition the problem into particular subproblems.

(2) Solve the subproblems.

(3) Combine the solutions to solve the original one.

In many cases, subproblems are independent. However, if subproblems overlap, i.e., they share the same subsubproblems, then divide-and-conquer becomes inefficient as it redundantly solves the same problems multiple times.

Dynamic programming (DP) solves every subproblem exactly once. It is an optimization method that computes solutions to subproblems and stores them in a table to be **reused** later.

Remark. We trade space for time.

Example. Consider a recursive computation of Fibonacci numbers, defined by

$$F(n) = \begin{cases} F(n-1) + F(n-2) & \text{if } n \geq 2, \\ 1 & \text{if } n = 1, \\ 1 & \text{if } n = 2. \end{cases}$$

Fibonacci numbers (recursive)

```
1: function FIB(n)
2:     if n = 1 then
3:         return 1
4:     else if n = 2 then
5:         return 1
6:     else return FIB(n − 1) + FIB(n − 2)
```

This recursive approach has a runtime of $T(n) = T(n-1) + T(n-2) + O(1)$, which results in an exponential complexity of $O(2^n)$. To optimize this, we use **memoization**:

(1) Initialize a global table (array) of size n to store solutions to subproblems.

(2) Before computing, check if the solution exists in the table.

(3) If it exists, return it immediately. Otherwise, compute the value, store it, and then return it.

Algorithm 13.1 Fibonacci numbers (DP)

```
1: table ← list of n + 1 values, all initialized to undefined
2: function FIBONACCI(x)
3:     if x = 0 then
4:         return 0
5:     else if x = 1 then
6:         return 1
7:     if table[x] ≠ undefined then
8:         return table[x]
9:     else
10:        table[x] = FIBONACCI(x − 1) + FIBONACCI(x − 2)
11:        return table[x]
```

Conceptually, we are creating a **solution space**. By using an array with indices $1, \ldots, n$, each subproblem is computed only once. Since each addition takes $O(1)$ and there are n subproblems, the total complexity is $O(n)$.

13.1 Basic DP with recurrences

The following are two main properties of a problem that suggest that it can be solved using DP:

- **Overlapping subproblems**: Solutions to the same subproblems are needed again and again.
- **Optimal substructure**: The best (optimal) solution of the problem can be built using the best (optimal) solutions of smaller subproblems.

Framework for solving dynamic programming problems:

(1) Create a **recurrence relation** that solves the problem, i.e., define the problem in terms of its subproblems.

(2) Translate said recurrence into an algorithm.

There are two ways to store solutions to subproblems so that these values can be reused:

- **Memoization**: Whenever we need the solution to a subproblem, first look into the lookup table. If the precomputed value is there, then return it; otherwise, calculate the value and store it in the lookup table.
- **Tabulation**: Build a table in a bottom-up fashion and return the last entry from the table.

(3) Analyse translated algorithm. Runtime is generally determined by:

$$\text{Total Complexity} = (\text{Number of states}) \times (\text{Time per state})$$

Template to formalise our approach to DP problems:

- Problem being solved
- Valid subproblems
- Contribution of each subproblem
- Combine the subproblems

13.1.1 Longest increasing subsequence

LeetCode: https://leetcode.com/problems/longest-increasing-subsequence/

Problem: Given an array $A[1 \ldots n]$ of distinct integers, find the length of the longest increasing subsequence.

Let $L(i)$ be the length of the longest increasing subsequence that *ends* at index i.

For each index i, look at all indices $j < i$ where $A[j] < A[i]$. Append $A[i]$ to the longest increasing subsequence ending before i:

$$L(i) = \begin{cases} 1 & \text{if } i = 1 \\ 1 + \max\limits_{\substack{j<i \\ A[j]<A[i]}} L(j) & \text{if } i \geq 1 \end{cases}$$

To find the solution, take the maximum across all possible ending positions: $\max_{1 \leq i \leq n} L(i)$.

Algorithm 13.2 Longest Increasing Subsequence

```
1: table ← array of n values, all initialized to undefined
2: function LIS(A, i)
3:     if i = 1 then
4:         return 1
5:     if table[i] ≠ undefined then
6:         return table[i]
7:     val ← 1
8:     for j = i − 1, . . . , 0 do
9:         if A[j] < A[i] then
10:            val ← max(LIS(A, j) + 1, val)
11:    table[i] ← val
12:    return val
13: LIS(A, n)
```

To compute the i-th state, we check all $j < i$ previous states. With memoization, retrieving each state costs $O(1)$. Thus, the i-th state costs $O(i)$ time to compute. Since i ranges from 1 to n, the total time is $O(n^2)$.

Alternatively, let $L(i)$ be the length of the longest increasing subsequence *starting* at index i. For each index i, look at all *following* indices $j > i$ where $A[j] > A[i]$. Prepend $A[i]$ to the longest increasing subsequence starting after i:

$$L(i) = \begin{cases} 1 & \text{if } i = n \\ 1 + \max\limits_{\substack{j>i \\ A[j]>A[i]}} L(j) & \text{if } i < n \end{cases}$$

Then the solution is $\max_{1 \le i \le n} L(i)$.

There are $O(n)$ subproblems, each taking $O(n)$ time, so the time complexity is $O(n^2)$.

13.1.2 House robber

Problem: Given an array *nums* of non-negative integers, where $nums[i]$ represents the amount of money in house i, find the maximum amount of money you can rob without triggering the alarm by robbing two adjacent houses.

Let $R(i)$ denote the maximum money that can be robbed up to house i. Consider the choice to rob or skip house i.

- If we rob house i, then add its value to the maximum from houses up to $i - 2$, skipping house $i - 1$.
- If we skip house i, then take the maximum from houses up to $i - 1$.

The base cases are when $i = 1$ (rob only the first house) or $i = 2$ (rob the maximum of the first two houses). Hence,

$$R(i) = \begin{cases} nums[1] & \text{if } i = 1 \\ \max(nums[1], nums[2]) & \text{if } i = 2 \\ \max(nums[i] + R(i-2), R(i-1)) & \text{otherwise} \end{cases}$$

13.1.3 Colouring fences

Problem: Given an array $F[1 \ldots n]$ of n arrays and an integer C, $F[i]$ specifies what colour you may colour the i-th fencepost. It is a sorted subset of $[1 \ldots C]$.

You cannot colour two adjacent fenceposts the same colour. Output how many possible ways to colour the n fenceposts.

Let $\text{soln}(i, c)$ represent the number of ways to colour the first i fenceposts, with the last fencepost being coloured c. Then the solution is obtained by summing up $\text{soln}(n, c)$ over all colours $c = 1, \ldots, C$:

$$\sum_{c=1}^{C} \text{soln}(n, c).$$

To evaluate $\text{soln}(i, c)$, first consider whether c is in the set of allowed colours $F[i]$.

- If $c \notin F[i]$, then the value is 0.
- If $c \in F[i]$, since the fencepost i is fixed at c, fencepost $i-1$ must not be coloured c. Thus, the number of colourings is
$$\sum_{c'=1}^{C} \text{soln}(i-1, c')$$
over all $c' \neq c$. The base case is when $i = 1$, so the value is 1.

Hence, the recurrence formula is

$$\text{soln}(i, c) = \begin{cases} 0 & \text{if } x \notin F[i] \\ 1 & \text{if } x \in F[i] \text{ and } i = 1 \\ \sum_{\substack{1 \le c' \le C \\ c' \neq c}} \text{soln}(i-1, c') & \text{if } x \in F[i] \text{ and } i > 1 \end{cases}$$

The solution space has $O(nC)$ states. Computing each subproblem takes $O(C)$. Hence, the runtime is $O(nC^2)$.

13.2 Knapsack problem

13.2.1 0-1 knapsack

Problem: We have n items, where the i-th item has value v_i, weight w_i. Suppose you are given a knapsack capable of holding total weight $W > 0$.
We wish to determine the subset $T \subseteq \{1, \ldots, n\}$ (of items to carry) that maximizes the total value $\sum_{i \in T} v_i$ subject to $\sum_{i \in T} w_i \le W$.

Remark. We cannot take parts of items, it is the whole item or nothing. (This is why it is called 0-1.)

For $1 \le i \le n$ and $0 \le w \le W$, let $K(i, w)$ denote the maximum (combined) value of any subset of items $\{1, \ldots, i\}$ of (combined) weight at most w. Then $K(n, W)$ is the solution to our problem.
To evaluate $K(i, w)$, consider the last item, i.e., item i.

- If $w_i > w$, then we must exclude item i, so the value becomes $K(i-1, w)$.
- If $w_i \le w$, we have two choices:
  - If we exclude item i, then the value becomes $K(i-1, w)$.
  - If we include item i, then the value becomes $v_i + K(i-1, w-w_i)$.

Hence, the recurrence relation is defined as

$$K(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ K(i-1, w) & \text{if } 1 \le i \le n, w < w_i \\ \max(K(i-1, w), v_i + K(i-1, w-w_i)) & \text{if } 1 \le i \le n, w \ge w_i \end{cases}$$

The number of states $K(i, w)$ is $O(n) \times O(W) = O(nW)$. By memoization, each state is computed only once. Computing each state takes $O(1)$. Hence, runtime is $O(nW)$.
In the following implementation, we store each state $K(i, w)$ in the corresponding entry $V[i, w]$ of a table.

Algorithm 13.3 0-1 Knapsack

```
1: Initialize table V[0...n][0...W] with undefined
2: function KNAPSACK(i, w)
3:     if i = 0 then                                   ▷ Base case: no items left
4:         return 0
5:     if V[i, w] is defined then                      ▷ Check memoization table
6:         return V[i, w]
7:     final ← 0
8:     if w_i > w then                                 ▷ Item i is too heavy for current capacity w
9:         final ← KNAPSACK(i − 1, w)
10:    else                                            ▷ Choice between excluding or including item i
11:        final ← max(KNAPSACK(i − 1, w), v_i + KNAPSACK(i − 1, w − w_i))
12:    V[i, w] ← final                                 ▷ Store result in table
13:    return final
```

13.2.2 Unbounded knapsack

Problem: This is a variant of the knapsack problem, where an infinite supply of each item type is available. Given n items, each with weight w_i and value v_i, the goal is to maximize the total value in a knapsack of capacity W.

Unlike the 0-1 version, where each item is used at most once, here any item i can be selected any number of times $k_i \geq 0$, provided the total weight $\sum k_i w_i \leq W$.

Let $K(i, w)$ denote the maximum value achievable using the first i items with a remaining capacity w.

For item i, we must decide how many instances k to include in the knapsack. Note that k is bounded by $kw_i \leq w$, so the range for k is $0 \leq k \leq \lfloor w/w_i \rfloor$.

Choosing k instances of item i adds $k \cdot v_i$ to our total value. After using $k \cdot w_i$ capacity, we move to the $(i-1)$ row to see the best we could have done with the remaining items at the reduced capacity $w - k \cdot w_i$. This is represented by $K(i-1, w - k \cdot w_i)$.

To find the optimal value, we take the maximum over all possible choices of k:

$$K(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ \max\limits_{0 \leq k \leq \lfloor w/w_i \rfloor} (K(i-1, w - kw_i) + kv_i) & \text{otherwise} \end{cases}$$

The solution space consists of $n + 1$ rows (items) and $W + 1$ columns (capacities), so there are $O(nW)$ states.

To compute each state $K(i, w)$, we iterate through all valid values of k. In the worst case ($w_i = 1$), there are w possible choices to check. Since $w \leq W$, the work per state is $O(W)$.

Hence, runtime is $O(nW) \times O(W) = O(nW^2)$.

13.2.3 Bounded knapsack

Problem: Given n items, the i-th item has weight w_i, value v_i, and a limit x_i representing the maximum number of times that item can be selected. The objective is to maximise the total value such that the total weight does not exceed the capacity W, and the count of each item k_i satisfies $0 \leq k_i \leq x_i$.

Let $K(i, w)$ be the maximum value achievable using the first i items with combined weight w.

For item i, we can choose to include k instances, where $0 \leq k \leq x_i$, and subject to the constraint $k \cdot w_i \leq w$. Combining these bounds gives $0 \leq k \leq \min(\lfloor w/w_i \rfloor, x_i)$.

After selecting k instances of item i, we solve the sub-problem for the first $i - 1$ items with capacity $w - (k \cdot w_i)$.

The choice to include k instances contributes $k \cdot v_i$ to the total value.

Finally, take max over all valid integers k within the bound.

Hence, the recurrence relation is defined as

$$K(i,w) = \begin{cases} 0 & \text{if } i = 0 \\ \max\limits_{0 \le k \le \min(\lfloor w/w_i \rfloor, x_i)} (kv_i + K(i-1, w-kw_i)) & \text{otherwise} \end{cases}$$

There are n item types and a maximum capacity W. This results in $O(nW)$ sub-problems to solve in the DP table.
For each item i, we must decide how many copies k to include. Note that $k \le \lfloor w/w_i \rfloor \le w \le W$. Thus, we have $O(W)$ choices.
Multiplying the number of states by the work done per state results in runtime $O(nW^2)$.

13.3 Changing recurrences

The strategy for optimizing DP solutions involves transitioning from a naive recurrence to one that exploits the specific structure of the problem, to reduce the number of choices evaluated at each state.

13.3.1 Unbounded knapsack

To optimize this, we can redefine the recurrence by considering a binary choice for item i.

- If $w < w_i$, we must ignore item i, and consider only the first $i-1$ items, so the value is $K(i,w) = K(i-1,w)$.
- If $w \ge w_i$, we have two choices:
  - Ignore the item, and recurse on the first $i-1$ items with the same capacity w.
  - Take one copy of item i, increase the total value by v_i, and recurse on the *same* i items with remaining capacity $w - w_i$.

Hence, the optimized recurrence relation is

$$K(i,w) = \begin{cases} 0 & i = 0 \\ K(i-1,w) & i \ge 1, w < w_i \\ \max \begin{cases} K(i-1,w), \\ K(i,w-w_i) + v_i \end{cases} & i \ge 1, w \ge w_i \end{cases}$$

Computing each state $K(i,w)$ requires looking up only 2 subproblems, which is $O(1)$ work. Since there are $O(nW)$ total states in the DP table, the overall complexity is $O(nW)$.

Remark. Now each problem only depends on constantly many subproblems.

Remark. One can also do DP bottom-up by filling up the DP table from the first row.

13.3.2 Bounded knapsack

To optimize this, observe that for item i, the state $K(i,w)$ depends only on previous states $K(i-1, w-j\cdot w_i)$, where the weights are all the same mod w_i.
We treat the computation of a row as a sliding window maximum problem, using a max queue to determine the optimal choice of j. We use an Add-To-All operation to account for the $j \cdot v_i$ term as the window slides along the capacity axis.

(1) For each item i, iterate through every possible remainder $k = 0, 1, \ldots, w_i - 1$.

(2) For a fixed remainder k, slide a window across the capacities $k, w_i + k, 2w_i + k, \ldots, W$ and initialize a max queue to keep track of the optimal values within the current chain.

(3) As the capacity multiplier m increases, apply an ADD-TO-ALL operation to the queue; this increments the potential value by v_i for each step forward, representing the addition of one more instance of item i.

(4) Enqueue the value from the previous row $K[i-1]$ at the current capacity and perform a `dequeue` if the queue size exceeds $x_i + 1$. This ensures that we never select more than x_i copies of the current item.

(5) The maximum value in the max queue represents the best combination of a previous state's value and the accumulated value of the selected copies of item i, so assign it to $K(i, w)$.

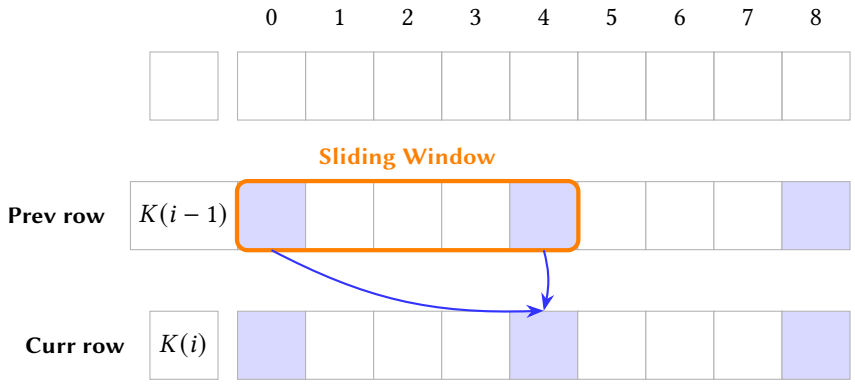


Figure 4: Visualization of remainder chains ($w_i = 4$). The max queue window slides over the previous row, jumping by w_i.

Algorithm 13.4 Bounded Knapsack (Optimized)

```
1:  Initialize K[0...n][0...W] to 0
2:  for i = 1, ..., n do
3:      for k = 0, ..., w_i − 1 do
4:          Q ← empty max queue
5:          m ← 0
6:          while m · w_i + k ≤ W do
7:              ADD-TO-ALL(Q, v_i)
8:              ENQUEUE(Q, K[i − 1, m · w_i + k])
9:              if Q.size > x_i + 1 then
10:                 DEQUEUE(Q)
11:             K[i, m · w_i + k] ← MAX(Q)
12:             m ← m + 1
```

Since each state is enqueued and dequeued at most once per item, the total time complexity is reduced to $O(nW)$.

13.4 More examples

13.4.1 Edit distance (Levenshtein distance)

LeetCode: https://leetcode.com/problems/edit-distance/

Problem: Given two strings S and T, return the minimum number of operations (insertion, deletion, substitution) needed to transform S into T.

Let EDIT-DIST(i, j) represent the minimum number of operations needed to convert the first i characters of S to the first j characters of T. Then the solution is EDIT-DIST(m, n), where $m = S.length$, $n = T.length$.

The base cases are EDIT-DIST$(0, j) = j$, since converting an empty string to the first j characters of T requires j insertions; EDIT-DIST$(i, 0) = i$, since converting the first i characters of S to an empty string requires i deletions.

To evaluate EDIT-DIST(i, j), we check if the current characters $S[i-1]$ and $T[j-1]$ match:

- If $S[i-1] = T[j-1]$, no operation is needed for these characters, so the value is $\text{Edit-Dist}(i-1, j-1)$.
- Else, if $S[i-1] \neq T[j-1]$, we have three choices:
  - Delete character i from S, and try to match the rest of S with T, so $\text{Edit-Dist}(i-1, j)$.
  - Insert character j of T into S, which moves us forward in T but stays at the same position in S, so $\text{Edit-Dist}(i, j-1)$.
  - Replace character i in S with character j in T, so $\text{Edit-Dist}(i-1, j-1)$.

  Take minimum of the above, and add 1.

Hence, the recurrence relation is

$$\text{Edit-Dist}(i,j) = \min \begin{cases} \text{Edit-Dist}(i-1,j)+1, \\ \text{Edit-Dist}(i,j-1)+1, \\ \text{Edit-Dist}(i-1,j-1)+\delta_{i-1,j-1} \end{cases}$$

where $\delta_{i-1,j-1} = 1$ if $S[i-1] \neq T[j-1]$ and 0 otherwise.
There are $O(mn)$ possible subproblems that we need to solve. Each takes $O(1)$ time. Hence, the runtime is $O(mn)$.

13.4.2 Longest common subsequence

LeetCode: https://leetcode.com/problems/longest-common-subsequence/

Problem: Given two arrays A and B, output the longest possible common subsequence C.

This is a little more challenging because now we need to output a solution, not just output a number. We first find the *length* of the longest common subsequence. We can then eventually modify this algorithm to also output the solution itself.

Let $L(i, j)$ represent the length of the longest common subsequence between first i elements of A and first j elements of B. Then the solution is $L(m, n)$, where $m = A.length$, $n = B.length$.

The base cases are $L(0, j) = 0$ and $L(i, 0) = 0$, since when either sequence is empty, the empty sequence is the longest possible common subsequence, which has length 0.

To evaluate $L(i, j)$, compare the current element i from A and element j from B.

- If they match, we extend the previous longest common subsequence by 1, so $L(i-1, j-1) + 1$.
- If they don't match, we take the maximum of either excluding the current element from A or excluding it from B, so $\max(L(i-1, j), L(i, j-1))$.

Hence, the recurrence relation is

$$L(i,j) = \max \begin{cases} L(i-1,j), \\ L(i,j-1), \\ L(i-1,j-1)+1 \quad \text{if } A[i] = B[j] \end{cases}$$

Remark. We only ever add 1 to the solution when we find matching elements, which makes sense!

Observe that if $A[i] = B[j]$, then deferring this match can't potentially lead to a strictly longer common subsequence, so we do not need to try the remaining cases. This gives the recurrence relation

$$L(i,j) = \begin{cases} \max \begin{cases} L(i-1,j) \\ L(i,j-1) \end{cases} & \text{if } A[i] \neq B[j] \\ L(i-1,j-1)+1 & \text{if } A[i] = B[j] \end{cases}$$

13.4.3 Vertex cover (on a tree)

Problem: https://www.spoj.com/problems/PT07X/

Definition 13.5. A **vertex cover** for a graph $G = (V, E)$ is a set $S \subseteq V$ such that for every edge $e = (u, v) \in E$, either $u \in S$ or $v \in S$.

That is, every edge is covered by one of the nodes in the set S.

Minimum vertex cover problem: Given a graph $G = (V, E)$, find a minimum sized set S that is a vertex cover for G.

However, we shall consider a simpler case, when the graph is a rooted tree T, and we only need to find the *size* of the vertex cover.

Let cover(r, in_cover) represent the size of the vertex cover of the subtree rooted at r, where r is in the cover if $in\_cover =$ **true**.

- If r is in the cover, then for all children $c_1, \ldots, c_t$ of r, they can be in or not in the cover, so take the minimum across both cases. Sum them up then + 1.

$$\text{cover}(r, \textbf{true}) = 1 + \sum_{i=1}^{t} \min(\text{cover}(c_i, \textbf{true}), \text{cover}(c_i, \textbf{false})).$$

- If r is not the cover, then for all children $c_1, \ldots, c_t$ of r, we need to cover the edges $(r, c_1), \ldots, (r, c_t)$, so all $c_1, \ldots, c_t$ must be in the cover. Don't plus 1, since we're not including r here.

$$\text{cover}(r, \textbf{false}) = \sum_{i=1}^{t} \text{cover}(c_i, \textbf{true}).$$

Finally, we don't know if the cover contains r or not, so take minimum across both cases.

For the base case, if r is a leaf node, then cover$(r, \textbf{true}) = 1$, cover$(r, \textbf{false}) = 0$.

13.4.4 Longest sawtooth subsequence

Problem: A *sawtooth sequence* is a sequence of numbers $s_1, \ldots, s_m$, where for every $i \leq m-2$, either $s_i < s_{i+1} > s_{i+2}$ or $s_i > s_{i+1} < s_{i+2}$.

We want to design a dynamic program that given an input sequence $x_1, \ldots, x_n$ of distinct numbers, finds the length of the longest sawtooth subsequence of $x_1, \ldots, x_n$. To this end, define:

- $A(i)$ to be the length of the longest sawtooth subsequence in the sequence $x_1, \ldots, x_i$ and whose last pair is ascending, and
- $D(i)$ to be the length of the longest sawtooth subsequence in the sequence $x_1, \ldots, x_i$ and whose last pair is descending.

The recurrence relations to compute $A(i)$ and $D(i)$ are

$$A(i) = \begin{cases} A(i-1) & \text{if } x_i < x_{i-1} \\ 1 + D(i-1) & \text{if } x_i > x_{i-1} \end{cases} \qquad D(i) = \begin{cases} D(i-1) & \text{if } x_i > x_{i-1} \\ 1 + A(i-1) & \text{if } x_i < x_{i-1} \end{cases}$$

The base cases are $A(1) = 1$, $D(1) = 1$. The solution is $\max(A(n), D(n))$.

The runtime is $O(n)$, since there are $O(n)$ states $A(1), \ldots, A(n), D(1), \ldots, D(n)$, and each state takes $O(1)$ time to compute.

A Tutorial Problems

A.1 Tutorial 1

Problem A.1.1. Given a sorted array of $n-1$ unique integers in the range $[1, n]$, find the missing element.

Solution. Check the element in the middle of the array (at index $\lfloor n/2 \rfloor$).

- If the element is $\lfloor n/2 \rfloor$, then all the elements in the left half of the array are present, so the missing element is in the right half. Recurse on the right half.
- Else, the element we are looking at is $\lfloor n/2 \rfloor + 1$, so the missing element is in the left half of the array. Recurse on the left half.

□

Problem A.1.2. You have n piles of homework and the i-th pile has $piles[i]$ pieces of homework. Unfortunately, you have h hours left before all your homework is due. In a moment of panic, you try to figure out the rate k (that is, the "pieces of homework"-per-hour) at which you need to do your homework at in order to finish everything on time.

At every hour, you choose a pile of homework and start clearing pieces of homework from that pile. If the pile has less than k pieces of homework remaining, you decide to just finish that pile itself, and not start on the next pile yet during the same hour.

To maintain your sanity, you want to minimise the number of pieces of homework you do per hour, i.e. k, while still finishing all piles of homework in time.

Find the minimum integer k such that you can finish all your homework within h hours.

Solution. First identify the possible range of values for k.

- The minimum is 1 (we have to do at least 1 piece of homework per hour in order to progress towards our goal of finishing all our homework).
- The maximum is the number of pieces of homework in the largest pile (e.g., if the largest pile contains 5 pieces of homework, any $k \geq 5$ would not make sense as we are trying to minimise k, while being subjected to the constraint of not being able to move on to a new pile within the same hour).

With this range of possible values of k, perform binary search on this range of possible values for k. Check whether the value in the middle of the current range of possible values is **valid**, i.e., you can complete all piles of homework within h hours when working at this rate k:

$$\sum_{i=1}^{n} \left\lceil \frac{piles[i]}{k} \right\rceil \leq h.$$

- If the middle value is valid, recurse on the left half to find a smaller k that is valid.
- If the middle value is invalid, recurse on the right half to find a larger k that is valid.

□

Problem A.1.3. A *convex polygon* is a polygon where all interior angles are less than 180 degrees. A *bounding box* is an axis-aligned rectangle (defined by 4 corner points) such that the entire polygon is contained within the box.

Given an array of n x and y-coordinates of an n-sided convex polygon in clockwise order, find a bounding box around the polygon.

Solution. This problem is equivalent to finding the minimum and maximum x and y-coordinates of the polygon. We first assume no two consecutive points will have the same x or y-coordinate value.

Observation: A convex polygon's x and y-coordinates have at most two changes in direction, i.e., increasing-decreasing-increasing, or decreasing-increasing-decreasing.

If you start at the minimum or maximum point, then there is only a single change in direction, i.e., increasing-decreasing or decreasing-increasing, or have none at all, i.e., simply increasing or decreasing. This question then becomes a problem of **peak-finding**.

We are able to verify which part of the array we are in easily, as long as we take note of

(1) the first element's value,

(2) the middle element's value, and

(3) the direction that the array starts with.

For example, in an array that goes increasing-decreasing-increasing,

- If our binary search lands on a decreasing portion, then we are between the global maximum and minimum.
- If we land on an increasing portion, then we are on the first increasing section if the value of the middle element is greater than that of the first element, and we are on the second increasing section if the value of the middle element is smaller than that of the first element.

A similar argument can be made for the decreasing-increasing-decreasing case.

Knowing this, we perform binary search in the right direction to find a peak/valley. You can then find the other extreme end with another simple binary search.

If two consecutive points have the same x-coordinates, those two points must be the top and bottom points of a fully vertical side, at the leftmost or rightmost side of the polygon. Similarly, same y-coordinates means they are the left and right points of a fully horizontal side at the top or the bottom of the polygon. Hence, any plateau is guaranteed to be a peak, so the binary search won't be stuck at a plateau. □

A.2 Tutorial 2

Problem A.2.1. Analyse the following code snippet and find the best asymptotic bound for the time complexity of the following function with respect to n.

(a)
```
public int badFunction(int n) {
    if (n <= 0) return 2040;
    if (n == 1) return 2040;
    return badFunction(n - 1) + badFunction(n - 2) + 0;
}
```

(b)
```
public String simpleFunction(int n) {
    String s = "";
    for (int i = 0; i < n; i++) {
        s += "?";
    }
    return s;
}
```

Solution.

(a) **Note:** A faulty argument is to say that this program looks like Fibonacci, so it takes $F(n)$ time, where $F(n)$ is the n-th Fibonacci number. This is because the recurrence for the work done is

$$T(n) = T(n-1) + T(n-2) + O(1),$$

which is not the same as $F(n) = F(n-1) + F(n-2)$.

Claim: $T(n) = F(n) - 1$. This implies that the runtime is $O(\phi^n)$.

We prove by strong induction. The base cases are $F(0) = F(1) = 1$, $T(0) = T(1) = 0$. Suppose n is a non-negative integer such that $T(k) = F(k) - 1$ for all $k < n$. Then

$$\begin{aligned} T(n) &= T(n-1) + T(n-2) + 1 \\ &= F(n-1) - 1 + F(n-2) - 1 + 1 \\ &= F(n-1) + F(n-2) - 1 \\ &= F(n) - 1. \end{aligned}$$

(b) Since Java strings are immutable, on each string concatenation, a new copy of the string is created, which takes $O(\text{length of string})$ time. During the i-th iteration of the loop, the string is $i - 1$ characters long. Summing up the individual runtimes,

$$T(n) = \sum_{i=0}^{n-1} i = \frac{n(n-1)}{2} = O(n^2).$$

□

Problem A.2.2. Consider an array of pairs (a, b). Your goal is to sort them by a in ascending order. If there are any ties, break them by sorting b in ascending order.

You are given two sorting functions Merge Sort and Selection Sort. You can use each sort at most once. How would you sort the pairs? Assume you can only sort by one field at a time.

Solution. Use Selection Sort to sort the pairs by b first, then use Merge Sort to sort the pairs by a. This is because Merge Sort is stable and would not affect the ordering of b (which is already sorted) when a has the same value. □

Problem A.2.3.

(a) A sequence of parentheses is said to be balanced as long as every opening parenthesis (is closed by a closing parenthesis). Using a stack, determine whether a string of parentheses is balanced.

(b) A sequence of opening and closing parentheses of three different types {}, (), and [] is given.

We will define a hyperbalanced parenthesis sequence in a recursive way. For the recursion base, we say that empty parentheses sequence is a hyperbalanced one. If X and Y are hyperbalanced sequences, then any of the sequences {X}, [X], (X), and XY is hyperbalanced.

Determine whether a given sequence of parentheses is hyperbalanced.

Solution.

(a) Push opening parentheses (onto the stack, then pop whenever you encounter a closing parenthesis).

The string is unbalanced if there is an attempt at:

- popping an empty stack, or
- if the stack is non-empty after reading the last character.

(b) The idea is similar to (a): Push an opening parenthesis to the stack, keeping track of its type. Whenever we encounter a closing parenthesis, we check whether the stack is not empty and whether the parenthesis at the back of the stack has the same type as the closing one. If these conditions are satisfied, we pop the stack and continue; otherwise, the sequence is not hyperbalanced. Also ensuring that the stack becomes empty after all the operations.

□

Problem A.2.4. A mountain range consists of n hills arranged in a straight line, numbered from 0 to $n-1$. Each hill i has a height of $a[i]$ meters and spans a length of 1 kilometer. As you travel, you want to know how far you can see to your left from each hill.
Standing on top of hill i, you can see j kilometers to the left if and only if all hills from $i-j$ to i are no taller than $a[i]$ meters. If there is no hill $j < i$ taller than hill i, the viewing distance for hill i is considered to be "infinity".
Find a solution to compute the viewing distance for each hill as efficiently as possible.

Solution. Key observation: For a fixed hill i, your viewing distance is blocked by the nearest hill to the left that is strictly taller than $a[i]$.

- If such a hill exists at index k, then the maximum visible distance is $i-k-1$.
- If no such hill exists, then the viewing distance is "infinity".

Use a monotonic decreasing stack. Consider the following algorithm: For each hill i,

(1) Pop from the stack while $a[\text{stack.top}] \leq a[i]$. These hills cannot block the view from hill i or anything to the right of hill i.

(2) If the stack is empty, then there is no taller hill to the left of hill i, so the viewing distance is "infinity".

(3) Otherwise, let $k = \text{stack.top}$ be the index of the nearest taller hill to the left of hill i. The viewing distance is $i-k-1$.

(4) Push i onto the stack.

□

A.3 Tutorial 3

Problem A.3.1. Consider a QuickSort implementation that uses a 3-way partitioning scheme. The specification is as follows:
Suppose we run $\textsc{QuickSort}(l, r)$ on an array A, where l and r are pointers to the left- and right-most indices of the subarray to be sorted. After the 3-way partitioning step using some pivot p, the array is permuted such that there exist indices a and b such that:

- $A[l \ldots a]$ only contains elements $< p$
- $A[a+1 \ldots b]$ only contains elements $= p$
- $A[b+1 \ldots r]$ only contains elements $> p$

Assume this partitioning step takes $O(n)$ time, where $n = r - l + 1$ is the size of the subarray. After partitioning, we recurse on $\textsc{QuickSort}(l, a)$ and $\textsc{QuickSort}(b+1, r)$.

(i) If an input array of size n contains all identical keys, what is the asymptotic bound for QuickSort?

(ii) If an input array of size n contains $k < n$ distinct keys, what is the asymptotic bound for QuickSort?

Solution.

(i) After the first partitioning, all elements are sorted into the middle segment $A[a+1 \ldots b]$, since they are equal to the pivot p. The left and right segments are empty, so there are no recursive calls; thus, the algorithm terminates.

Since we only run partitioning once, the algorithm runs in $O(n)$.

(ii) Since there are only k distinct keys in the array, at most k distinct pivots can be chosen throughout the entire QuickSort run. Each time we partition on a pivot, all elements equal to that pivot are placed in the middle segment and never considered again in future recursive calls. With no further information about the array or pivot selection, we must assume that partitioning passes for recursive calls run in $O(n)$ time.

This is straightforward to see with an example: Suppose we choose the first element as the pivot. Then a possible worst-case input is $[1, 2, \ldots, k-1, k, k, \ldots]$.

This gives us an asymptotic bound of $O(nk)$.

□

Problem A.3.2.

(a) Given an array A, decide if there are any duplicated elements in the array.

(b) Given an array A, output another array B with all the duplicates removed. Note the order of the elements in B does not need to follow the same order in A.

For example, if $A = [3, 2, 1, 3, 2, 1]$, then your algorithm can output $[1, 2, 3]$.

(c) Given arrays A and B, output a new array C containing all the distinct items in both A and B. You are given that A and B already have their duplicates removed.

(d) Given array A and a target value, output two elements $x, y \in A$ where $x + y$ equals the target value.

Solution.

(a) First sort the array. Then traverse the array, check if two adjacent elements are equal.

The overall runtime is $O(n \log n)$.

(b) First sort the array in ascending order, taking $O(n \log n)$ time. Then traverse the array to remove duplicates, by keeping track of the largest element encountered so far.

The overall runtime is $O(n \log n)$.

(c) We can employ the same idea as in MergeSort. Sort both arrays in ascending order, then use the MERGE step from MergeSort to output array C, with the modification: If the element has already been added to array C, discard the element, and advance our pointer in array A or B.

The runtime is $O(n \log n)$.

(d) Sort the array, then use two pointer approach.

The runtime is $O(n \log n)$.

□

Problem A.3.3. Given n plates of chicken rice, your goal is to identify which plate of chicken rice is best. However, you have a very poor taste memory! You can only compare the plates of chicken rice pairwise, and you forget the taste of both plates immediately after comparing them.

(a) A simple algorithm:

- Put the first plate on your table.
- Go through the remaining plates. Take a bite out of the chicken rice on the table and the chicken rice on the new plate to compare them. If the new plate is better than the one on the table, replace the plate on your table with the new plate.
- When you are done, the plate on your table is the winner.

Assume each plate contains n bites of chicken rice in the beginning. When you are done, in the worst-case, how much chicken rice is left on the winning plate?

(b) We want to make sure that there is as much chicken rice left on the winning plate as possible. Design an algorithm to maximize the amount of remaining chicken rice on the winning plate, once you have completed the testing/tasting process.

How much chicken rice is left on the winning plate? How much chicken rice have you had to consume in total?

(c) Now, instead find the median chicken rice. Again, design an algorithm to maximize the amount of remaining chicken rice on the median plate, once you have completed the testing/tasting process.

How much chicken rice is left on the median plate? How much chicken rice have you had to consume in total? (Give a tight asymptotic bound. If your algorithm is randomized, give your answers in expectation.)

Solution.

(a) In the worst case, the first plate is the best plate of chicken rice.

Every comparison thereafter, you keep holding onto the same plate of chicken rice, and compare it to the remaining $n-1$ plates, so you end up taking $n-1$ bites out of the same plate.

(b) Use a tournament tree. Work your way up the tree comparing plates, until you get to the root. That is the best chicken rice, and it only took $\log n$ bites off that plate.

In total for each comparison you will only need to consume 2 bites, and in total you need to make $\leq 2n$ comparisons, so you only need to make at most $O(n)$ bites.

(c) Run quick select. First, notice that in one level of QuickSelect (when the subarray is size n), almost all the plates in the subarray have one bite eaten from them; the unlucky "pivot" plate has $n-1$ bites eaten.

By choosing the pivot at random, the median plate has an expected cost of $(1/n)(n-1) + (1-1/n)1 \leq 2$. So at every level of recursion, there are at most two bites consumed from the median plate, in expectation. Since the recursion terminates in $O(\log n)$ levels, the median plate only has $O(\log n)$ bites consumed. In total, you have consumed $O(n)$ bites of chicken rice.

□

Problem A.3.4 (Child Jumble)**.** After a chaotic birthday party, you must match 20 toddlers to their shoes from a mixed pile. Each child and pair of shoes has a unique size, but sizes are very similar.

You cannot compare shoes to shoes or children's feet to each other. The only allowed operation is to have a child try on a pair of shoes, which tells you if the shoes fit, are too big, or too small.

Design an efficient algorithm to match each child with their correct shoes, and analyze its time complexity in terms of the number of children.

Solution. This is a classic QuickSort problem. Choose any pair of shoes (e.g., red Reeboks), and use it to **partition the kids** into "bigger" and "smaller" groups.

Along the way, you find one kid, say Alex, for whom the red Reeboks fit. Now, use Alex to **partition the shoes** into two piles: those that are too big for Alex, and those that are too small.

Then match the big shoes to the kids with big feet, the small shoes to the kids with small feet, and recurse on the two piles.

The runtime is $O(n \log n)$, where n is the number of children. □

Problem A.3.5 (Integer Sort)**.**

(a) Given an array consisting of only 0's and 1's, what is the most efficient way to sort it?

(b) Consider an array A consisting of elements with keys being integers between 0 and M. What is the most efficient way to sort it?

(c) Consider the following sorting algorithm for sorting integers represented in binary (each specified with the same number of bits):

(1) First, use the in-place algorithm from part (a) to sort by the most significant bit (MSB). That is, use the MSB of each integer as the key to sort with.

(2) Once this is done, you have divided the array into two parts: the first part contains all the integers that begin with 0 and the second part contains all the integers that begin with 1. In other words, all the elements of the (binary) form 0xxxxxxx will come before all the elements of the (binary) form 1xxxxxxx.

(3) Now, sort the two parts using the same algorithm, but using the 2nd bit instead of the 1st. Then, sort each of the resulting parts using the 3rd bit, and so on.

Assuming that each integer is 64 bits, what is the running time of this algorithm? When do you think this sorting algorithm would be faster than QuickSort?

(d) Can you improve on this by using the algorithm from part (b) instead to do the partial sorting? What are the trade-offs involved?

Solution.

(a) We use **Hoare's partition algorithm**. Use a left pointer i and right pointer j. In each iteration, move i forward until you find 1, and move j backward until you find 0. Swap these elements. Repeat until the pointers meet each other.

The runtime is $O(n)$.

(b) Use **Counting Sort**, with runtime $O(n + M)$.

(c) Each level takes $O(n)$ time to be sorted, and we repeat this for each bit of the 64-bit integers, making the recursion at most 64 levels deep. Thus, the time complexity of our algorithm is $O(n)$.

(d) One solution is to divide each integer up into 8 chunks of 8 bits each. Then use the algorithm in (b) to do the sorting, where the values range from 0 to 255.

This takes $O(n)$ time for each partial sort, but now the "recursion" only goes 8 levels deep.

□

A.4 Tutorial 4

Problem A.4.1. Given a data set containing n unique elements, return the largest k elements. Your solution must be better than $O(n \log n)$. Note that:

- You do not have to return the elements in sorted order
- Your solution does not have to be deterministic (wink wink)

Solution. We can use Quickselect to find the k^{th} largest element in expected $O(n)$ time. During this process, the collection gets partitioned and all elements strictly larger than the k^{th} largest element will be on the right of the pivot. To find the k^{th} largest element, we need to Quickselect the element with rank $(n - k)$. These elements together with the pivot are exactly what we want. □

Problem A.4.2 (k-d trees)**.** Let's move to building a type of tree for storing points in k dimensions, called a k-d tree (short for k-dimensional tree). For simplicity, we only deal with 2 dimensions (i.e., $k = 2$). Also, we are going to assume that our n points in 2D space are given to us upfront and will never change.

A k-d tree internal node stores the following data:

- A splitting line, and an indicator on whether it is horizontal or vertical.
- If the splitting line is horizontal, then it holds an "up" subtree reference/pointer and a "down" subtree reference/pointer.

- Otherwise, the splitting line is vertical, then it holds a "left" subtree reference/pointer and a "right" subtree reference/pointer.

A k-d tree leaf node stores simply:

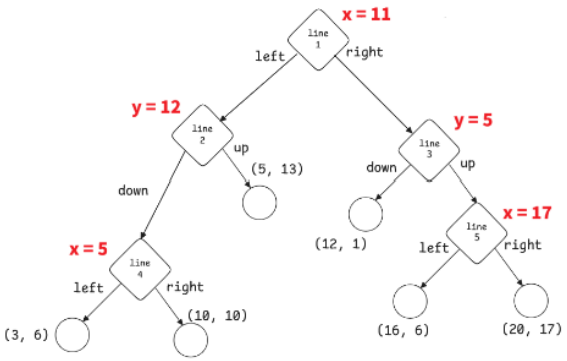
- The coordinate of a single point (x, y).

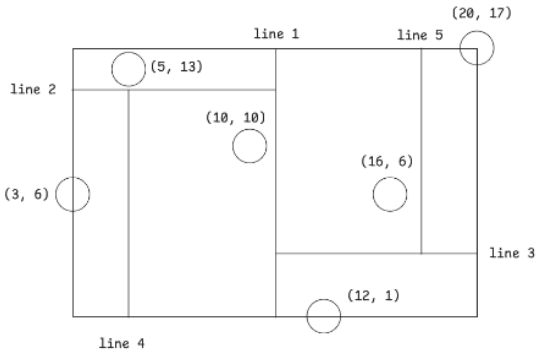
The idea behind a k-d tree is that it needs to support the following operations efficiently:

- `lookup(x, y)`: Should return true if there is indeed a point stored at those coordinates, and false otherwise.
- `nearest-neighbour(x, y)`: Should return the nearest stored point to the query coordinates (x, y).
- `range-query(R)`: Should return the number of points that are within the rectangle R.

To do this, the k-d tree (for the special case of $k = 2$) has the following rough invariants for the internal nodes:

- If the node contains a horizontal splitting line $y = c$, then:
  - All points (x, y) in the "up" subtree must be above the splitting line ($y \geq c$)
  - All points (x, y) in the "down" subtree must be below the splitting line ($y < c$)
- Similarly, if the node contains a vertical splitting line $x = c$, then:
  - All points (x, y) in the "right" subtree must be to the right of the splitting line ($x \geq c$)
  - All points (x, y) in the "left" subtree must be to the left of the splitting line ($x < c$)
- If a node contains a horizontal splitting line, the immediate subtrees should contain a vertical splitting line. Otherwise, the node contains a vertical splitting line, then the immediate subtrees should contain a horizontal splitting line.
- Importantly, at every internal node, the number of leaves in both subtrees should be about the same (or almost half each).





(a) Given an array of n points in 2D space, design an algorithm that builds a balanced k-d tree such that each internal node stores roughly half the points in its left subtree and half the points in its right subtree.

(b) Given a k-d tree of n points and query coordinates (x, y), design an algorithm that reports whether there is indeed a point at (x, y).

(c) Given a k-d tree of n points and a query rectangle R, design an efficient algorithm that counts the number of points within the rectangle R.

(d) Next, let's show that your algorithm runs in $O(\sqrt{n})$ time.

(i) Consider a vertical line L with equation $x = c$. Show that, for a subtree containing n points, the number of regions within this subtree that intersect L satisfies the recurrence:

$$T(n) = 2T\left(\frac{n}{4}\right) + 1.$$

(ii) Show that the recurrence solves to $O(\sqrt{n})$.

Solution.

(a) We first split the points vertically (you can start horizontally too if you want). Quickselect to partition the array based on the median x-coordinate, call it c. Then create the splitting line $x = c$. The array is now partitioned, so recurse to build the left and right subtrees with the left and the right portions of the array respectively.

At the next level we need to split the points horizontally instead. So do the same algorithm but this time finding the median y-coordinate.

Since at each level, given n nodes to build the tree we run Quickselect, this takes $O(n)$ at the current level. We also recurse on around $n/2$ nodes on either side. This makes the recurrence $T(n) = 2T(n/2) + O(n)$, which resolves to $O(n \log n)$.

(b) Let's define the solution recursively. If the current node we are at is a leaf node, compare the coordinates at the node with (x, y). Return true if they are the same, otherwise return false.

Otherwise, compare (x, y) against the splitting line. Suppose the line is horizontal, $y = c$. If $y \geq c$, then recurse on the "up" subtree. Otherwise recurse on the "down" subtree. We can do the same for vertical lines.

The maximum depth of our k-d tree is $O(\log n)$, so our algorithm runs in $O(\log n)$.

(c) Consider the following cases:

(a) If R completely contains the node's region, you should just add its count and return.

(b) If R partially intersects the node's region, you should recurse on both of its children.

(c) If R does not intersect the node's region, you should not recurse on it, and just return.

(d) Let's assume the current node's splitting line is vertical. The analysis is similar if it was horizontal instead.

The node's splitting line $x = c'$ will either be to the left or the right of L. If it's on the left of L, then all the regions in the left child's subtree will not intersect L (since they have x-coordinate $< c'$, and $c' \leq c$). Similarly, if the splitting line is on the right of L, then all the regions in the right child's subtree will not intersect L.

Thus, we only visit at most 1 of the current node's children. Within this child, it will have a horizontal splitting line, and 2 grandchildren. We can't assume anything else, so let's be pessimistic and assume we have to visit both grandchildren. These 2 grandchildren have vertical splitting lines, so we can repeat the same reasoning as above and recurse.

To recap, the current node has 4 grandchildren, but we only visit up to 2 of them. Each grandchild has size $n/4$, so we have our recurrence: $T(n) = 2T(n/4) + 1$.

Let the maximum depth of our recursion tree be d. Each internal node has 2 children, so there are $O(2^d)$ nodes in total. However, each time we recurse, we divide by 4, so $d = O(\log_4 n)$.

$$O(2^d) = O(2^{\log_4 n}) = O(\sqrt{4^{\log_4 n}}) = O(\sqrt{n})$$

□

Problem A.4.3. Given an array of positive 32-bit integers, find 2 numbers such that their XOR is maximum.

Solution. For each integer, insert its binary representation into a **trie**, from the most significant bit to the least significant bit. Then, for each integer x in the array, we want to find an integer y in the trie such that x XOR y is maximised.

To find this y, start from the root of the trie and traverse downwards, constructing the XOR bit by bit. At each position i, we try to maximize the i-th XOR bit. That is, if the i-th bit of x is 0, then move down the 1 child to get a XOR bit of 1; similarly, if the i-th bit of x is 1, move down the 0 child.

(Since we are maximizing our bits along the way, and we start from the most significant bit, we can be sure that our XOR is maximal.)

Do this for all integers x in the array and keep the best XOR value. □

A.5 Tutorial 5

Problem A.5.1 (Coupon Chaos)**.** Mr. Nodle has n coupons of t distinct types ($t \ll n$), given in an array (e.g., $[5, 20, 5, 20, 3, 20, 3, 20]$).

He uses one coupon per day, exhausting all coupons of a lower type before moving to a higher type.

Design an efficient algorithm to output the coupons in ascending order (e.g., $[3, 3, 5, 5, 20, 20, 20, 20]$).

Solution. The naive solution is to sort the input array, which runs in $O(n \log n)$ time.

There are a few possible solutions to this problem, but it boils down to sorting an array with many duplicates.

- **AVL tree:** Modify a height balanced binary search tree to also store the counts of each item.

  Using this, insert all items, then do an in-order traversal, outputting the value for as many times as it was inserted.

  This takes $O(n \log t)$ time, since the size of the tree is at most t, and we need to do n insertion operations.

- **Hash table:** Hash the coupon types and keep track of how much of each type we have seen (frequency). Processing the input takes expected $O(n)$ time.

  Then sort the list of keys, which takes $O(t \log t)$ time.

  In the sorted order, look up the counts for each key, and add the key to the calendar as many times as it was counted based on the hash table. Since there were a total of n items, this takes $O(n)$ time.

  In total, this solution takes $O(n + t \log t)$ time, which is equivalent to $O(n)$ since $t \ll n$.

□

Problem A.5.2 (Average Value Subarray)**.** Given an array of n integers (possibly negative), and a value k, decide if there exists a contiguous sub-array whose average value is k.

Solution. The naive solution is to try every pair of starting and ending points as a subarray, which runs in $O(n^3)$.

First perform a linear shift of every element in the array by k. Then we want to find a subarray with sum 0. Consider precomputing **prefix sums**: for each $i = 1, \ldots, n$, define

$$pref[i] = A[1] + \cdots + A[i].$$

Then the sum of subarray $A[l \ldots r]$ is $A[r] - A[l-1]$, so we check if $pref[r] = pref[l-1]$, i.e., $pref$ contains duplicates (this can be done in $O(n)$ using a hash table). □

Problem A.5.3 (Data Structure 3.0)**.** Let's try to improve upon the kind of data structures we've been using so far a little.

(a) In a height-balanced binary search tree, how would you find the predecessor and successor of a given node in $O(\log n)$ time?

(b) Implement a data structure with the following operations:

- Insert in $O(\log n)$ time
- Delete in $O(\log n)$ time
- Lookup in $O(1)$ time
- Find successor and predecessor in $O(1)$ time

Solution.

(a) To find the successor of a node x:

- If x has a right child, move to the right child once, then keep moving left until you cannot.
- Otherwise, search for x from the root while keeping track of the lowest ancestor whose key is larger than x.

The predecessor is symmetric:

- If x has a left child, move to the left child once, then keep moving right until you cannot.
- Otherwise, search for x from the root while keeping track of the lowest ancestor whose key is smaller than x.

(b) Use both a height-balanced BST (e.g., AVL tree) and a hash table. The hash table here will map each key to the node that it corresponds to.

Modify the BST: for each node of the BST, also store a pointer to its predecessor and successor (call the pointers "previous" and "next").

- **For insertions and deletions:** perform the usual insertion and deletion procedure on the BST, and add/remove the corresponding entry in the hash table, which takes $O(\log n)$ time.

  There is also the added work of maintaining the two additional pointers. After we insert, run the predecessor and successor algorithm from part (a), which takes $O(\log n)$ time. After those are found, we need to: (1) set the predecessor's "next" to the newly inserted node, (2) set the newly inserted node's "previous" to the predecessor and its "next" to the successor, and (3) set the successor's "previous" to the newly inserted node. Deletion works the same way in reverse by splicing the deleted node out of this linked structure.
- **For lookup queries:** use the hash table to find the corresponding node in $O(1)$ time.
- **For successor and predecessor queries:** first perform a lookup query to obtain the node in the BST, and then use either the "next" or "previous" pointer to get the corresponding node in $O(1)$ time.

□

Problem A.5.4 (The Missing Element)**.** Let's revisit the same old problem that we've discussed at the beginning of the semester, finding missing items in the array.

Given n items in no particular order, but this time possibly with duplicates, find the first missing number (if we were to start counting from 1), or output "all present" if all values 1 to n were present in the input.

Solution. Use a **hash set** to store all the values that we've seen in the array and ignore duplicates. Then, go through the hash set to check if each number was present, starting from 1. This takes expected $O(n)$ time.

Note that we could actually just use an array of size n instead to accomplish the same goal (by naively using element value as the array index)! Sometimes it's important to realise when a simple array would suffice and using something more complex such as a hash set can be unnecessary. □

A.6 Tutorial 6

Problem A.6.1 (DFS/BFS)**.** When running DFS on a graph, we can record how we discovered each node. If node u calls node v during DFS, we store $parent[v] = u$. After running DFS, if we treat each parent pointer as an edge, we get a graph.

Explain why this graph is a tree.

Solution. We need to show that the graph is **connected** and **acyclic**.

Every visited node v (except the source) has $parent[v]$ set to the node that discovered it. Following parent pointers from any node traces a path back to the source, so all visited nodes are connected.

A cycle would require some node to be visited more than once. Since DFS marks nodes as visited and never revisits them, this is impossible, therefore acyclic. □

Problem A.6.2 (Is it a tree?)**.** You are given an undirected and connected graph with n nodes and m edges as an adjacency list. (You are given n but not m; assume each adjacency list is given as a linked list, so you do not have access to its size.)

Give an algorithm to determine if this graph is a tree. Recall that a tree is a connected graph with no cycles.

Solution. Any connected, undirected graph containing n nodes and $> n - 1$ edges has a cycle. Thus, simply count edges in the graph, stopping when you find at least n edges.

Notice, though, that since the graph is given as an adjacency list, each edge is represented twice: in the list of both the source and the destination. Thus, if you find $> 2n - 2$ edges in the adjacency list, you know there is a cycle, and the graph is not a tree.

Otherwise, since we are given that the graph is connected, and have verified that it has $n - 1$ edges, it is a tree. □

A.7 Tutorial 7

Problem A.7.1 (Pizza Pirate Encounters)**.** Welcome to Mel's Pizzeria! We're here to deliver pizzas. We are given an undirected graph $G = (V, E)$ that represents the locations that we need to deliver to. The source node s represents Mel's Pizzeria, and a node t represents our destination. We plan on delivering pizzas to places that we pass by on the way to home.

Here's the catch: The city is rife with Pizza Pirates. With every edge e that we take, the pirates will leave us with $0 < f_e \leq 1$ fraction of whatever pizza we were carrying.

(a) Design and analyze an algorithm to determine the path from a given start vertex s to a given target vertex t that maximizes the fraction of pizza left.

We want to run Dijkstra's. Think about which parts of the algorithm we should change in order to make it work.

(b) Are there any other ways of finding the safest path without modifying Dijkstra's algorithm? Can we modify the graph instead?

Solution.

(a) Maintain a distance estimate array d (you can call it fraction estimate if you want) for each node. Initially each node has fraction 0 (nothing), except for node s, which has fraction 1 (entire pizza). Enqueue nodes based on max-priority.

In terms of the relax operation, instead of adding the estimates, we instead multiply them: For a given edge (u, v), if $d[v] < d[u] \times w(u, v)$, then increase the priority of node v to $d[u] \times w(u, v)$.

(b) Note that

$$\text{maximise} \prod_{e \in P} f_e \iff \text{minimise} \prod_{e \in P} \frac{1}{f_e} \iff \text{minimise } \log\left(\prod_{e \in P} \frac{1}{f_e}\right) = \sum_{e \in P} (-\log f_e).$$

Thus, modify all the weights f_e such that they become $-\log(f_e)$ instead. Since all edges are now positively weighted, we can apply Dijkstra's algorithm to solve SSSP.

□

Problem A.7.2 (Informative SSSP)**.** In this problem, we try to find the shortest path in a graph G, even if there are negative cycles.

In particular, we are given a directed, weighted graph $G = (V, E)$ possibly with negative edges, and a source node $s \in V$. Our task is to compute, for each node $u \in V$, the value $dist[u]$ defined as:

$$dist[u] = \begin{cases} \infty & \text{if } u \text{ is reachable from } s \\ -\infty & \text{if } u \text{ is unreachable from } s \text{ with } \geq 1 \text{ negative cycle in between} \\ d(s,u) & \text{otherwise} \end{cases}$$

where $d(s,u)$ is the weight of a shortest path from s to u. We will devise an algorithm that does this in the following sections.

(a) For any node that is either (a) unreachable from s or (b) only reachable from s with 0 negative cycles, how do we calculate $dist[u]$? Disregard all nodes u which are reachable from s with ≥ 1 negative cycle for now.

(b) Nodes that are reachable from s with ≥ 1 negative cycle will have $dist[u] = y < \infty$. How do we find these nodes and set $dist[u] = -\infty$ accordingly (and thus solving the problem)?

Solution.

(a) We should not use Dijkstra's algorithm, as the graph might have negative edges. Instead, we turn to the Bellman-Ford algorithm.

First initialise an array $dist$, where $dist[u]$ represents the shortest distance from s to u seen so far. On initalization, set $dist[u] = \infty$ (for $u \neq s$), and $dist[s] = 0$.

We then sequentially consider all paths going from s to u that use at most 1 edge, 2 edges, up until $n-1$ edges. For each relaxation step, we will iterate over every $u \in V$. For each u, we will skip if $dist[u] = \infty$ (as we have not found a path from s to u yet), and otherwise, look at each neighbour of u, say v, connected by an edge e (in other words, $e = (u, v) \in E$) with some weight w. If $dist[u] + w < dist[v]$, then we have found a shorter path from s to v and thus we update $dist[v] = dist[u] + w$.

On termination, for nodes u that are unreachable from s, $dist[u] = \infty$ as they are never updated. Otherwise, any shortest path from s to u has no more than $n-1$ edges, so we should have found one such path, i.e. $dist[u] = d(s,u)$, as required.

(b) First identify all nodes that are part of a negative cycle, and then perform a multi-source BFS, with these nodes being the sources, to identify nodes that can be reached from some negative cycle in G.

First, initialize a queue for the BFS. After the $n-1$ rounds of relaxations from the previous part, we perform one more round.

For any nodes u for which smaller $dist[u]$ has been found, we add u into the queue and set $dist[u] = -\infty$. We then perform the BFS, and for each of the nodes v that are visited, set $dist[v] = -\infty$.

Once the BFS terminates, we would have set $dist[u] = \infty$ for every u reachable from s with ≥ 1 negative cycle, whereas the remaining entries of $dist$ remain unchanged. We have thus solved the problem.

□

Problem A.7.3 (All Roads Lead to Rome)**.** Bob is currently planning a grad trip to Europe. In order to make the most out of his (short) trip, he wants to find out, from any city in Europe, what is the smallest number of roads he needs to travel through in order to visit Rome (assume that all roads lead to Rome).

(a) He hands you a map of Europe that you quickly processed and represented as an unweighted, directed graph $G = (V, E)$, where Rome is represented as a target node $t \in V$.

For every node $u \in V$, calculate the length of the shortest path between u and t as efficiently as possible.

(b) Bob now realises that some roads are longer than others. He obtained another map which includes the distance of every road, thus making G a weighted, directed graph with non-negative edge weights (everything else remains unchanged from the previous part). With that graph as input, devise an algorithm to help him out.

Solution.

(a) The naive solution is to run BFS from each $u \in V$. Then the time complexity is $O(V(V + E))$.

However, observe that any shortest path from u to t is also a shortest path from t to u, when the direction of every edge is reversed. With this observation, simply **reverse** all the edges in E, and run one BFS starting from t to find the length of a shortest path from t to each u.

Thus, we only need one BFS call, so the total time complexity is $O(V + E)$.

(b) The solution makes use of the same observation as the previous part, but the SSSP algorithm we should use is different as the graph is weighted.

Similar to (a), reverse all the edges in E, and run Dijkstra's algorithm starting from t to find the length of a shortest path from t to each u. The overall time complexity is $O(E \log V)$.

□

A.8 Tutorial 8

Problem A.8.1 (Power plant)**.** Given a network of power plants, houses and potential cables to be laid, along with its associated cost, find the cheapest way to connect every house to at least one power plant. The connections can be routed through other houses if required.

Solution. Run Prim's algorithm with the priority queue initialized to contain all of the power plant nodes. This initial setup ensures that every node discovered later can be traced back to exactly one of the power plant nodes.
You can also run Kruskal's or Boruvka's algorithm with all the power plant nodes initialised to be in the same component.
Alternatively, you can insert an additional super node into the graph that is connected to each of the power plant nodes via edges of weight 0, then run any MST algorithm.
All of these approaches will run in $O(E \log V)$, where E is the number of potential cables and V is the sum of the number of power plants and houses. □

Problem A.8.2 (Lightest bottleneck from a source)**.** You are given a connected undirected graph $G = (V, E)$ with a weight $w(e) > 0$ for each edge e, and a distinguished source vertex s.
For a path P from s to v, define its *bottleneck* to be $\max_{e \in P} w(e)$, namely, the weight of the heaviest edge on P.
For each vertex v, define $heaviest[v]$ to be the minimum possible bottleneck among all paths from s to v:

$$heaviest[v] = \min_{\text{paths } P \text{ from } s \text{ to } v} \max_{e \in P} w(e).$$

Design an algorithm that fills in the array $heaviest$ in $O((V + E) \log V)$ time (or better).

Solution. Compute a minimum spanning tree T of G (Prim, Kruskal, etc.). For an undirected graph, the s-v path in T minimizes the bottleneck (maximum edge weight along the path) among all s-v paths in G.
Run DFS or BFS from s on T only. For each tree edge from parent u to child v with weight w, set

$$heaviest[v] = \max(heaviest[u], w),$$

with $heaviest[s] = 0$. (We only store the largest edge.)

The walk over T costs $O(V)$ (or $O(V)$ edges of T). Together with MST construction, total time is dominated by $O(E \log V)$ or $O(E + V \log V)$ for standard implementations - then $O(V)$ for the walk. □

A.9 Tutorial 9 + 10

Problem A.9.1 (Coin change)**.** YR wants to buy a drink. The stall owner is very fierce. He hates counting coins, so he only accepts exact change with as few coins as possible. If you don't give him exact change, or use too many coins, he will flip out.

YR does not want to be scolded. Luckily, YR has an unlimited supply of coins in n different denominations. Help YR figure out the minimum number of coins needed to make exactly k cents.

Input. You are given an array C of n denominations, and a target value k.

Output. The minimum number of coins to make k cents. If it is impossible, return ∞.

(a) A friend (who surely has not taken CS2040S) suggests greedily picking the highest denomination coin first. Give a counterexample showing why this does not work.

(b) Define solve(m, v) as the minimum number of coins to make v using the first m denominations. Come up with the recurrence and solve this problem.

(c) What is the time complexity of the algorithm?

Solution.

(a) Consider $C = [1, 3, 4]$ and $k = 6$.

If you greedily start from the highest denomination, you will end up with 4 + 1 + 1 which uses 3 coins, instead of 3 + 3 which uses two.

(b) For $m = 0$, we can't use any coins, so solve$(0, 0) = 0$ and solve$(0, v) = \infty$ for all $v > 0$. This will be our base case.

For $m > 0$, consider denomination m, i.e., $C[m]$. We may choose to use it some number of times, up to $\left\lfloor \frac{v}{C[m]} \right\rfloor$ times.

Hence, our final recurrence is:

$$\text{solve}(m, v) = \begin{cases} 0 & \text{if } m = 0 \text{ and } v = 0 \\ \infty & \text{if } m = 0 \text{ and } v > 0 \\ \min\limits_{j=0}^{\left\lfloor \frac{v}{C[m-1]} \right\rfloor} \left(\text{solve}(m-1, v - j \cdot C[m]) + j\right) & \text{if } m > 0 \end{cases}$$

The answer to the original problem is solve(n, k).

(c) There are $O(nk)$ subproblems, and each subproblem takes $O(k)$ time, therefore the time complexity is $O(nk^2)$.

(d) To optimize this, keep the definition of solve(m, v). For $m > 0$, consider the last of the first m denominations, i.e., $C[m]$. We may choose to use it some number of times. In particular, that either means we **don't use** it at all, or **use at least 1** such coin.

- If we don't use coin m at all, then we immediately go to solve$(m - 1, v)$.
- Otherwise, use coin m at least once. Then the target drops by $C[m]$, and we are again given the choice to either keep using it, or stop and move on to the first $m - 1$ denominations.
- Of course, we are not even allowed to take coin m if $v < C[m]$.

Hence, our final recurrence is

$$\text{solve}(m, v) = \begin{cases} 0 & \text{if } m = 0 \text{ and } v = 0 \\ \infty & \text{if } m = 0 \text{ and } v > 0 \\ \text{solve}(m-1, v) & \text{if } m > 0 \text{ and } v < C[m] \\ \min \begin{cases} \text{solve}(m-1, v) \\ \text{solve}(m, v - C[m]) + 1 \end{cases} & \text{if } m > 0 \text{ and } v \geq C[m] \end{cases}$$

The answer to the original problem is still solve(n, k). This is pretty much the same as the unbounded knapsack problem shown in the lectures.

□

Problem A.9.2 (Fancy posts)**.** Congrats! You've gotten an internship at Blinstagram, a company developing a new doomscrolling app where they will show multiple posts/videos per row. You're working in the display algorithms department, where they've given you an important task.

Input. You are given a sequence of n posts, where post i has height h_i pixels and width w_i pixels. You are also given a screen width k. You are promised that for every $1 \leq i \leq n$, we have $0 \leq w_i \leq k$.

To be able to fit as many posts as possible within a certain amount of scrolling, your product manager wants you to implement a new feature. You can place as many posts as you want in a single row, so long as the total width does not exceed k. The height of each row is determined by the tallest post in that row.

Your goal is to minimize the total height of all rows.

(a) Another intern suggested the following idea: "Why not just put as many posts as possible on a single row? This way we minimize the number of rows we need to make and hence the total height."

Prove them wrong by coming up with a sequence of posts and a value k for which their algorithm would not output the correct solution.

(b) Define solve(n) as the minimum height of the first n posts.

What is the base case for the recurrence solve(n)?

(c) Consider solve$(n-1) + h_n$. What solution corresponds to this value (i.e., what kind of arrangement would this be)?

(d) Consider solve$(n-i) + \max_{j=0}^{i-1} h_{n-j}$. What solution corresponds to this value (i.e., what kind of arrangement would this be)?

(e) Consider the recurrence from before. What kind of values of i for which should we consider solutions to? What is the set of i such that solve(n) depends on solve$(n-i)$?

(f) Write out the recurrence relation.

Solution.

(a) Consider the following three posts and $k = 10$:

- Post 1: $h_1 = 1$, $w_1 = 5$
- Post 2: $h_2 = 10$, $w_2 = 5$
- Post 3: $h_3 = 10$, $w_3 = 5$

Their algorithm would put the first two posts on the same row, resulting in two rows of height 10 each. However, the optimal solution would be to put Posts 2 and 3 together, resulting in a total height of 11.

(b) In the case of no posts at all, i.e., $n = 0$, we have solve$(0) = 0$.

(c) This solution represents finding the optimal solution for the first $n-1$ posts, and then placing the last post on its very own row.

(d) This solution represents finding the optimal solution for the first $n-i$ posts, and then placing the last i posts on their very own row.

(e) Considering the constraint of maximum width k, we only consider values i for which $\sum_{j=0}^{i-1} h_{n-j} \leq k$.

(f) The recurrence relation is

$$\text{solve}(n) = \begin{cases} 0 & \text{if } n = 0 \\ \min_{i:\sum_{j=0}^{i-1} h_{n-j} \leq k} \left\{ \text{solve}(n-i) + \max_{j=0}^{i-1} h_{n-j} \right\} & \text{if } n > 0 \end{cases}$$

Now consider the time complexity. There are $O(n)$ subproblems to compute: solve(1), …, solve(n). Within each subproblem, we have a double for loop to compute the optimal solution, which costs $O(n^2)$.

□

Problem A.9.3 (Road trip)**.** Your friend is working at MalaMove, an online platform that delivers Mala to customers, with same day delivery. Recently, their fuel costs have gone through the roof and they need to optimise their fuel consumption.

Input. You are given a sequence of n petrol stations that the van will visit on the way to the destination. Petrol station i is d_i kilometres away from the starting point. Fuel consumption is 1 litre per kilometre. The cost per litre of petrol at petrol station i is c_i.

MalaMove's van can hold L litres of petrol, and is full at the start of the trip. The van can arrive at a station just as it runs out of petrol, and the trip is complete at the final station.

MalaMove does not start from a petrol station; it starts at a depot. You can think of it as petrol station 0. By definition, $d_0 = 0$, and c_0 is undefined.

We guarantee that any two consecutive stations are at most L kilometres away. In other words, it is always possible to reach the final station.

Output. Compute the minimal fuel cost needed to reach the final station.

(a) Let solve(i, p) denote the lowest petrol cost from petrol station i to n, assuming we currently have p litres of petrol in the tank.

What is our final solution?

(b) What is the base case for solve(i, p)?

(c) How much would it cost to drive from station i to station j (where $j > i$), starting with p litres of petrol, and ending with q? Assume that we only topped up our petrol at station j and no other station.

In other words, we start from station i with p litres, drive all the way to station j without buying petrol along the way, and top up petrol at station j (if needed) until we end with q litres.

(d) Let's call the above cost (in the previous subproblem) as cost(i, j, p, q). Consider the scenario that

$$\text{solve}(i, p) = \text{solve}(j, q) + \text{cost}(i, j, p, q).$$

What does that mean?

(e) Let $R(i, p)$ be the furthest reachable station that you can visit from station i with initially p units of petrol (without buying more). What is the recursive case for solve(i, p)?

Solution.

(a) The final solution is solve$(0, L)$, since we start from the depot with an initial amount of petrol L.

(b) The base case is when $i = n$. Since we are already at the destination, $\text{solve}(n, p) = 0$.

(c) It takes $d_j - d_i$ litres to travel from petrol station i to j. Thus, the amount of petrol left is $p - (d_j - d_i)$.

Since we end up with q litres of petrol, we have to pay for the difference, which is $t = q - (p - (d_j - d_i))$.

If $t \geq 0$, then the cost is simply $c_j \cdot t$. However, if $t < 0$, it is impossible to end with q litres (which is less than what we are left with). Then we must define the cost to be ∞ to avoid reaching this state.

(d) It means the optimal way to reach the destination starting from station i with p litres of petrol, is by going to station j then topping up the tank up to q, and then paying the cost of $\text{cost}(i, j, p, q)$ while doing so.

(e) The recurrence relation is

$$\text{solve}(i, p) = \begin{cases} 0 & \text{if } i = n \\ \min\limits_{\substack{j=i+1,\ldots,R(i,p) \\ q=0,1,\ldots,L}} \left\{\text{solve}(j, q) + \text{cost}(i, j, p, q)\right\} & \text{if } i < n \end{cases}$$

From station i, we want to try to reach all reachable stations j, with any possible amount of petrol $0, 1, \ldots, L$.

There are $O(nL)$ subproblems. Within each subproblem, we have to iterate through all possible stations j, and all possible ending amounts q, so this is at most $O(nL)$. This means our algorithm is $O(n^2L^2)$.

(f) To optimize, from $S[i]$, we will visit $S[i + 1]$ next. Then we either keep going or top up some petrol.

Notice that our $\text{cost}(i, i + 1, p, q)$ already accounts for not buying petrol at all. Since we are trying all possible ending amounts q, one of the values would be $q = p - (d_{i+1} - d_i)$, meaning we didn't buy any petrol at $S[i + 1]$ and instead chose to keep traveling.

In fact, we don't need $R(i, p)$ anymore. If we can't even reach $S[i + 1]$ from $S[i]$ with p litres, then we clearly can't reach any further station, so this case should resolve to ∞.

Let $\delta = d_{i+1} - d_i$ be the distance to the next station. Hence, our recurrence now looks like this:

$$\text{solve}(i, p) = \begin{cases} 0 & \text{if } i = n \\ \infty & \text{if } i < n, p < \delta \\ \min\limits_{q=0}^{L} \left\{\text{solve}(i + 1, q) + \text{cost}(i, i + 1, p, q)\right\} & \text{if } i < n, p \geq \delta \end{cases}$$

□

Problem A.9.4 (One last party)**.**

Inspired by: https://leetcode.com/problems/tallest-billboard/description/

Sheldon throws a pizza party for his friends. Unfortunately, Alice and Bob are running late. All that remained were n slices of pizza, each of varying weights. To be fair to each other, they would like to eat the same total weight of pizza.

Each person claims some subset of the remaining slices. Any unclaimed slices are left in the box. Only one person may take each slice (sharing slices is unhygienic, you know?)

Find the maximum total weight Alice and Bob eat, if they must both eat the same amount.

Input. An array of n positive integers, where w_i is the weight of slice i.

Output. The maximum equal total weight both people can eat.

Define $\text{can}(m, a, b)$ to be whether it is possible, considering only the first m slices, for Alice to have eaten a total weight of a and Bob to have eaten a total weight of b.

(a) What is the base case? The recursive case? The answer to the original problem?

(b) Let T be the total weight of all n slices. How many subproblems do we have to compute? How much does each subproblem cost?

By now you should know that this solution is not fast enough. But we're gonna do something different here. Let's change the function entirely and start over.

Define solve(m, d) to be the maximum *combined* total weight that Alice and Bob can eat, considering only the first m slices, and requiring that the difference between the amounts they eat individually must be d.

In other words, if Alice eats a total of a, and Bob eats a total of b, then we require $a - b = d$, and $a + b$ should be maximal. Again, we're only considering the first m slices.

(c) What is the answer to the original problem?

(d) Write the full recurrence. Don't worry about dealing with negative d, we'll fix that shortly. Just write the recurrence as-is for now.

(e) We're almost there, but currently d may be negative, which is not ideal. No worries, all we need to do is to make a "simple" observation.

Briefly justify why solve(m, d) = solve$(m, -d)$ for any possible difference d. Can you now rewrite the recurrence to guarantee d is always non-negative?

(f) Finally, what is the time complexity of our algorithm?

Solution.

(a) The base case is when $m = 0$. Then

$$\text{can}(0, a, b) = \begin{cases} \textbf{true} & \text{if } a = 0 \text{ and } b = 0 \\ \textbf{false} & \text{otherwise} \end{cases}$$

For the recursive case ($m > 0$), consider the last slice (slice m). Then either Alice could've taken it, or Bob could've taken it, or neither of them took it. Hence, the recurrence is

$$\text{can}(m, a, b) = \begin{cases} \textbf{true} & \text{if can}(m-1, a, b) \\ \textbf{true} & \text{if } a \geq w_m \text{ and can}(m-1, a-w_m, b) \\ \textbf{true} & \text{if } b \geq w_m \text{ and can}(m-1, a, b-w_m) \\ \textbf{false} & \text{otherwise} \end{cases}$$

The answer to the original problem is then the largest x such that can(n, x, x) is true.

(b) For any subproblem, we have $0 \leq a \leq T$, $0 \leq b \leq T$, and $0 \leq m \leq n$. So there are $O(nT^2)$ subproblems. It's clear that each costs $O(1)$ time to compute.

(c) It is $\frac{1}{2}$solve$(n, 0)$. Don't forget to divide by 2!

(d) We use the same logic as before. Recall that $d = a - b$, the difference between the amount Alice and Bob ate. Recursive case:

- Neither of them took the last slice. Then d does not change when we move to the remaining slices.
- Alice took the last slice. Then d decreases by w_m, because a has decreased by w_m in the subproblem with the first $m - 1$ slices.
- Bob took the last slice. Then d increases by w_m, because b has decreased by w_m in the subproblem with the first $m - 1$ slices.

$$\text{solve}(m, d) = \begin{cases} 0 & \text{if } m = 0 \text{ and } d = 0 \\ -\infty & \text{if } m = 0 \text{ and } d \neq 0 \\ \max \begin{cases} \text{solve}(m-1, d) \\ \text{solve}(m-1, d-w_m) + w_m \\ \text{solve}(m-1, d+w_m) + w_m \end{cases} & \text{if } m > 0 \end{cases}$$

(e) Consider the optimal solution for the first m slices and a given difference d. Then, if Alice and Bob swapped their slices, the difference $d = a - b$ is now negated.

Thus, any optimal solution for solve(m, d) can be transformed into an optimal solution for solve$(m, -d)$, and vice versa.

So all we have to do now is add an absolute sign, like this:

$$\text{solve}(m, d) = \begin{cases} 0 & \text{if } m = 0 \text{ and } d = 0 \\ -\infty & \text{if } m = 0 \text{ and } d \neq 0 \\ \max \begin{cases} \text{solve}(m-1, d) \\ \text{solve}(m-1, |d - w_m|) + w_m \\ \text{solve}(m-1, d + w_m) + w_m \end{cases} & \text{if } m > 0 \end{cases}$$

(f) For any subproblem, $0 \leq m \leq n$ and $0 \leq d \leq T$, so there are $O(nT)$ subproblems,

Each subproblem costs $O(1)$ time to compute.

Hence, the total time complexity is $O(nT)$.

□